
LMGC90_GUI

Documentation Officielle

Version 0.4.5

© 2026 LMG90_GUI

Interface graphique · Matériaux · Modèles
Avatars · Boucles · Granulométrie
Conditions aux limites · Lois de contact
Maçonnerie · Corps déformables · Calcul
Post-traitement · Variables dynamiques · Architecture

TABLE DES MATIÈRES

1. Introduction à l'interface
2. Assistant de configuration
3. Matériaux
4. Modèles
5. Avatars — Corps rigides
6. Avatar Vide
7. Bibliothèque
8. Boucles paramétriques
9. Granulométrie
10. Conditions aux limites (DOF)
11. Lois de contact
12. Tables de visibilité
13. Assistant de maçonnerie
14. Corps déformables (EF)
15. Calcul (chipy)
16. Post-traitement
17. Variables dynamiques
18. Visualisation
19. Architecture & Guide développeur

Introduction à l'interface graphique de LMGC90_GUI

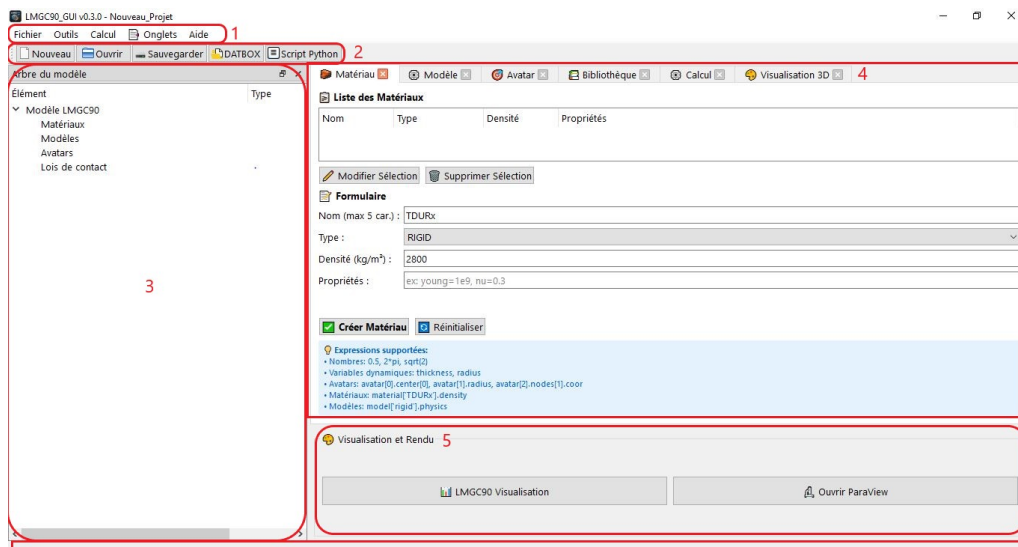
LMGC90_GUI est une interface graphique moderne conçue pour faciliter la création de modèles numériques avec le module **pre** (pré-processeur) de **LMGC90**.

Il est également possible de lancer vos calculs directement depuis LMGC90_GUI via le module **chipy**.

L'interface est organisée de manière claire et ergonomique pour accompagner l'utilisateur du début à la fin du processus de modélisation : création des éléments, conditions aux limites, post-traitement, génération des fichiers et enfin lancement des calculs.

La vidéo suivante présente une vue d'ensemble des différentes parties de l'interface.

Fenêtre principale



Vue globale de l'interface

L'interface est divisée en **Six zones principales** :

1. Menu
2. Barre d'outils (en haut)
3. Arbre du modèle (à gauche)
4. Onglets de création (centre haut)
5. Zone de rendu (centre bas)
6. Barre d'état (en bas)

1. Menu et Barre d'outils

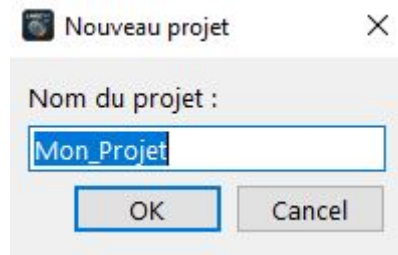
Menu

Fichier

Action	Raccourci	Description
Nouveau	Ctrl+N	Crée un nouveau projet. Une boîte de dialogue s'ouvre pour saisir le nom du projet.
Ouvrir	Ctrl+O	Ouvre un projet existant. Une boîte de dialogue permet de naviguer jusqu'au fichier <code>.json</code> du projet.
Sauvegarder	Ctrl+S	Sauvegarde le projet courant à son emplacement actuel.
Sauvegarder sous...	Ctrl+Shift+S	Sauvegarde le projet sous un nouveau nom ou dans un nouvel emplacement.
Quitter	Ctrl+Q	Ferme l'application.

Nouveau projet

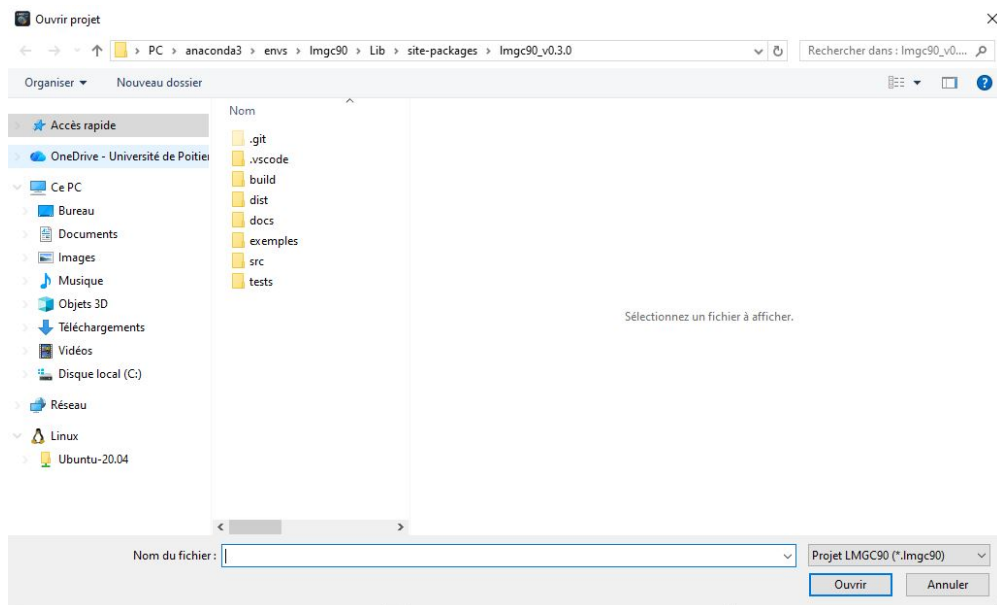
Cliquez sur le bouton **Nouveau** de la barre d'outils, utilisez le menu **Fichier** → **Nouveau**, ou appuyez sur **Ctrl+N**. Une boîte de dialogue s'ouvre pour renseigner le nom du projet



captures/nouveau_projet.JPG

Ouvrir un projet

Cliquez sur le bouton **Ouvrir** de la barre d'outils, utilisez le menu **Fichier** → **Ouvrir**, ou appuyez sur **Ctrl+O**. Il vous suffira ensuite de spécifier le chemin et le nom de votre projet



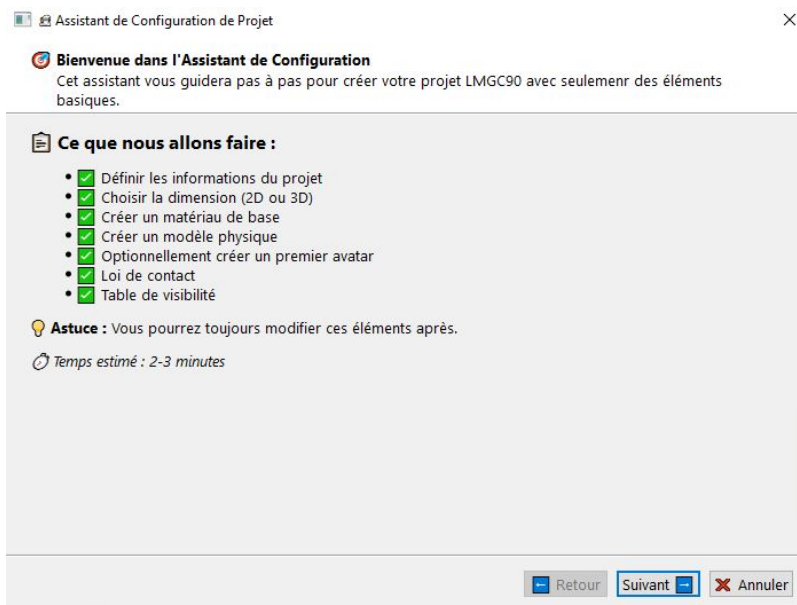
captures/ouvrir_projet.JPG

Sauvegarder Sert à sauvegarder vos projets dans votre disque dur, cliquez sur le bouton **sauvegarder** de la barre d'outils, ou de cliquez sur le menu **Fichier** → **Sauvegarder**, ou avec le raccourci clavier **Ctrl+S**,

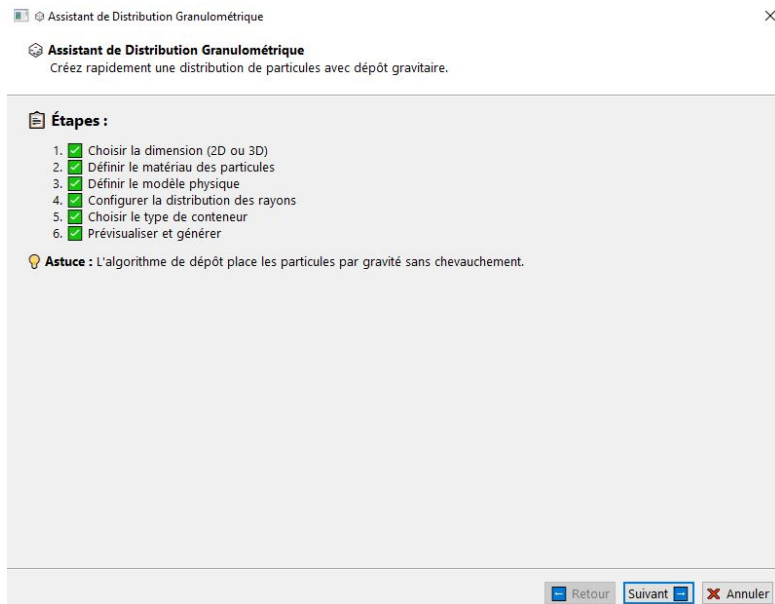
Assistants

Les assistants guident l'utilisateur étape par étape dans les tâches les plus courantes. Ils peuvent être utilisés autant de fois que nécessaire au cours d'un même projet.

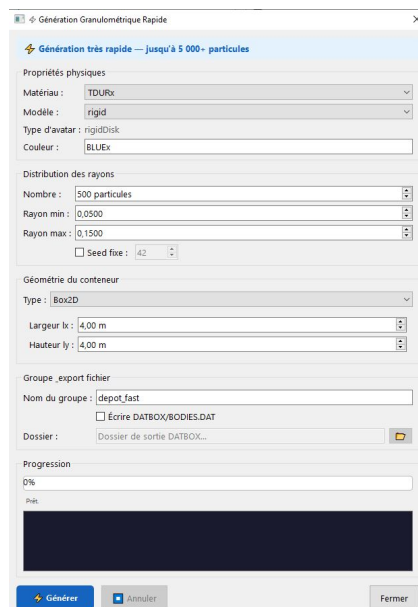
Assistant	Raccourci	Description
Configuration de projet	Ctrl+Shift+N	Guide pas à pas la création et la configuration initiale d'un nouveau projet (matériau, modèle, avatar de référence, loi de contact).
Granulométrie pylmgc90	Ctrl+Shift+G	Génère une distribution granulométrique avec dépôt gravitaire via les routines pylmgc90. Recommandé pour les assemblages de moins de 8 000 avatars. Au-delà, le rafraîchissement de l'interface peut être lent.
Granulométrie numpy _(bêta)_	—	Génère et dépose des avatars à partir de numpy, sans passer par les routines pylmgc90. Recommandé pour les assemblages de plus de 5 000 avatars.
Déformable	Ctrl+Shift+D	Guide la création ou l'importation d'éléments déformables (maillages rectangles, disques, sphères, cylindres ou fichiers externes <code>.msh</code> / <code>.geo</code>).
Maçonnerie	Ctrl+Shift+M	Spécialisé dans la création d'empilements de briques 2D et 3D (<code>brick2D</code> / <code>brick3D</code>) selon différents appareillages (standard, running bond, paneressse simple, paneressse double, etc.).
Factory	Ctrl+Shift+F	Spécialisé dans la Factory d'avatars 2D/3D, les faire apparaître visuellement ou dans le calcul.



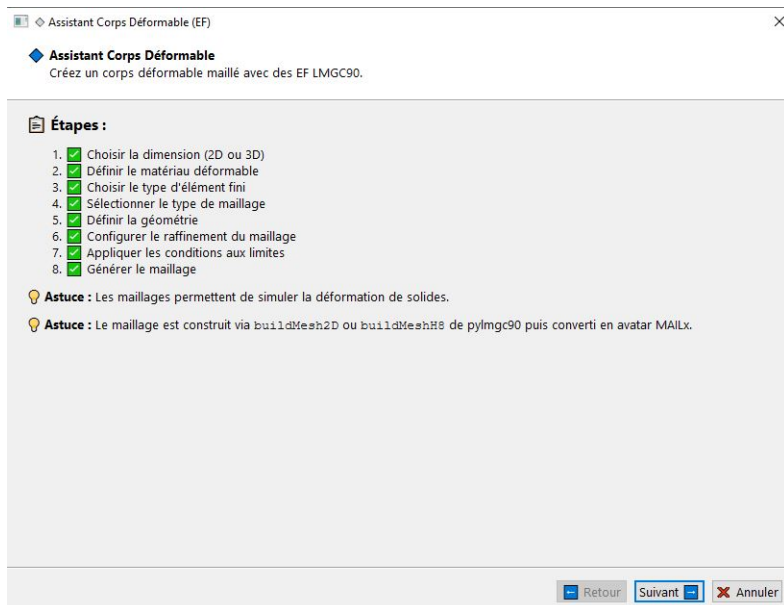
captures/assistant_projet.JPG



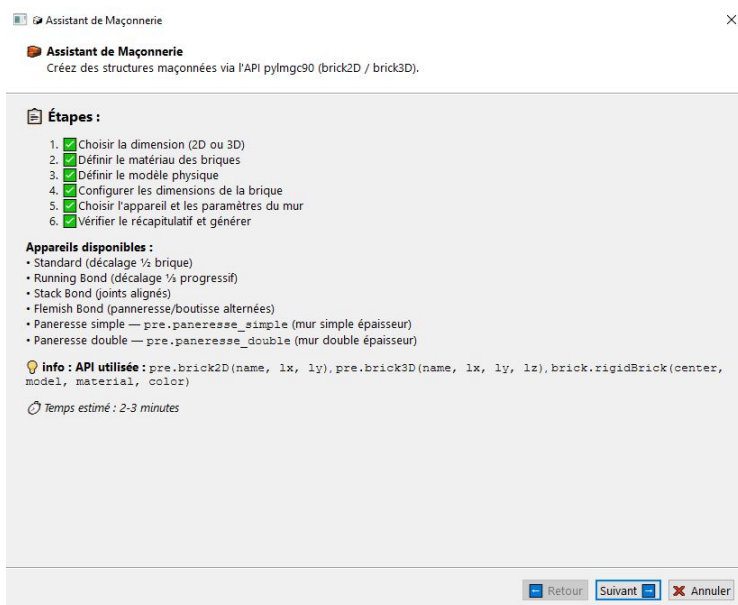
captures/assistant_granulo_pylmgc90.JPG



captures/assistant_granulo_numpy.JPG



captures/assistant_defor_page1.JPG



captures/assistant_maçon_page1.JPG

Remarque : les assistants peuvent être relancés à tout moment pendant la session. Chaque exécution ajoute les éléments générés à la suite du projet existant, sans effacer ce qui a été créé auparavant.

La duplication des éléments qui portent le même nom entraîne des erreurs, pensez à nommer différemment vos éléments.

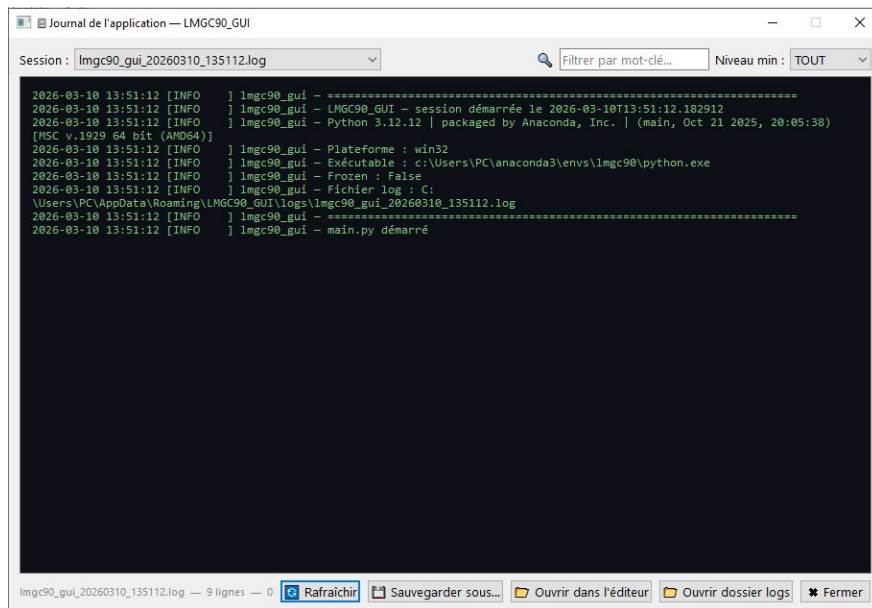
Outils

Action	Raccourci	Description
Générer DATBOX	—	Génère les fichiers .dat utilisés par LMGC90 pour le calcul (DATBOX/).
Générer Script Python	—	Génère le script Python de pré-traitement (pre.py) reproduisant l'intégralité du modèle construit dans l'interface.

Action	Raccourci	Description
Convertir script lmgc90	Ctrl+Shift+C	Une boîte de dialogue qui scanne un script python pylmgc90 et le convertie vers un fichier .lmgc90 version bêta
Variables dynamiques	Ctrl+V	Ouvre une boîte de dialogue permettant de définir des variables réutilisables dans les champs numériques de l'interface (rayon, espacement, offset, etc.). C'est également une fenêtre d'inspection des propriétés des objets LMGC90 présents en mémoire. Voir la page Variables dynamiques.
Préférences	Ctrl+,	Ouvre la boîte de dialogue de configuration de l'application. Voir section Préférences.

Calcul

Action	Raccourci	Description
Paramètres de calcul	Ctrl+F5	Ouvre la boîte de dialogue de configuration des routines chipy : physique, détecteurs de contact, extractions, pilotage, inspection.
Lancer calcul	F5	Lance le calcul chipy directement depuis l'interface, dans un processus séparé pour ne pas bloquer l'interface.
Générer Script Calcul	—	Génère le script Python de calcul (chipy.py) depuis la configuration des routines.
Voir logs LMGC90	F6	Affiche en temps réel les sorties console du calcul en cours.
Journal de l'application	F7	Affiche le journal interne de LMGC90_GUI : erreurs non gérées, avertissements Python, appels pylmgc90 échoués. Utile pour diagnostiquer les problèmes qui ne génèrent pas de message visible dans l'interface.



captures/journal_app.JPG

Onglets

Action	Raccourci	Description
Ouvrir	—	Ouvre un onglet spécifique parmi la liste complète. Voir section Onglets de création.
Fermer les autres	—	Ferme tous les onglets ouverts sauf l'onglet actif.
Fermer tous (sauf essentiels)	—	Ferme tous les onglets non essentiels.
Onglets par défaut	Ctrl+Alt+D	Restaure la disposition d'onglets par défaut.

Aide

Action	Description
À propos	Affiche les informations sur la version de LMGC90_GUI et les dépendances.
Aide en ligne	Ouvre la documentation en ligne dans le navigateur par défaut.

Barre d'outils

La barre d'outils regroupe les actions les plus fréquentes pour un accès rapide :

Bouton	Équivalent menu
Nouveau projet	Fichier → Nouveau
Ouvrir projet	Fichier → Ouvrir
Sauvegarder projet	Fichier → Sauvegarder
Générer Script Python	Outils → Générer Script Python
Générer DATBOX	Outils → Générer DATBOX _(depuis v0.2.6)_

2. Arbre du modèle (à gauche)

Zone fixe affichant l'**arborescence complète du modèle en cours**. Elle se met à jour automatiquement après chaque création, modification ou suppression d'élément.

Sections affichées

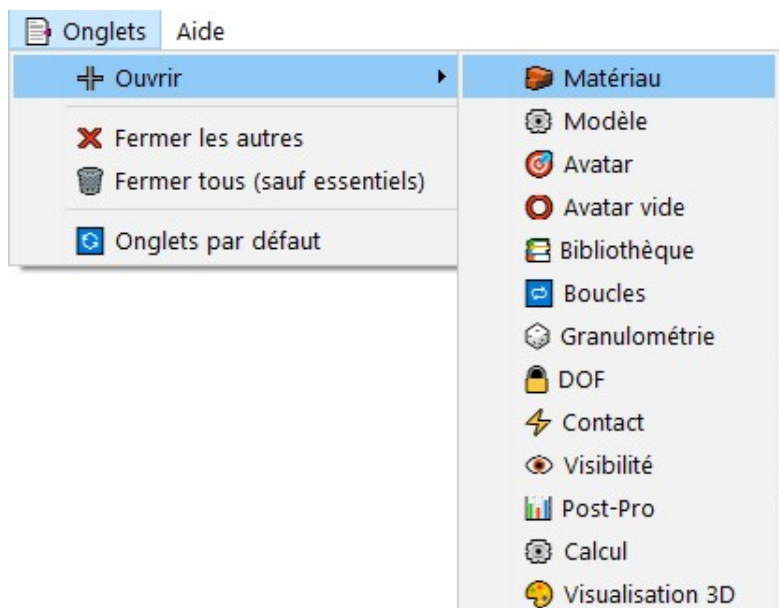
Section	Contenu
Matériaux	Liste de tous les matériaux définis dans le projet.
Modèles	Liste de tous les modèles éléments finis (physique, élément, dimension).
Avatars	Liste de tous les corps du projet : rigides, vides, déformables.
Groupes d'avatars	Groupes créés par les boucles, la granulométrie ou l'assistant de maçonnerie.
Lois de contact	Lois de comportement de contact définies (frottement, cohésion, rigidité).
Tables de visibilité	Règles de visibilité chipy pour les avatars pendant le calcul.
PostPro	Commandes de post-traitement configurées.

Fonctionnalités

- **Clic sur un élément** : ouvre l'onglet correspondant en mode édition et charge l'élément dans le formulaire.
- **Clic droit** : menu contextuel avec les actions Modifier, Supprimer et Informations selon l'élément sélectionné.
- **Vue hiérarchique** : les groupes d'avatars sont développables pour afficher les avatars membres.

3. Onglets de création (zone centrale supérieure)

Zone principale de travail. Chaque onglet est dédié à une étape de modélisation. Pour ouvrir un onglet, utilisez le menu **Onglets** → **Ouvrir** et choisissez l'onglet souhaité.



captures/onglets.jpg

Onglet	Raccourci	Description
Matériau	Ctrl+1	Création et gestion des matériaux (RIGID, ELAS, ELAS_PLAS, THERMO_ELAS, PORO_ELAS, etc.).
Modèle	Ctrl+2	Définition des modèles physiques et éléments finis (MECAX, THERx, POROx, MULTI).
Avatar	Ctrl+3	Création de corps rigides standards : disque, jonc, polygone, mur rugueux, sphère, cylindre, polyèdre, etc.
Avatar vide	Ctrl+4	Création d'avatars à contacteurs personnalisés, ou ajout de contacteurs sur un corps déformable existant.
Bibliothèques	Ctrl+5	Avatars préconfigurés (formes complexes, assemblages courants) prêts à être insérés dans le projet.
Boucles	Ctrl+6	Génération paramétrique de séries d'avatars : cercle, grille, ligne, spirale ou placement manuel.
Granulométrie	Ctrl+7	Génération de dépôts avec distribution statistique des rayons et dépôt gravitaire.
DOF	Ctrl+8	Conditions aux limites : translations imposées, rotations bloquées, vitesses imposées, couplages de degrés de liberté.
Contact	Ctrl+9	Définition des lois de comportement de contact (frottement Coulomb, cohésion, rigidité normale et tangentielle).
Visibilité	—	Création de tables de visibilité chipy pour afficher ou masquer des avatars pendant le calcul.
Postpro	—	Configuration des commandes de post-traitement : bilan énergétique, suivi de corps, extraction de champs.
Visualisation 3D	—	Affichage interactif des avatars du modèle avec modes de navigation, sélection et mesure.

Raccourcis clavier : les touches Ctrl+1 à Ctrl+9 ouvrent directement les neuf premiers onglets de la liste.

4. Zone de rendu (zone centrale inférieure)

Zone dédiée à la visualisation et aux sorties du calcul.

Boutons disponibles

Bouton	Description
LMGC90 Visualisation	Lance la visualisation intégrée pylmgc90 via <code>pre.visuAvatars()</code> . Ouvre une fenêtre externe indépendante.
ParaView	Ouvre automatiquement les fichiers de sortie du calcul dans ParaView (par défaut <code>rigids.pvd</code>). Nécessite que ParaView soit installé sur la machine et qu'un calcul ait déjà été effectué.

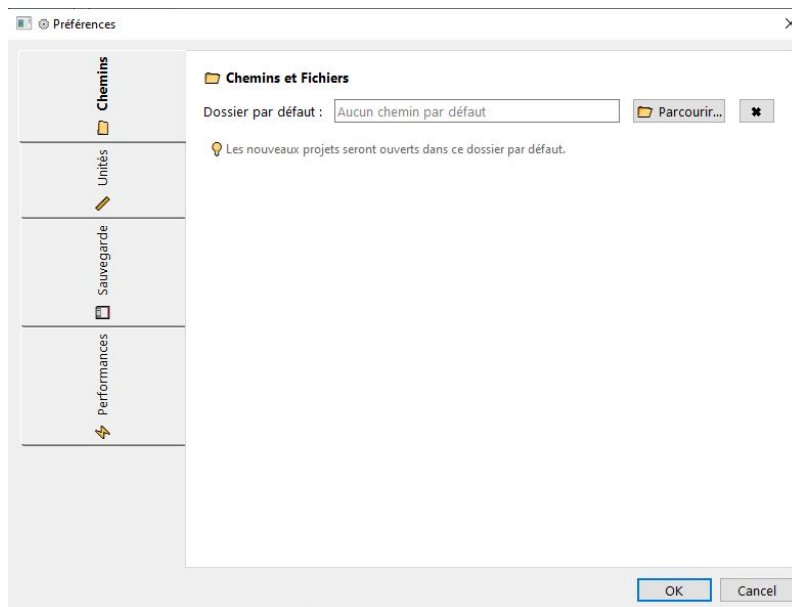
Modes interactifs du viewer 3D

Mode	Description
■ ■ ■ Navigation	Mode par défaut : rotation (clic gauche + glisser), zoom (molette), panoramique (clic droit + glisser).
■ Sélection	Clic sur un avatar pour le mettre en évidence (surlignage jaune) et afficher ses informations dans la barre d'état.
■ Règle	Mesure de distance : cliquer sur un premier point (A) puis un second point (B) pour afficher la distance en mètres.

Les vues rapides **XY**, **XZ**, **YZ** et **Iso** sont accessibles depuis la barre d'outils du viewer.

5. Préférences

Accessible via **Outils** → **Préférences** ou le raccourci **Ctrl+,**. La boîte de dialogue de préférences regroupe les paramètres de configuration de l'application.

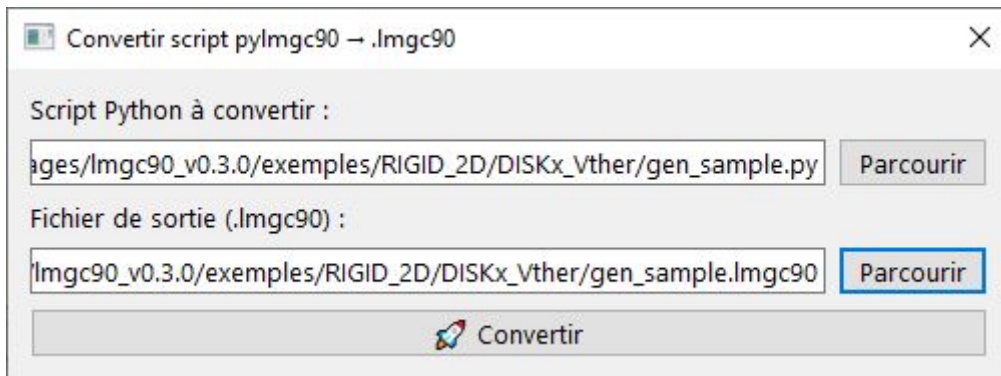


captures/preferences.JPG

Paramètre	Description
Dossier des projets	Chemin par défaut utilisé lors de l'ouverture et de la sauvegarde des projets. Cliquer sur Parcourir pour le modifier.
Système d'unités	Choix entre SI (mètre, kilogramme, seconde) et CGS (centimètre, gramme, seconde). _(Non encore implémenté dans cette version.)_
Sauvegarde automatique	Options pour activer la sauvegarde automatique à intervalles réguliers et à la fermeture de l'application.
Historique des projets récents	Nombre maximum de projets conservés dans la liste Fichier → Projets récents .
Affichage des avatars	Active ou désactive l'affichage des avatars dans l'arbre du modèle et dans le tableau de l'onglet Avatar. Désactiver cette option améliore les performances sur les projets comportant un grand nombre d'avatars.

Convertir

LMGC90_GUI inclut un convertisseur bêta (script python) vers un .lmgc (JSON). Il vous suffit seulement de donner le chemin de votre script puis le nom de votre fichier et de cliquer sur le bouton "Convertir".



captures/convert.JPG

Remarque : traduit selon la complexité de votre script.

Résumé des raccourcis clavier

Raccourci	Action
Ctrl+N	Nouveau projet
Ctrl+O	Ouvrir un projet
Ctrl+S	Sauvegarder
Ctrl+Shift+S	Sauvegarder sous...
Ctrl+Q	Quitter
Ctrl+Shift+N	Assistant de configuration de projet
Ctrl+Shift+G	Assistant de granulométrie pylmgc90
Ctrl+Shift+D	Assistant de déformable
Ctrl+Shift+M	Assistant de maçonnerie
Ctrl+V	Variables dynamiques
Ctrl+,	Préférences
Ctrl+F5	Paramètres de calcul
F5	Lancer le calcul
F6	Voir les logs LMGC90
F7	Journal de l'application
Ctrl+Alt+D	Onglets par défaut
Ctrl+1 ... Ctrl+9	Ouvrir l'onglet correspondant

6. Barre d'état

Bande horizontale en bas de la fenêtre affichant des messages contextuels sur les opérations en cours : création d'un avatar, génération d'un script, résultat d'une mesure dans le viewer 3D, erreur de validation, etc.

LMGC90_GUI est conçue pour être **intuitive** et **entièrement visuelle**, tout en conservant une compatibilité totale avec les scripts Python traditionnels de LMG90.

Assistant de configuration de projet

L'**Assistant de configuration de projet** guide pas à pas la création d'un projet LMGC90 complet avec tous ses éléments essentiels : matériau, modèle, avatar de test, loi de contact et table de visibilité.

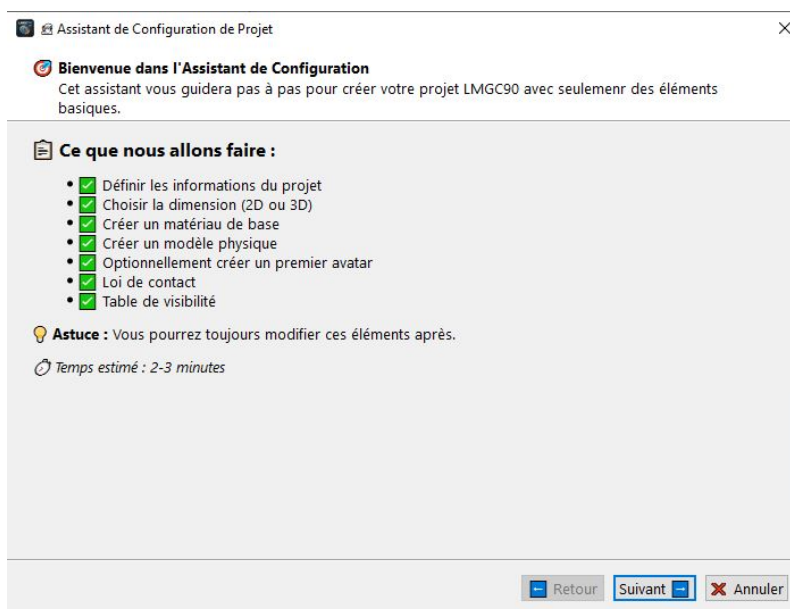
Il est particulièrement utile pour démarrer rapidement un nouveau projet sans avoir à configurer chaque onglet séparément.

■ ■ **Temps estimé : 2 à 3 minutes**

Lancer l'assistant

Trois façons d'ouvrir l'assistant :

Méthode	Action
Menu	Fichier → Assistant de projet... ou Assistants → Configuration de projet
Barre d'outils	Bouton dédié (selon la configuration de la barre)
Raccourci clavier	Ctrl+Shift+N



Ouverture de l'assistant

Annulation possible à tout moment : cliquer sur ■ **Annuler** à n'importe quelle étape ferme l'assistant sans modifier le projet. L'état du projet (nom, dimension, chemin) est entièrement restauré.

Vue d'ensemble des étapes

L'assistant est composé de **9 pages** parcourues séquentiellement. Les boutons **Retour** et **Suivant** permettent de naviguer librement entre les pages.

Étape	Page	Description	Obligatoire
0	Introduction	Présentation de l'assistant	—
1	Informations du projet	Nom et description	■ Oui (nom requis)
2	Dimension	2D ou 3D	■ Oui
3	Matériau	Créer ou réutiliser un matériau	■ Oui
4	Modèle	Créer ou réutiliser un modèle physique	■ Oui
5	Avatar	Créer un premier avatar de test	■ Optionnel
6	Loi de contact	Définir le comportement de contact	■ Optionnel
7	Table de visibilité	Définir qui interagit avec qui	■ Optionnel (requiert étapes 5 et 6)
8	Récapitulatif	Vérification avant création	—

Page 0 — Introduction

Page d'accueil présentant les étapes à venir. Aucune saisie requise.

Cliquer sur **Suivant** pour commencer.

Page 1 — Informations du projet

Assistant de Configuration de Projet

Informations du Projet

Définissez les informations de base de votre projet.

Nom du projet :

modèle_20avatars_disks

Description :

Description optionnelle du projet...

Retour

Suivant

Annuler

Page nom du projet

Champ	Description	Contraintes
Nom du projet	Identifiant du projet. Utilisé comme nom de fichier lors de la sauvegarde.	Requis — 50 caractères maximum. Le bouton Suivant est désactivé tant que ce champ est vide.
Description	Texte libre décrivant l'objectif ou le contexte du projet.	Optionnel

Nom requis : le champ Nom est marqué d'un astérisque (*) dans l'assistant — il s'agit d'un champ obligatoire. Le bouton Suivant ➡ reste grisé tant qu'il est vide.

Page 2 — Dimension du problème

Assistant de Configuration de Projet

Dimension du Problème
Choisissez si votre simulation sera en 2D ou 3D.

Dimension spatiale

☒ 2D - Problème bidimensionnel
Exemples : compression biaxiale, essai œdométrique, écoulement granulaire 2D, tambour rotatif 2D

☐ 3D - Problème tridimensionnel
Exemples : compression triaxiale, trémie 3D, tambour cylindrique, mélangeur 3D

Retour Suivant Annuler

Page dimension

Choisir entre deux options exclusives (boutons radio) :

Choix	Code interne	Exemples d'usage
2D — Problème bidimensionnel	<code>dimension = 2</code>	Compression biaxiale, essai œdométrique, écoulement granulaire 2D, tambour rotatif 2D
3D — Problème tridimensionnel	<code>dimension = 3</code>	Compression triaxiale, trémie 3D, tambour cylindrique, mélangeur 3D

La valeur **2D** est sélectionnée par défaut.

Effet sur les étapes suivantes : la dimension choisie ici conditionne automatiquement : - La liste d'éléments proposés à l'étape Modèle (`Rxx2D` ou `Rxx3D`) - Le type d'avatar proposé à l'étape Avatar (`rigidDisk` ou `rigidSphere`) - Le type de corps et de contacteur dans la table de visibilité (`RBDY2/DISKx` ou `RBDY3/SPHER`)

Page 3 — Matériau

Assistant de Configuration de Projet

Matériau

Créez un matériau de base pour vos particules.

☒ Créer un matériau maintenant

Paramètres du matériau

Nom (max 5 car.) : rockx

Type : RIGID

Densité : 2500,00 kg/m³

Conseil : Pour des simulations granulaires simples, utilisez RIGID avec une densité typique de 2500 kg/m³ (sable/gravier).

Retour Suivant Annuler

Page matériau

Cette page propose deux modes selon l'état du projet :

Mode A — Utiliser un matériau existant (si le projet contient déjà des matériaux)

Une liste déroulante affiche tous les matériaux déjà définis dans l'onglet Matériau. Sélectionner l'un d'eux pour l'associer au projet sans en créer un nouveau.

Mode B — Créer un nouveau matériau (coché automatiquement si aucun matériau n'existe)

Cocher **Créer un nouveau matériau à la place** pour afficher le formulaire de création :

Champ	Description	Valeur par défaut
Nom	Identifiant du matériau. 5 caractères maximum.	rockx
Type	Type de comportement mécanique.	RIGID
Densité	Masse volumique (kg/m³).	2500 kg/m³

Conseil de l'assistant : pour des simulations granulaires simples, utilisez le type **RIGID** avec une densité de 2 500 kg/m³ (sable/gravier typique). Les propriétés élastiques ne sont pas configurables dans l'assistant — utiliser l'onglet Matériau pour les types **ELAS**, **ELAS_PLAS**, etc.

Page 4 — Modèle physique

Assistant de Configuration de Projet

Modèle Physique

Créez un modèle de comportement mécanique.

☒ Créer un modèle maintenant

Paramètres du modèle

Nom (max 5 car.) :

rigid

Physique :

MECAx

Élément :

Rxx2D

Conseil : Pour des corps rigides, utilisez Rxx2D (2D) ou Rxx3D (3D).

Retour

Suivant

Annuler

Page modèle

Même logique que la page Matériau : réutiliser un modèle existant ou en créer un nouveau.

Mode A — Utiliser un modèle existant

Liste déroulante des modèles déjà définis dans l'onglet Modèle.

Mode B — Créer un nouveau modèle

Champ	Description	Valeur par défaut
Nom	Identifiant du modèle. 5 caractères maximum.	rigid
Physique	Famille de physique.	MECAx (seule option dans l'assistant)
Élément	Type d'élément fini. Adapté automatiquement à la dimension.	Rxx2D (2D) ou Rxx3D (3D)

Conseil de l'assistant : pour des corps rigides (DEM), utilisez Rxx2D en 2D ou Rxx3D en 3D. Ces éléments n'ont aucune option numérique et sont les plus simples à configurer. Pour des modèles éléments finis déformables, créer le modèle directement dans l'onglet Modèle après la fin de l'assistant.

Page 5 — Premier avatar _(optionnel)_

The screenshot shows a software window titled "Assistant de Configuration de Projet" with a close button (X) in the top right corner. Inside, there's a section titled "Premier Avatar (Optionnel)" with the instruction "Créez optionnellement un premier avatar de test." Below this is a checkbox labeled "Créer un avatar de test" which is checked. Underneath the checkbox is a sub-section titled "Paramètres de l'avatar" containing three input fields: "Type" with a dropdown menu showing "rigidDisk", "Rayon" with a text input showing "0,10 m", and "Couleur" with a text input showing "BLUEx". At the bottom right of the dialog are three buttons: "Retour" (disabled), "Suivant" (active), and "Annuler" (disabled).

Page avatar

Cette page est **optionnelle**. Elle permet de créer un avatar de test positionné à l'origine (centre = $[0, 0]$ en 2D ou $[0, 0, 0]$ en 3D).

Cocher **Créer un avatar de test** pour afficher le formulaire :

Champ	Description	Valeur par défaut
Type	Type d'avatar. Adapté automatiquement à la dimension.	rigidDisk (2D) ou rigidSphere (3D)
Rayon	Rayon du disque ou de la sphère (m).	0.1 m
Couleur	Code couleur LMGC90 à 5 caractères.	BLUEx

Dépendance : l'avatar utilise automatiquement le matériau et le modèle définis aux étapes précédentes. Si aucun matériau ou modèle valide n'est disponible, l'avatar ne sera pas créé même si la case est cochée.

Position : l'avatar est créé à l'origine du repère. Modifier sa position après création via l'onglet Avatar.

Page 6 — Loi de contact (optionnelle)

Assistant de Configuration de Projet

Loi de Contact
Définissez comment les particules interagissent.

☐ Créer une loi de contact

Paramètres de la loi

Nom :

Type de loi :

Coefficient de friction :

IQS_CLB : Loi de contact avec friction de Coulomb
Friction typique : 0.3 (sable), 0.5 (gravier), 0.1 (surfaces lisses)

Page loi de contact

Définit le comportement mécanique lors des contacts entre avatars. Cocher **Créer une loi de contact** (coché par défaut) pour afficher le formulaire :

Champ	Description	Valeur par défaut
Nom	Identifiant de la loi. Jusqu'à 20 caractères.	iqsc0
Type de loi	Type de comportement de contact. Voir tableau ci-dessous.	IQS_CLB
Coefficient de friction	Visible uniquement pour les lois avec frottement de Coulomb.	0.3

Types de lois disponibles

Type	Nom complet	Friction	Description
IQS_CLB	Contact Unilatéral Quasi-Statique + Coulomb	■ Oui	Loi standard rigide non-lissée. La plus courante pour les simulations DEM.
IQS_CLB_G0	IQS_CLB avec gap nul	■ Oui	Variante avec gap initial nul.
COUPLED_DOF	Degrés de liberté couplés	■ Non	Couplage cinématique entre corps.
IQS_DS_CLB	Contact Discret Rigide + Coulomb	■ Non	Loi discrète avec rigidités normales et tangentielles.
IQS_MOHR_DS_CLB	Mohr-Coulomb Discret	■ Non	Critère de Mohr-Coulomb pour les joints ou interfaces fragiles.
IQS_MAC_CZM	Zone Cohésive	■ Non	Loi de zone cohésive pour la fissuration.
ELASTIC_WIRE	Câble élastique	■ Non	Liaison unilatérale en traction (câble).
BRITTLE_ELASTIC_WIRE	Câble élastique fragile	■ Non	Câble avec rupture fragile au-delà d'un seuil.
ELASTIC_ROD	Barre élastique	■ Non	Liaison bilatérale en traction et compression (barre).

Type	Nom complet	Friction	Description
ELASTIC_REPELL_CLB	Répulsion élastique + Coulomb	■ Non	Contact répulsif avec frottement.

Valeurs de friction typiques :

Matériau	Coefficient de friction
Surfaces lisses	0.1
Sable fin	0.3
Gravier	0.5
Béton rugueux	0.6–0.8

La friction n'est configurable que pour les lois IQS_CLB et IQS_CLB_G0. Pour les autres types, le champ est masqué automatiquement.

Page 7 — Table de visibilité _(optionnelle)_

Assistant de Configuration de Projet

Table de Visibilité
Définissez quels avatars peuvent interagir entre eux.

☐ Créer une table de visibilité

💡 La table de visibilité définit quels contacteurs peuvent se voir et avec quelle loi. Par défaut, tous les avatars de même couleur interagissent entre eux.

Configuration de visibilité

Couleur candidat : BLUEx

Couleur antagoniste : BLUEx

Distance d'alerte : 0,10 m

Configuration automatique :

- Corps : RBDY2 (2D) ou RBDY3 (3D)
- Contacteur : DISKx (2D) ou SPHER (3D)
- Loi : celle créée à l'étape précédente

Retour Suivant Annuler

Page visibilité

La table de visibilité définit **quels contacteurs peuvent se détecter mutuellement** et avec quelle loi de contact. Cocher **Créer une table de visibilité** (coché par défaut) pour afficher le formulaire :

Champ	Description	Valeur par défaut
Couleur candidat	Couleur des contacteurs candidats (corps actif).	BLUEx (synchronisé avec la couleur de l'avatar si l'étape 5 est active)
Couleur antagoniste	Couleur des contacteurs antagonistes (corps passif).	BLUEx
Distance d'alerte	Distance maximale de détection de contact (m). Au-delà, deux corps ne sont pas considérés en contact potentiel.	0.1 m

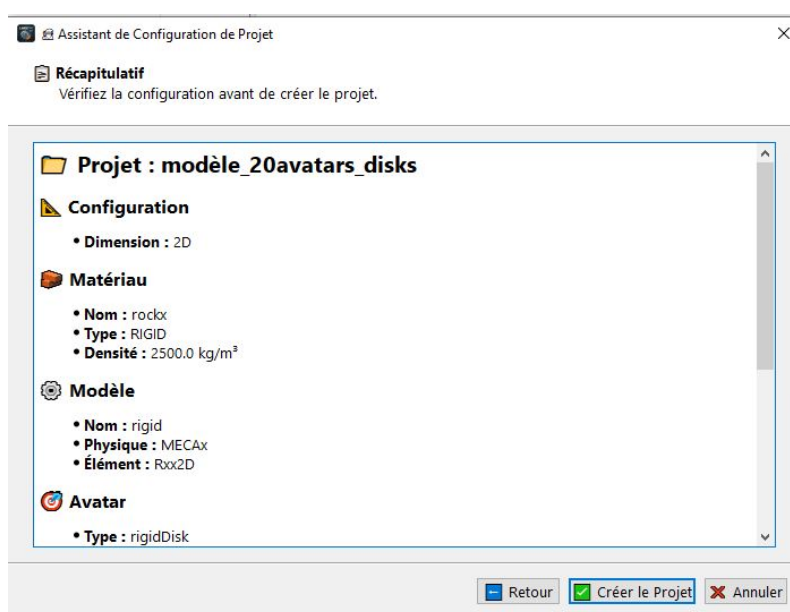
Configuration automatique selon la dimension :

Paramètre	2D	3D
Type de corps	RBDY2	RBDY3
Type de contacteur	DISKx	SPHER
Loi de contact	Celle créée à l'étape 6	Celle créée à l'étape 6

Dépendance : la table de visibilité n'est créée que si une loi de contact a été définie à l'étape précédente **et** qu'un avatar a été créé à l'étape 5. Si l'une ou l'autre de ces conditions n'est pas remplie, la table est ignorée même si la case est cochée.

Synchronisation des couleurs : si un avatar a été créé à l'étape 5, les couleurs candidat et antagoniste sont automatiquement pré-remplies avec la couleur de cet avatar.

Page 8 — Récapitulatif



Page récapitulatif

La dernière page affiche un résumé complet de tous les éléments qui seront créés. Vérifier les informations avant de valider.

Contenu du récapitulatif

Section	Informations affichées
Projet	Nom du projet, dimension (2D/3D)
Matériau	Nom, type, densité — ou matériau existant sélectionné
Modèle	Nom, physique, élément — ou modèle existant sélectionné
Avatar	Type, rayon — ou « Aucun avatar créé »
Loi de contact	Nom, type, coefficient de friction — ou « Aucune loi créée »
Table de visibilité	Corps, contacteur, couleurs, loi appliquée, distance d'alerte — ou « Aucune table créée »

Cliquer sur **■ Créer le Projet** pour finaliser. Tous les éléments sont créés simultanément et apparaissent immédiatement dans l'arbre du modèle et les onglets correspondants.

En cas d'erreur : si la création échoue (nom de matériau en double, paramètre invalide, etc.), une boîte d'erreur s'affiche avec le détail du problème. L'état du projet est entièrement restauré à son état avant l'ouverture de l'assistant.

Après la création

Une fois l'assistant terminé, tous les éléments générés sont modifiables librement via leurs onglets respectifs :

Élément créé	Onglet pour modifier
Matériau	Matériau (Ctrl+1) — sélectionner dans la liste et cliquer sur  Modifier
Modèle	Modèle (Ctrl+2)
Avatar	Avatar (Ctrl+3)
Loi de contact	Contact (Ctrl+9)
Table de visibilité	Visibilité

Récapitulatif des raccourcis

Action	Raccourci
Ouvrir l'assistant	Ctrl+Shift+N
Page suivante	Entrée ou Suivant 
Page précédente	 Retour
Annuler	 Annuler (restaure l'état précédent)
Créer le projet	 Créer le Projet (dernière page)

Conseils d'utilisation

Commencer simple : pour un premier projet LMGC90_GUI 2D, utiliser le type **RIGID** pour le matériau, **Rxx2D** pour le modèle, créer un disque de test et la loi **IQS_CLB** avec un frottement de 0.3. Le projet sera fonctionnel en moins de 2 minutes.

Réutiliser l'existant : si un matériau ou un modèle a déjà été créé dans le projet courant, l'assistant le propose automatiquement en première option. Il n'est pas nécessaire d'en créer un nouveau à chaque fois.

Compléter après : l'assistant crée des éléments de base. Pour vos configurations avancées (éléments finis déformables, lois plastiques, conditions aux limites, boucles granulaires), utiliser directement les onglets spécialisés après la fin de l'assistant.

Création d'un Matériau

L'onglet **Matériau** explique comment créer/modifier/supprimer et configurer un matériau dans LMGC90_GUI.

Interface

L'onglet **Matériau** est divisé en deux parties :

- **Liste des matériaux** (en haut) : tableau affichant tous les matériaux définis dans le projet avec leur nom, type, densité et un aperçu des propriétés. Les matériaux utilisés par au moins un avatar sont affichés en vert.
- **Formulaire de création / modification** (en bas) : champs de saisie adaptés au type sélectionné.

Champs du formulaire

Champ	Description
Nom	Identifiant unique du matériau. 5 caractères maximum (contrainte interne LMGC90). Exemples : TDURx , ROCKx , steel , BEton .
Type	Liste déroulante des types de matériaux supportés. Détermine les propriétés attendues dans le champ suivant.
Densité	Masse volumique en kg/m ³ (système SI). Valeur par défaut : 2800 .
Propriétés	Champ texte libre pour les paramètres spécifiques au type, au format cle=valeur , cle=valeur . Rempli automatiquement avec une suggestion cohérente à chaque changement de type.

Suggestion automatique : à chaque sélection d'un nouveau type, le champ Propriétés et le nom sont remplis automatiquement avec des valeurs typiques. Ces valeurs servent de point de départ et doivent être adaptées au matériau réel.

Types de matériaux disponibles

LMGC90_GUI propose **10 types** de matériaux, correspondant aux types acceptés par `pre.material(materialType=...)` dans `pymgc90`.

Note : les types marqués **_(avancé)_** — **DISCRETE**, **USER_MAT**, **EXTERNAL** — ne disposent pas de suggestion automatique dans l'interface. Leurs paramètres doivent être renseignés manuellement.

Tableau des paramètres

Type	Nom complet	Paramètres principaux	Physique compatible
RIGID	Corps rigide	_(aucun)_	MECAx

Type	Nom complet	Paramètres principaux	Physique compatible
ELAS	Élastique linéaire	elas, young, nu, anisotropy	MECAx
ELAS_DILA	Élastique avec dilatation thermique	elas, young, nu, anisotropy, dilatation, T_ref_meca	MECAx, THERx
VISCO_ELAS	Visco-élastique	elas, young, nu, anisotropy, viscous_model, viscous_young, viscous_nu	MECAx
ELAS_PLAS	Élasto-plastique	elas, young, nu, anisotropy, critere, isoh, iso_hard, isoh_coeff, cinh, visc	MECAx
THERMO_ELAS	Thermo-élastique couplé	elas, young, nu, anisotropy, dilatation, T_ref_meca, conductivity, specific_capacity	THERx, THMx
PORO_ELAS	Poro-élastique (Biot)	elas, young, nu, anisotropy, hydro_cpl, conductivity, specific_capacity	POROX, THMx
DISCRETE _(avancé)_	Éléments discrets (masse-ressort)	masses, stiffnesses, viscosities	MECAx
USER_MAT _(avancé)_	Loi de comportement personnalisée	density, file_mat	MECAx
EXTERNAL _(avancé)_	Interface avec code externe	_(définis par le code externe)_	MECAx

Tableau des usages typiques

Type	Applications typiques	Exemples concrets	Domaines
RIGID	Méthode des éléments discrets (DEM)	Empilements de grains, écoulements granulaires, assemblages de particules	Génie civil, pharmacie, agroalimentaire
ELAS	Structures en régime élastique	Bâtiments, ponts, pièces mécaniques, structures métalliques	Génie civil, mécanique
ELAS_DILA	Contraintes thermiques unilatérales	Structures soumises à des variations de température, dilatation différentielle	Bâtiment, mécanique, électronique
VISCO_ELAS	Matériaux à comportement visqueux	Polymères, asphalte, matériaux amortissants, joints d'étanchéité	Routes, automobile, aéronautique
ELAS_PLAS	Déformations plastiques permanentes	Formage des métaux, impact, endommagement, usinage	Métallurgie, automobile, aéronautique
THERMO_ELAS	Couplage thermo-mécanique complet	Dissipation thermique, chocs thermiques, freinage	Électronique, automobile, nucléaire
PORO_ELAS	Milieux poreux saturés	Consolidation de sols, réservoirs pétroliers, aquifères, stockage CO ₂	Géotechnique, hydrogéologie, pétrole

Type	Applications typiques	Exemples concrets	Domaines
DISCRETE	Systèmes masse-ressort-amortisseur	Isolateurs sismiques, suspensions, liaisons élastiques discrètes	Génie parasismique, automobile
USER_MAT	Lois de comportement sur mesure	Matériaux spécifiques, lois issues de l'expérience	Recherche, matériaux innovants
EXTERNAL	Couplage avec un code externe	Interface avec d'autres logiciels de simulation	Simulation multi-physique

Propriétés détaillées par type

RIGID — Corps rigide

Aucun paramètre de propriété n'est requis. Le champ Propriétés doit rester vide.

Nom : BRIQx
Type : RIGID
Densité : 2000
Propriétés : (vide)

ELAS — Élastique linéaire

```
elas='standard', young=2.1e11, nu=0.3, anisotropy='isotropic'
```

Paramètre	Description	Valeur typique
elas	Formulation élastique : toujours 'standard'.	'standard'
young	Module de Young (Pa).	Acier : 2.1e11 · Béton : 3e10 · Roche : 5e10
nu	Coefficient de Poisson (sans dimension).	0.2 à 0.35
anisotropy	Type d'anisotropie : 'isotropic' ou 'orthotropic'.	'isotropic'
G	Module de cisaillement (Pa)	Acier : 8.1e10

ELAS_DILA — Élastique avec dilatation thermique

```
elas='standard', young=3e10, nu=0.2, anisotropy='isotropic', dilatation=1.2e-5, T_ref_meca=20.0
```

Paramètre	Description	Valeur typique
elas	Formulation élastique.	'standard'
young	Module de Young (Pa).	3e10
nu	Coefficient de Poisson.	0.2
anisotropy	Anisotropie.	'isotropic'
dilatation	Coefficient de dilatation thermique linéique (K ⁻¹).	1e-5 à 2e-5

Paramètre	Description	Valeur typique
<code>T_ref_meca</code>	Température de référence mécanique (°C ou K) — déformation thermique nulle à cette valeur.	<code>20.0</code>

VISCO_ELAS — Visco-élastique

```
elas='standard', anisotropy='isotropic', young=1.17e11, nu=0.35,
viscous_model='KelvinVoigt', viscous_young=1.17e9, viscous_nu=0.35
```

Paramètre	Description	Valeurs
<code>elas</code>	Formulation élastique.	<code>'standard'</code>
<code>young</code>	Module de Young de la branche élastique (Pa).	<code>1.17e11</code>
<code>nu</code>	Coefficient de Poisson élastique.	<code>0.35</code>
<code>anisotropy</code>	Anisotropie.	<code>'isotropic'</code>
<code>viscous_model</code>	Modèle rhéologique. <code>KelvinVoigt</code> = ressort et amortisseur en parallèle (fluage réversible).	<code>'KelvinVoigt' . 'none'</code>
<code>viscous_young</code>	Module de Young de la branche visqueuse (Pa).	<code>1.17e9</code>
<code>viscous_nu</code>	Coefficient de Poisson de la branche visqueuse.	<code>0.35</code>

ELAS_PLAS — Élasto-plastique

```
elas='standard', anisotropy='isotropic', young=2.1e11, nu=0.3,
critere='Von-Mises', isoh='linear', iso_hard=2.5e8, isoh_coeff=1e9,
cinh='none', visc='none'
```

Paramètre	Description	Valeurs
<code>elas</code>	Formulation élastique.	<code>'standard'</code>
<code>young</code>	Module de Young (Pa).	<code>2.1e11</code>
<code>nu</code>	Coefficient de Poisson.	<code>0.3</code>
<code>anisotropy</code>	Anisotropie.	<code>'isotropic' . 'orthotropic'</code>
<code>critere</code>	Critère de plasticité.	<code>'Von-Mises' . 'none'</code>
<code>isoh</code>	Type d'écrouissage isotrope.	<code>'none', 'linear'</code>
<code>iso_hard</code>	Limite d'élasticité initiale σ_H (Pa).	<code>2.5e8</code>
<code>isoh_coeff</code>	Module d'écrouissage isotrope H (Pa).	<code>1e9</code>
<code>cinh</code>	Écrouissage cinématique.	<code>'none'</code>
<code>visc</code>	Viscoplasticité.	<code>'none'</code>

THERMO_ELAS — Thermo-élastique couplé

```
elas='standard', young=3e10, nu=0.2, anisotropy='isotropic',
dilatation=1.2e-5, T_ref_meca=20.0, conductivity=1.8, specific_capacity=880.0
```

Paramètre	Description	Valeur typique
elas	Formulation élastique.	'standard'
young	Module de Young (Pa).	3e10
nu	Coefficient de Poisson.	0.2
anisotropy	Anisotropie.	'isotropic'
dilatation	Coefficient de dilatation thermique (K ⁻¹).	1.2e-5
T_ref_meca	Température de référence mécanique.	20.0
conductivity	Conductivité thermique (W/m/K) ou 'field'	
specific_capacity	Capacité thermique massique (J/kg/K) ou 'field'	

PORO_ELAS — Poro-élastique (Biot)

```
elas='standard', young=5e7, nu=0.3, anisotropy='isotropic',
hydro_cpl=0.8, conductivity=1e-8, specific_capacity=1e-10
```

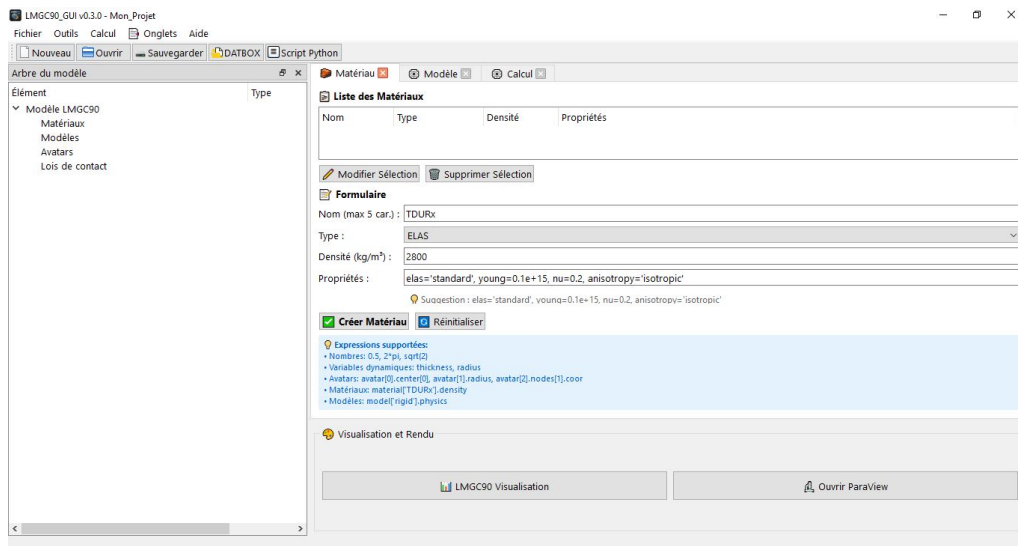
Paramètre	Description	Valeur typique
elas	Formulation élastique.	'standard'
young	Module de Young du squelette solide (Pa).	5e7
nu	Coefficient de Poisson du squelette.	0.3
anisotropy	Anisotropie.	'isotropic'
hydro_cpl	Coefficient de couplage de Biot (0 à 1).	0.8
conductivity	Conductivité hydraulique (m/s) ou 'field'	
specific_capacity	Capacité de stockage hydraulique (Pa ⁻¹) ou 'field'	

Exemple de création — acier élastique

1. Ouvrir l'onglet **Matériau** (Ctrl+1).
2. Sélectionner le type **ELAS** dans la liste déroulante — le champ Propriétés se remplit automatiquement.
3. Modifier les valeurs dans le champ Propriétés :

```
elas='standard', young=2.1e11, nu=0.3, anisotropy='isotropic'
```

4. Régler la densité à **7850** kg/m³.
5. Saisir un nom : **steel** ou **ACIEx**.
6. Cliquer sur **Créer Matériau**.

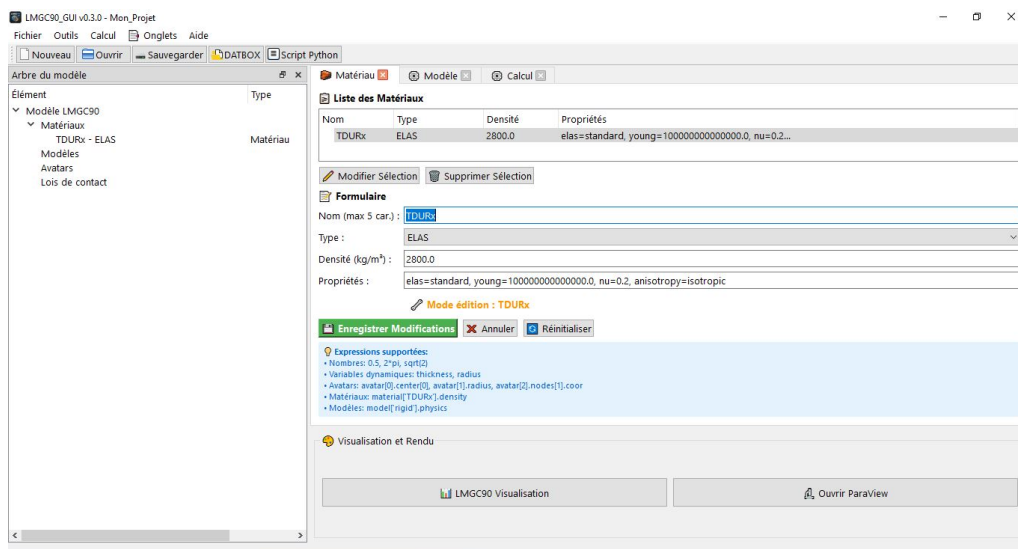


Création d'un matériau acier

Modification et suppression

Dans le tableau de la liste des matériaux, sélectionnez le matériau à modifier puis cliquez sur le bouton **Modifier Sélection**. Toutes les données du matériau sont chargées dans le formulaire en mode **Édition**.

- Apportez vos modifications dans les champs.
- Cliquez sur **Enregistrer Modifications** pour valider.
- Cliquez sur **Annuler** pour ignorer les changements et revenir au mode normal.



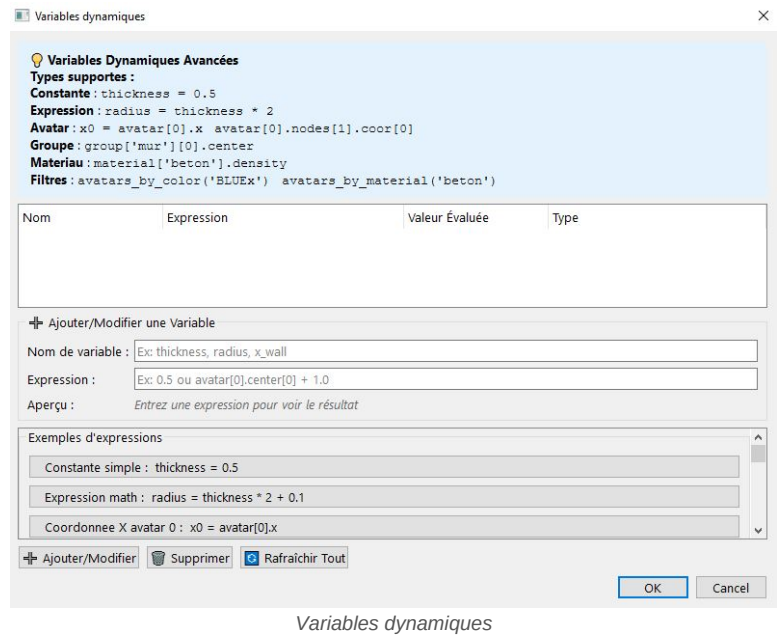
Modification d'un matériau

Suppression : un matériau utilisé par au moins un avatar ne peut pas être supprimé directement. Un message d'avertissement indique les avatars concernés. Il faut d'abord réassigner ces avatars à un autre matériau, ou les supprimer.

Variables dynamiques

Les variables dynamiques permettent de définir des valeurs ou expressions réutilisables dans tous les champs numériques de l'interface, y compris dans le champ Propriétés des matériaux.

Ouvrez la boîte de dialogue via **Outils** → **Variables dynamiques** ou le raccourci **Ctrl+V**.



Comment créer une variable

1. Dans la boîte de dialogue, cliquez sur l'un des exemples pour le charger comme base (optionnel).
2. Saisissez le **nom** de la variable (ex : **young_ref**).
3. Saisissez la **valeur ou expression** (ex : **2.1e11**).
4. Cliquez sur **Ajouter ou modifier**.
5. Cliquez sur **OK** pour fermer.

Les variable dynamiques sont désormais utilisable dans n'importe quel champ de l'application en tapant son nom directement. Voir Variables dynamiques.

Exemples d'expressions

Expression saisie	Résultat	Usage
<code>young_ref = 2.1e11</code>	Constante numérique	<code>young=young_ref</code> dans les propriétés
<code>young_beton = young_ref / 7</code>	Expression calculée	Rapport de rigidité béton/acier
<code>nu_courant = 0.3</code>	Constante	Coefficient de Poisson par défaut
<code>radius = avatar[0].radius * 2</code>	Propriété d'un avatar existant	Rayon dérivé d'un autre avatar

Remarque : les expressions sont évaluées via **SafeEvaluator**, qui autorise les opérations mathématiques Python standard (+, -, <i>, /, </i>*, **math.sqrt(...)**, etc.) ainsi que l'accès aux propriétés des avatars et matériaux du projet.

Astuces

- **Nom limité à 5 caractères :** Les noms en LMGC90 sont sur cinq caractères. Préférez des codes courts comme **steel**, **BETon**, **GRAN1**.

-
- **Matériau RIGID sans propriétés** : pour les corps rigides, le champ Propriétés doit rester vide. Seule la densité est utilisée pour le calcul de la masse et du moment d'inertie.
 - **Valeur 'field'** : pour `conductivity` et `specific_capacity` des types `THERMO_ELAS` et `PORO_ELAS`, la valeur `'field'` indique que ce paramètre est défini par le modèle éléments finis et non par une constante scalaire.
 - **Cohérence matériau / modèle** : le type du matériau doit être compatible avec la physique du modèle associé à l'avatar. Par exemple, un matériau `PORO_ELAS` doit être associé à un modèle de physique `POROX` ou `MULTI`.
 - **Variables dynamiques** : utilisez `Ctrl+V` pour accéder rapidement aux variables et éviter les saisies répétitives dans les champs Propriétés.

Création d'un Modèle

Le modèle définit le **solveur éléments finis** qui sera utilisé pour les corps déformables du projet. Il associe une physique, un type d'élément fini, une dimension spatiale et des options numériques.

Interface

L'onglet **Modèle** (**Ctrl+2**) est divisé en deux parties :

- **Liste des modèles** (en haut) : tableau affichant tous les modèles définis dans le projet avec leur nom, physique, élément et dimension. Les modèles utilisés par au moins un avatar sont affichés en vert.
- **Formulaire de création / modification** (en bas) : champs de saisie dont la section Options s'adapte automatiquement à l'élément sélectionné.

Champs du formulaire

Champ	Description
Nom	Identifiant unique du modèle. sur 5 caractères (contrainte interne LMGC90). Exemples : rigid , MECAX , elas2 , ther3 .
Physique	Famille de physique résolue. Détermine la liste d'éléments disponibles. Voir section Physiques disponibles.
Dimension	Dimension spatiale : 2 (2D) ou 3 (3D). La liste d'éléments se met à jour automatiquement.
Élément	Type d'élément fini. La liste dépend de la physique ET de la dimension sélectionnées.
Options	Paramètres numériques affichés automatiquement selon l'élément. Les éléments rigides (Rxx2D , Rxx3D) n'ont aucune option.

Mise à jour automatique : à chaque changement de physique ou de dimension, la liste d'éléments disponibles est rechargée. Si l'élément précédemment sélectionné existe dans la nouvelle liste, il est conservé.

Physiques disponibles

Les quatre physiques sont toutes implémentées dans LMGC90_GUI.

Code	Nom complet	Description	Matériaux compatibles
MECAX	Mécanique des solides	Déformation, contrainte, dynamique.	RIGID , ELAS , ELAS_DILA , VISCO_ELAS , ELAS_PLAS
THERX	Thermique	Diffusion de chaleur, convection, rayonnement.	THERMO_ELAS
POROX	Poromécanique	Couplage solide / fluide selon la théorie de Biot.	PORO_ELAS
MULTI	Thermo-hydraulique-mécanique (THM)	Couplage complet mécanique + thermique + hydraulique.	PORO_ELAS , THERMO_ELAS

Éléments disponibles par physique

MECAx — Éléments mécaniques 2D

Élément	Géométrie	Nœuds	Ordre	Description
Rxx2D	Point	1	—	Corps rigide 2D. Aucune option.
T3xxx	Triangle 3 nœuds	3	1	Triangle linéaire standard.
T3Lxx	Triangle 3 nœuds	3	1	Triangle linéaire enrichi (incompatible).
T6xxx	Triangle 6 nœuds	6	2	Triangle quadratique.
DKTxx	Triangle 3 nœuds	3	1	Triangle de Kirchhoff discret (plaques minces).
Q4xxx	Quadrangle 4 nœuds	4	1	Quadrangle bilinéaire standard.
Q4P0x	Quadrangle 4 nœuds	4	1	Quadrangle bilinéaire + pression constante (quasi-incompressible).
Q8xxx	Quadrangle 8 nœuds	8	2	Quadrangle sérendipité.
Q8Rxx	Quadrangle 8 nœuds	8	2	Quadrangle sérendipité à intégration réduite.
Q9xxx	Quadrangle 9 nœuds	9	2	Quadrangle Lagrange biquadratique.
BARxx	Segment 2 nœuds	2	1	Barre / treillis 1D.
SPRG2	Segment 2 nœuds	2	1	Ressort 2D (discrete=yes__ ajouté automatiquement).

MECAx — Éléments mécaniques 3D

Élément	Géométrie	Nœuds	Ordre	Description
Rxx3D	Point	1	—	Corps rigide 3D. Aucune option.
TE4xx	Tétraèdre 4 nœuds	4	1	Tétraèdre linéaire.
TE4Lx	Tétraèdre 4 nœuds	4	1	Tétraèdre linéaire enrichi (F-bar).
TE10x	Tétraèdre 10 nœuds	10	2	Tétraèdre quadratique.
H8xxx	Hexaèdre 8 nœuds	8	1	Hexaèdre trilineaire.
H20xx	Hexaèdre 20 nœuds	20	2	Hexaèdre sérendipité.
H20Rx	Hexaèdre 20 nœuds	20	2	Hexaèdre sérendipité à intégration réduite.
PRI6x	Prisme 6 nœuds	6	1	Prisme linéaire.

Élément	Géométrie	Nœuds	Ordre	Description
SHB6x	Prisme 6 nœuds	6	1	Prisme solide-coque SHB6.
PRI15	Prisme 15 nœuds	15	2	Prisme quadratique.
BARxx	Segment 2 nœuds	2	1	Barre / treillis 1D.
SPRG3	Segment 2 nœuds	2	1	Ressort 3D (discrete=yes__ ajouté automatiquement).

THERx — Éléments thermiques 2D

Élément	Nœuds	Ordre	Options thermiques spécifiques
Rxx2D	1	—	aucune
T3xxx	3	1	mass_storage, convection, radiation
T6xxx	6	2	mass_storage, convection, radiation
DKTxx	3	1	mass_storage, convection, radiation
Q4xxx	4	1	mass_storage, convection, radiation
Q4P0x	4	1	mass_storage, convection, radiation
Q8xxx	8	2	mass_storage, convection, radiation
Q8Rxx	8	2	mass_storage, convection, radiation
SPRG2	2	1	mass_storage uniquement
S2xth	2	1	mass_storage uniquement — segment thermique 1D

THERx — Éléments thermiques 3D

Élément	Nœuds	Ordre	Options thermiques spécifiques
Rxx3D	1	—	aucune
TE4xx	4	1	mass_storage, convection, radiation
TE10x	10	2	mass_storage, convection, radiation
H8xxx	8	1	mass_storage, convection, radiation
H20xx	20	2	mass_storage, convection, radiation
H20Rx	20	2	mass_storage, convection, radiation
PRI6x	6	1	mass_storage, convection, radiation

Élément	Nœuds	Ordre	Options thermiques spécifiques
PRI15	15	2	mass_storage, convection, radiation
SPRG3	2	1	mass_storage uniquement

POROX — Éléments poromécaniques (éléments mixtes déplacement-pression)

Élément	Dim.	Géométrie	Nœuds	Ordre	Description
T33xx	2D	Triangle	3	1	Triangle mixte P1/P1.
T63xx	2D	Triangle	6	2	Triangle mixte P2/P1 — satisfait la condition LBB.
Q44xx	2D	Quadrangle	4	1	Quadrangle mixte Q1/Q1.
Q84xx	2D	Quadrangle	8	2	Quadrangle mixte Q2/Q1 — satisfait la condition LBB.
TE44x	3D	Tétraèdre	4	1	Tétraèdre mixte P1/P1.
TE104	3D	Tétraèdre	10	2	Tétraèdre mixte P2/P1 — satisfait la condition LBB.
H88xx	3D	Hexaèdre	8	1	Hexaèdre mixte Q1/Q1.
H208x	3D	Hexaèdre	20	2	Hexaèdre mixte Q2/Q1 — satisfait la condition LBB.

MULTI — Éléments THM 2D et 3D

Mêmes éléments mixtes que POROX, avec en plus H8xxx en 3D pour l'interpolation uniforme des trois champs couplés.

Élément	Dim.	Description
T33xx	2D	Triangle mixte P1/P1.
T63xx	2D	Triangle mixte P2/P1.
Q44xx	2D	Quadrangle mixte Q1/Q1.
Q84xx	2D	Quadrangle mixte Q2/Q1.
TE44x	3D	Tétraèdre mixte P1/P1.
TE104	3D	Tétraèdre mixte P2/P1.
H8xxx	3D	Hexaèdre trilineaire (interpolation uniforme THM).
H88xx	3D	Hexaèdre mixte Q1/Q1.
H208x	3D	Hexaèdre mixte Q2/Q1.

Tableau récapitulatif par usage

Usage	Dimension	Éléments recommandés	Remarque
Corps rigides 2D (DEM)	2D	Rxx2D	Aucune option
Corps rigides 3D (DEM)	3D	Rxx3D	Aucune option
Ressorts / barres 2D	2D	SPRG2, BARxx	discrete=yes__ auto
Ressorts / barres 3D	3D	SPRG3, BARxx	discrete=yes__ auto
Structures 2D — précision standard	2D	Q4xxx, T3xxx	Rapide, adapté aux maillages structurés
Structures 2D — géométrie complexe	2D	T6xxx, Q8xxx	Maillages automatiques non structurés
Plaques minces 2D	2D	DKTxx	Formulation Kirchhoff
Quasi-incompressibilité 2D	2D	Q4P0x	Pression constante par élément
Structures 3D — maillage structuré	3D	H8xxx, H20xx	Hexaèdres recommandés
Structures 3D — maillage automatique	3D	TE10x, TE4xx	Tétraèdres adaptatifs
Coques épaisses 3D	3D	SHB6x, PRI6x	Solidescoques
Thermique 2D	2D	Q4xxx, T3xxx	Physique THERx
Thermique 3D	3D	H8xxx, TE4xx	Physique THERx
Segment thermique 1D	2D / 3D	S2xth	Physique THERx uniquement
Poro-mécanique 2D (Biot)	2D	T63xx, Q84xx	LBB satisfaite — recommandé
Poro-mécanique 3D (Biot)	3D	TE104, H208x	LBB satisfaite — recommandé
THM couplé 3D	3D	H8xxx, TE104	Physique MULTI

Options du modèle

Options MECAx — spécifiques à l'élément

Ces trois options s'affichent pour tous les éléments MECAx non rigides.

kinematic — Hypothèse cinématique

Valeur	Description	Cas d'usage
small	Petites déformations (HPP). Relation linéaire déplacement-déformation. Géométrie de référence supposée constante.	Béton, acier en élasticité, la plupart des structures du génie civil.
large	Grandes déformations. Relation non linéaire géométrique. Mise à jour de la configuration à chaque pas.	Caoutchouc, matériaux souples, formage, impact, crash.

formulation — Formulation lagrangienne

Pertinent uniquement pour kinematic=large.

Valeur	Description
UpdtL	Lagrangien actualisé. La configuration de référence est mise à jour à chaque pas. Contraintes de Cauchy. Recommandé pour les grandes déformations continues (métaux).
Total	Lagrangien total. La configuration de référence reste la configuration initiale. Contraintes de Piola-Kirchhoff. Recommandé pour les grandes déformations réversibles (élastomères).

mass_storage — Stockage de la matrice de masse

Valeur	Description	Cas d'usage
lump_	Masse concentrée. Matrice diagonale. Inversion immédiate. Moins précis sur les hautes fréquences.	Dynamique explicite, schémas à pas de temps petit.
coher	Masse cohérente. Matrice pleine intégrée. Plus précis. Plus coûteux à inverser.	Dynamique implicite, analyse modale, hautes fréquences.

Options MECaX — communes à tous les éléments (hors rigides)

Ces trois options s'affichent en complément des options spécifiques à l'élément.

material — Loi de comportement locale

Valeur	Comportement	Matériaux associés
elas_	Élasticité linéaire standard	ELAS, ELAS_DILA, RIGID
elasd	Élasticité endommageable	ELAS_PLAS (avec variable d'endommagement)
J2iso	Plasticité J2 isotrope	ELAS_PLAS (isoh='linear' ou 'nonlinear')
J2mix	Plasticité J2 mixte (isotrope + cinématique)	ELAS_PLAS (isoh et cinh tous les deux actifs)
kvisc	Visco-élasticité de Kelvin-Voigt	VISCO_ELAS

anisotropy — Anisotropie

Valeur	Description
iso__	Isotrope. Propriétés identiques dans toutes les directions. 2 paramètres : E (Young), ν (Poisson).
ortho	Orthotrope. Propriétés différentes selon 3 directions principales. Nécessite les 9 constantes d'élasticité ($E_x, E_y, E_z, \nu_{xy}, \nu_{yz}, \nu_{xz}, G_{xy}, G_{yz}, G_{xz}$). Adapté aux composites, au bois, aux matériaux stratifiés.

external_model — Modèle externe

Valeur	Description
MatL_	Loi interne LMGC90 (comportement standard). Valeur par défaut.
Demfi	Interface DemFi — couplage avec un modèle DEM externe.

Valeur	Description
Umat__	Interface UMAT — routine utilisateur de type ABAQUS (Fortran / C).
no__	Désactivé.
yes__	Activé (selon le contexte de l'option).

Options THERx

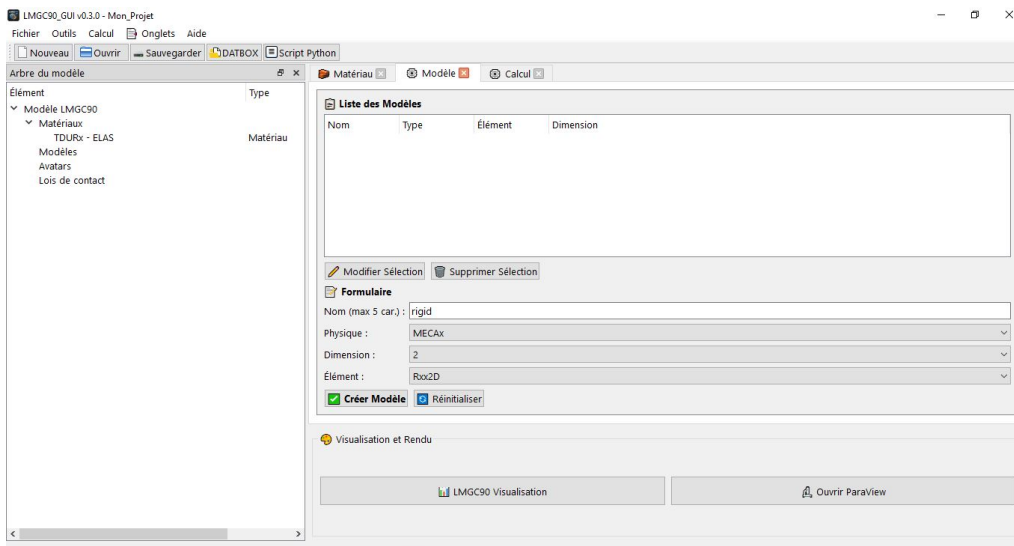
Ces options s'affichent pour tous les éléments thermiques non rigides.

Option	Valeurs	Description
mass_storage	lump__ · coher	Stockage de la matrice de capacité thermique. Même signification qu'en MECAx.
convection	no__ · yes__	Active les termes de convection thermique en surface.
radiation	no__ · yes__	Active les termes de rayonnement thermique en surface (loi de Stefan-Boltzmann).
anisotropy	iso__ · ortho	Anisotropie de la conductivité thermique.
external_model	MatL · Demfi · Umat__ · no__ · yes__	Interface avec un modèle thermique externe.

Note : SPRG2, SPRG3 et S2xth n'ont que l'option mass_storage. Les options convection et radiation ne s'affichent pas pour ces éléments 1D.

Exemple de création — corps rigide 2D

1. Ouvrir l'onglet **Modèle** (Ctrl+2).
2. Saisir le nom : **rigid**.
3. Sélectionner la physique : **MECAx**.
4. Sélectionner la dimension : **2**.
5. Sélectionner l'élément : **Rxx2D** — la section Options disparaît (aucune option pour les éléments rigides).
6. Cliquer sur **■ Créer Modèle**.



Création d'un modèle rigide 2D

Exemple de création — mécanique 2D élastique

1. Nom : **elas2**.
2. Physique : **MECAX** · Dimension : **2** · Élément : **Q4xxx**.
3. Options affichées automatiquement :
 - **kinematic** → **small**
 - **formulation** → **UpdtL**
 - **mass_storage** → **lump_**
 - **material** → **elas_**
 - **anisotropy** → **iso_**
 - **external_model** → **MatL_**
4. Cliquer sur **Créer Modèle**.

Modification et suppression

Sélectionnez un modèle dans la liste puis cliquez sur **Modifier Sélection** pour le charger en mode **Édition**. Apportez vos modifications puis cliquez sur **Enregistrer Modifications**, ou sur **Annuler** pour ignorer les changements.

Suppression : un modèle utilisé par au moins un avatar ne peut pas être supprimé directement. La boîte de dialogue d'avertissement liste les avatars concernés.

Tableau de compatibilité matériau / physique

Matériau	MECAX	THERx	POROX	MULTI
RIGID	■	■	■	■
ELAS	■	■	■	■
ELAS_DILA	■	■	■	■
VISCO_ELAS	■	■	■	■

Matériau	MECAx	THERx	PORox	MULTI
ELAS_PLAS	■	■	■	■
THERMO_ELAS	■	■	■	■
PORO_ELAS	■	■	■	■
DISCRETE	■	■	■	■
USER_MAT	■	■	■	■
EXTERNAL	■	■	■	■

Tableau de compatibilité matériau / option `material`

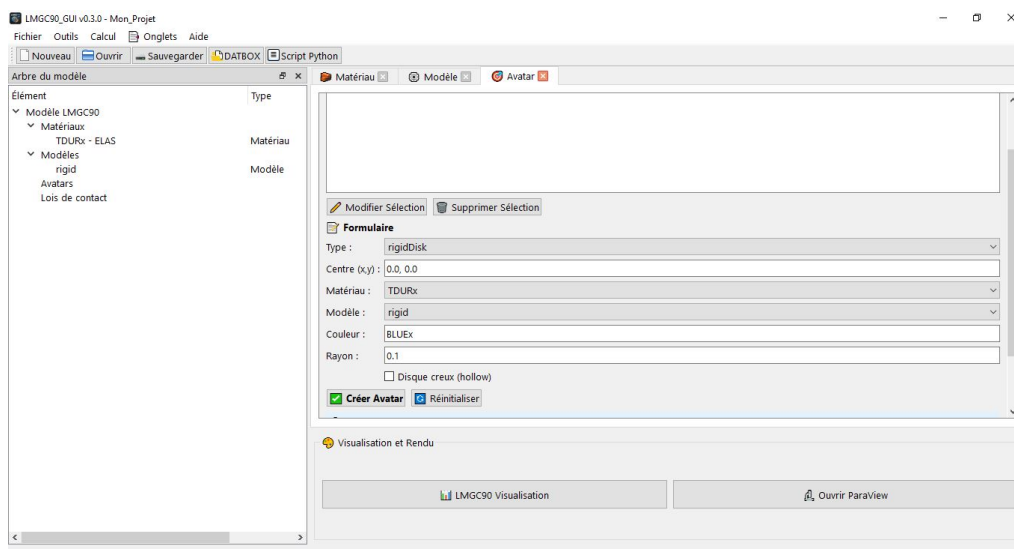
Matériau	elas_	elasd	J2iso	J2mix	kvisc
RIGID	■	—	—	—	—
ELAS	■	—	—	—	—
ELAS_DILA	■	■	—	—	—
VISCO_ELAS	—	—	—	—	■
ELAS_PLAS	—	■	■	■	—
THERMO_ELAS	■	■	—	—	—

Astuces

- **Nom limité à 5 caractères** : LMGC90 ignore silencieusement les caractères supplémentaires. Préférez `rigid`, `elas2`, `ther3`, `poro2`.
- **Élément Rxx** : les corps rigides utilisent exclusivement `Rxx2D` (2D) ou `Rxx3D` (3D). Ces éléments n'ont aucune option numérique.
- **Ressorts** : pour `SPRG2` et `SPRG3`, l'option `discrete=yes__` est ajoutée automatiquement à la création — il n'est pas nécessaire de la saisir manuellement.
- **LBB en poromécanique** : pour les calculs de consolidation ou de diffusion de pression, privilégiez les éléments d'ordre supérieur (`T63xx`, `Q84xx`, `TE104`, `H208x`) qui satisfont la condition de Ladyzhenskaya-Babuška-Brezzi et évitent les oscillations parasites de pression.
- **kinematic=large avec formulation** : le champ `formulation` n'a d'effet que si `kinematic=large`. En petites déformations, la valeur de `formulation` est ignorée par le solveur.
- **Modèles non utilisés** : les modèles non associés à un avatar apparaissent en noir dans la liste. Ils n'ont aucun effet sur le calcul.

Création d'un Avatar (Corps rigide simple)

Onglet Avatar Permet de créer, modifier et supprimer les corps rigides du projet : disques, sphères, joncs, polygones, murs, cylindres, polyèdres et plus encore. Chaque avatar est défini par un **type**, un **centre**, un **matériau**, un **modèle** et des **paramètres géométriques** spécifiques à son type.



captures/avatar_disque.JPG

Interface générale

L'onglet est divisé en deux zones :

- **Liste des avatars** (en haut) : arbre affichant tous les avatars du projet avec leur index, type, couleur et centre. Double-clic sur une ligne pour l'éditer. Clic droit pour accéder au menu contextuel (Modifier, Supprimer, Informations).
- **Formulaire de création / modification** (en bas) : champs adaptés au type d'avatar sélectionné.

Liste des avatars par dimension

Avatars 2D

Type pylmgc90	Description courte	Paramètres clés
<code>rigidDisk</code>	Disque rigide	<code>r</code> , <code>is_hollow</code>
<code>rigidJonc</code>	Ellipse rigide	<code>axe1</code> , <code>axe2</code>
<code>rigidPolygon</code>	Polygone rigide	<code>generation_type</code> , <code>nb_vertices</code> , <code>radius</code> ou <code>vertices</code>
<code>rigidOvoidPolygon</code>	Ovoïde rigide	<code>ra</code> , <code>rb</code> , <code>nb_vertices</code>
<code>rigidDiscreteDisk</code>	Disque discret	<code>r</code>

Type pylmgc90	Description courte	Paramètres clés
<code>rigidCluster</code>	Cluster de disques	<code>r</code> , <code>nb_disk</code>
<code>roughWall</code>	Mur rugueux	<code>l</code> , <code>r</code> , <code>nb_vertex</code>
<code>fineWall</code>	Mur fin	<code>l</code> , <code>r</code> , <code>nb_vertex</code>
<code>smoothWall</code>	Mur lisse	<code>l</code> , <code>h</code> , <code>nb_polyg</code>
<code>granuloRoughWall</code>	Mur granulaire	<code>l</code> , <code>rmin</code> , <code>rmax</code> , <code>nb_vertex</code>

Avatars 3D

Type pylmgc90	Description courte	Paramètres clés
<code>rigidSphere</code>	Sphère rigide	<code>r</code>
<code>rigidPlan</code>	Plan rigide	<code>axe1</code> , <code>axe2</code> , <code>axe3</code>
<code>rigidCylinder</code>	Cylindre rigide	<code>r</code> , <code>h</code>
<code>rigidPolyhedron</code>	Polyèdre rigide	<code>generation_type</code> , <code>nb_vertices</code> , <code>radius</code> ou <code>vertices</code> + <code>faces</code>
<code>roughWall3D</code>	Mur rugueux 3D	<code>lx</code> , <code>ly</code> , <code>r</code>
<code>granuloRoughWall3D</code>	Mur granulaire 3D	<code>lx</code> , <code>ly</code> , <code>rmin</code> , <code>rmax</code>

Champs communs à tous les types

Champ	Description
Type	Type d'avatar pylmgc90. Détermine les champs supplémentaires affichés.
Centre	Coordonnées du centre de référence. Format <code>x</code> , <code>y</code> en 2D ou <code>x</code> , <code>y</code> , <code>z</code> en 3D. Accepte les expressions Python (<code>avatar[0].x + 0.5</code>).
Matériau	Sélection parmi les matériaux définis dans l'onglet Matériaux.
Modèle	Sélection parmi les modèles définis dans l'onglet Modèles.
Couleur	Couleur d'affichage LMGC90 en 5 caractères. Voir liste des couleurs ci-dessous.

Couleurs LMGC90 disponibles

Exemples de couleurs que vous pourriez utiliser :

Code	Couleur
<code>BLUEX</code>	Bleu
<code>REDXX</code>	Rouge
<code>VERTX</code>	Vert
<code>JAUNX</code>	Jaune
<code>GRAYX</code>	Gris
<code>BLACX</code>	Noir
<code>WHITX</code>	Blanc
<code>ORANX</code>	Orange
<code>CYANX</code>	Cyan
<code>MAGEX</code>	Magenta

Code	Couleur
VIOLx	Violet
ROSEx	Rose

Types d'avatars 2D

1. rigidDisk — Disque rigide 2D

Corps circulaire rigide 2D. C'est le type le plus courant pour les simulations granulaires.

Paramètre	Description	Exemple
r (rayon)	Rayon du disque (m).	0.1
is_hollow	Si coché, crée un disque creux (<code>is_Hollow=True</code>). Utile pour les anneaux rigides.	case à cocher

rigidJonc — Jonc / Ellipse rigide 2D

Corps elliptique rigide 2D. Défini par deux demi-axes.

Paramètre	Description	Exemple
axe1	Demi-axe principal (m) — longueur.	0.15
axe2	Demi-axe secondaire (m) — largeur.	0.05

rigidPolygon — Polygone rigide 2D

Corps polygonal rigide 2D. Trois modes de génération disponibles selon `generation_type`.

Paramètre	Description	Valeurs
generation_type	Mode de génération de la forme.	regular · full · bevel
nb_vertices	Nombre de sommets (pour <code>regular</code> et <code>full</code>).	3 à 20
radius	Rayon du cercle circonscrit (m) — utilisé pour <code>regular</code> . Non utilisé pour <code>full</code> et <code>bevel</code> .	0.1
vertices	Liste explicite de sommets <code>[[x1, y1], [x2, y2], ...]</code> — utilisé pour <code>full</code> et <code>bevel</code> .	<code>[[-0.1, -0.1], [0.1, -0.1], [0., 0.1]]</code>

Modes de génération :

- `regular` : polygone régulier (tous les côtés égaux). Défini par `nb_vertices` et `radius` (rayon du cercle circonscrit).
- `full` : polygone quelconque à partir d'une liste de sommets explicites. Le rayon n'est pas utilisé.
- `bevel` : polygone avec chanfreinage automatique des angles pour éviter les singularités de contact.

rigidOvoidPolygon — Ovoïde rigide 2D

Corps ovoïde (ellipse polygonale) rigide 2D. Approximation polygonale d'une ellipse.

Paramètre	Description	Exemple
<code>ra</code>	Demi-axe principal (m).	<code>0.15</code>
<code>rb</code>	Demi-axe secondaire (m).	<code>0.08</code>
<code>nb_vertices</code>	Nombre de sommets de l'approximation polygonale.	<code>20</code>

rigidDiscreteDisk — Disque discret rigide 2D

Corps circulaire rigide 2D à cinématique discrète. Même géométrie qu'un `rigidDisk`, mais avec un contacteur de type `xKSID` (disque discret). Utilisé dans certains modèles discrets avancés.

Paramètre	Description	Exemple
<code>r</code> (rayon)	Rayon du disque (m).	<code>0.1</code>

rigidCluster — Cluster de disques rigides 2D

Corps rigide 2D composé de plusieurs disques liés rigidement. Permet de créer des formes non convexes complexes.

Paramètre	Description	Exemple
<code>r</code>	Rayon de chaque disque dans le cluster (m).	<code>0.05</code>
<code>nb_vertices</code> (<code>nb_disk</code>)	Nombre de disques dans le cluster.	<code>4</code>

Le paramètre est nommé `nb_disk` dans le script `pymgc90` généré (pas `nb_vertices`).

roughWall — Mur rugueux 2D

Paroi 2D rugueuse composée de disques alignés. Utilisé pour les parois confinantes avec rugosité géométrique.

Paramètre	Description	Exemple
<code>l</code>	Longueur totale du mur (m).	<code>2.0</code>
<code>r</code>	Rayon des disques constituant la rugosité (m).	<code>0.05</code>
<code>nb_vertex</code>	Nombre de disques le long du mur. Défaut : <code>10</code> .	<code>20</code>

fineWall — Mur fin 2D

Paroi 2D fine composée de disques très petits. Mêmes paramètres que `roughWall` mais avec une rugosité plus fine.

Paramètre	Description	Exemple
<code>l</code>	Longueur totale du mur (m).	<code>2.0</code>
<code>r</code>	Rayon des disques (m). Typiquement très petit (0,001 à 0,01).	<code>0.005</code>
<code>nb_vertex</code>	Nombre de disques. Défaut : <code>10</code> .	<code>50</code>

smoothWall — Mur lisse 2D

Paroi 2D lisse définie par une demi-largeur et une hauteur. Contacteur **CLxxx** (surface continue).

Paramètre	Description	Exemple
l	Demi-longueur du mur (m) — la longueur totale est $2 \times l$.	1.0
h	Demi-hauteur (épaisseur) du mur (m).	0.01
nb_polyg	Nombre de segments polygonaux. Défaut : 10 .	20

granuloRoughWall — Mur granulaire rugueux 2D

Paroi 2D avec rugosité aléatoire générée par distribution granulométrique de disques.

Paramètre	Description	Exemple
l	Longueur totale du mur (m).	2.0
rmin	Rayon minimal des disques (m).	0.01
rmax	Rayon maximal des disques (m).	0.05
nb_vertex	Nombre de disques. Défaut : 10 .	30

Types d'avatars 3D

rigidSphere — Sphère rigide 3D

Corps sphérique rigide 3D. Équivalent 3D du **rigidDisk**.

Paramètre	Description	Exemple
r (rayon)	Rayon de la sphère (m).	0.1

rigidPlan — Plan rigide 3D

Surface plane rigide 3D. Défini par trois vecteurs directeurs formant un repère local.

Paramètre	Description	Exemple
axe1	Vecteur du premier axe du plan (direction X locale).	[1.0, 0.0, 0.0]
axe2	Vecteur du deuxième axe du plan (direction Y locale).	[0.0, 1.0, 0.0]
axe3	Normale au plan (direction Z locale).	[0.0, 0.0, 1.0]

Les trois vecteurs doivent former une base orthonormée directe.

rigidCylinder — Cylindre rigide 3D

Corps cylindrique rigide 3D.

Paramètre	Description	Exemple
<code>r</code> (rayon)	Rayon du cylindre (m).	<code>0.1</code>
<code>h</code>	Hauteur (longueur axiale) du cylindre (m). Défaut : <code>1.0</code> si absent.	<code>0.5</code>

rigidPolyhedron — Polyèdre rigide 3D

Corps polyédrique rigide 3D. Deux modes de génération disponibles selon `generation_type`.

Paramètre	Description	Valeurs
<code>generation_type</code>	Mode de génération.	<code>regular</code> · <code>vertices</code>
<code>nb_vertices</code>	Nombre de sommets (pour <code>regular</code>).	8 (cube), 12 (icosaèdre)...
<code>radius</code>	Rayon du polyèdre régulier (m) — pour <code>regular</code> .	<code>0.1</code>
<code>vertices</code>	Liste explicite de sommets 3D [[<code>x</code> , <code>y</code> , <code>z</code>],...] — pour <code>vertices</code> .	[[<code>-1,-1,-1</code>], [<code>1,-1,-1</code>], ...]
<code>faces</code>	Connectivité des faces [[<code>i</code> , <code>j</code> , <code>k</code>],...] (dans <code>wall_params</code>) — pour <code>vertices</code> .	[[<code>0,1,2</code>], [<code>2,3,0</code>], ...]

Modes de génération :

- `regular` : polyèdre régulier (similaire à une sphère polygonale). Défini par `nb_vertices` et `radius`.
- `vertices` : polyèdre quelconque défini par une liste explicite de sommets et la connectivité des faces.

roughWall3D — Mur rugueux 3D

Paroi 3D rugueuse composée de sphères alignées sur une surface rectangulaire.

Paramètre	Description	Exemple
<code>lx</code>	Dimension du mur en X (m).	<code>2.0</code>
<code>ly</code>	Dimension du mur en Y (m).	<code>2.0</code>
<code>r</code>	Rayon des sphères de rugosité (m).	<code>0.05</code>

granuloRoughWall3D — Mur granulaire rugueux 3D

Paroi 3D rugueuse avec sphères de tailles aléatoires.

Paramètre	Description	Exemple
<code>lx</code>	Dimension en X (m).	<code>2.0</code>
<code>ly</code>	Dimension en Y (m).	<code>2.0</code>
<code>rmin</code>	Rayon minimal des sphères (m).	<code>0.01</code>
<code>rmax</code>	Rayon maximal des sphères (m).	<code>0.05</code>

Remarques générales

Expressions Python dans les champs numériques : tous les champs numériques (centre, rayon, dimensions) acceptent des expressions Python évaluées via `SafeEvaluator`. Exemples : `avatar[0].radius <i> 2, thickness + 0.1, math.sqrt(2) </i> r_base`.

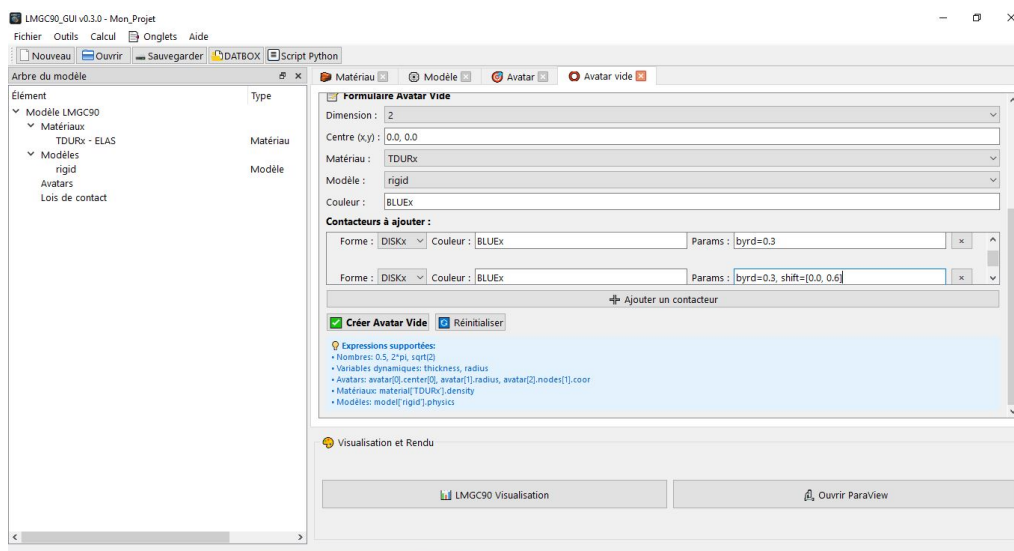
Matériau RIGID : les avatars rigides doivent utiliser un matériau de type `RIGID`. Un matériau élastique peut techniquement être assigné, mais n'a pas d'effet mécanique sur un corps rigide.

Modèle Rxx : les avatars rigides utilisent un modèle avec élément `Rxx2D` (2D) ou `Rxx3D` (3D). Ces éléments n'ont aucune option numérique.

Avatars non utilisés : les avatars dont le matériau ou le modèle a été supprimé restent dans la liste mais génèrent une erreur à la création du script. Une validation est effectuée avant la génération.

Avatar Vide (Personnalisation avancée)

Onglet Avatar vide Permet de créer des **avatars à contacteurs personnalisés** : soit un corps vide (**emptyAvatar**) entièrement défini par ses contacteurs, soit l'ajout de contacteurs à un **corps déformable existant** (maillage FEM). C'est l'outil le plus flexible de LMGC90_GUI pour les cas avancés qui ne rentrent pas dans les types d'avatars rigides standards.



captures/avatar_vide_2disques.JPG

Deux modes de fonctionnement

Le champ **Mode** en haut du formulaire bascule entre deux comportements distincts :

Mode	Description
Avatar vide (emptyAvatar)	Crée un nouveau corps pylmgc90 entièrement assemblé à partir de ses contacteurs. Le corps est rigide mais sa géométrie est définie manuellement.
Corps déformable existant	Ajoute des contacteurs de contact sur un corps déformable (maillage FEM) déjà créé via l'assistant de maillage. Appelle body.addContactors() directement sur l'objet pylmgc90.

Mode 1 — Avatar vide (emptyAvatar)

Principe

Un avatar vide est un corps rigide dont la géométrie n'est pas prédéfinie. pylmgc90 l'assemble en trois étapes :

1. Création d'un corps vide **pre.avatar(dimension=...)**

2. Ajout d'un bulk rigide `pre.rigid2d()` ou `pre.rigid3d()`
3. Ajout d'un nœud principal et des **contacteurs** qui définissent la forme

Les propriétés inertielles (masse, moment d'inertie) sont calculées automatiquement depuis la géométrie des contacteurs via `body.computeRigidProperties()`.

Champs du formulaire

Champ	Description
Dimension	2 ou 3. Détermine la liste de formes de contacteurs disponibles et le format du centre.
Centre (x,y) ou (x,y,z)	Coordonnées du nœud principal de référence. Accepte les expressions Python.
Matériau	Matériau de type RIGID assigné au corps.
Modèle	Modèle avec élément Rxx2D (2D) ou Rxx3D (3D).
Couleur	Couleur pour l'interaction LMGC90 (5 caractères), n'est pas très importante.

Mode 2 — Corps déformable existant

Principe

Ce mode ne crée pas de nouvel avatar — il enrichit un corps déformable (maillage FEM) déjà présent dans le projet en lui ajoutant des contacteurs de surface. Cela est nécessaire pour que le corps déformable puisse interagir avec d'autres corps (rigides ou déformables).

La fonction appelée sur l'objet `pylmgc90` est : `body.addContactors(shape=..., color=..., **params)`.

Champs spécifiques à ce mode

Champ	Description
Corps déformable	Liste déroulante des corps de type MESH_DEFORMABLE présents dans le projet. Le format affiché est <code>#index — géométrie (matériau/modèle)</code> .
Groupe (group=)	Paramètre <code>group</code> passé à <code>addContactors()</code> . Détermine sur quel groupe de nœuds du maillage le contacteur est appliqué (ex : 102 pour le groupe d'index 102). Laisser vide pour appliquer sur tous les nœuds.

Important : le corps déformable doit avoir été créé et reconstruit en mémoire (présent dans `_pylmgc_bodies`) avant d'ajouter des contacteurs. Si le projet vient d'être chargé depuis un fichier JSON sans reconstruction, le corps `pylmgc90` n'existe pas encore en mémoire et l'ajout de contacteurs échouera.

Gestion des contacteurs

Chaque ligne de la liste **Contacteurs à ajouter** correspond à un appel `body.addContactors()`. Cliquer sur **Ajouter un contacteur** pour créer une nouvelle ligne. Cliquer sur **x** pour supprimer une ligne.

Colonnes d'une ligne de contacteur

Colonne	Description
Forme	Type de contacteur pylmgc90. La liste s'adapte au mode (avatar vide ou déformable) et à la dimension.
Couleur	Couleur du contacteur (5 caractères LMGC90). Indépendante de la couleur du corps, mais très importante pour la détection de contact
Params	Paramètres géométriques du contacteur au format <code>cle=valeur, cle=valeur</code> . Rempli automatiquement avec une suggestion selon la forme choisie.

Formes de contacteurs disponibles

Avatar vide 2D — shapes_2d

Forme	Description	Paramètres	Suggestion par défaut
DISKx	Disque 2D	byrd = rayon	byrd=0.3
xKSID	Disque discret 2D	byrd = rayon	byrd=0.3
JONCX	Jonc / ellipse 2D	axe1 = demi-axe long, axe2 = demi-axe court	axe1=1.0, axe2=0.1
POLYG	Polygone 2D	nb_vertices = nombre de sommets, vertices = liste [[x,y],...]	nb_vertices=4, vertices=[[-1., -1.], [1., -1.], [1., 1.], [-1., 1.]]
PT2Dx	Nœud ponctuel 2D (FEM)	aucun	(vide)

Avatar vide 3D — shapes_3d

Forme	Description	Paramètres	Suggestion par défaut
SPHER	Sphère 3D	byrd = rayon	byrd=0.3
PLANx	Plan 3D	axe1, axe2, axe3 = dimensions des axes	axe1=1.0, axe2=1.0, axe3=0.1
CYLND	Cylindre 3D	byrd = rayon, High = hauteur	byrd=0.5, High=1.0
DNLYC	Cylindre creux 3D	byrd = rayon, High = hauteur	byrd=0.5, High=1.0
POLYR	Polyèdre 3D	nb_vertices = nombre de sommets, vertices = liste [[x,y,z],...]	nb_vertices=8, vertices=[[-1,-1,-1], [1,-1,-1], ...]
PT3Dx	Nœud ponctuel 3D (FEM)	aucun	(vide)

Corps déformable 2D — mesh_shapes_2d

Ces formes sont destinées à être ajoutées sur un maillage FEM 2D. Elles définissent des contacteurs de surface pour les interactions rigide-déformable.

Forme	Description	Usage
ALpxx	Contacteur ligne pour maçonnerie FEM 2D	Interactions ALpMECAx (CLALp / MECAX)
CLxx	Contacteur ligne continu 2D	Interactions DKMECAx (disque / MECAX)
DISKL	Disque sur nœud FEM 2D	Interaction disque-disque sur maillage

Forme	Description	Usage
PT2TL	Point de transmission 2D	Couplage nœud-nœud FEM

Corps déformable 3D — mesh_shapes_3d

Forme	Description	Usage
ASpxx	Contacteur surface pour sphères FEM 3D	Interactions SPMECAx (sphère / MECAx 3D)
CSpxx	Contacteur surface continu 3D	Interactions rigide-déformable 3D génériques
PT3Dx	Nœud ponctuel FEM 3D	Couplage nœud-nœud FEM 3D

Détails des paramètres par forme

DISKx / xKSID / SPHER — Disque, disque discret, sphère

byrd=0.3

Paramètre	Description
byrd	Rayon du contacteur (m). Correspond au rayon de contact utilisé dans les détecteurs.

JONCx — Jonc / Ellipse 2D

axe1=1.0, axe2=0.1

Paramètre	Description
axe1	Demi-axe principal (m) — axe long de l'ellipse.
axe2	Demi-axe secondaire (m) — axe court de l'ellipse.

POLYG — Polygone 2D

nb_vertices=4, vertices=[[-1., -1.], [1., -1.], [1., 1.], [-1., 1.]]

Paramètre	Description
nb_vertices	Nombre de sommets du polygone.
vertices	Liste des coordonnées locales des sommets [[x1, y1], [x2, y2], ...] . Les coordonnées sont relatives au centre du corps. Les sommets doivent être dans l'ordre trigonométrique (anti-horaire).

PLANx — Plan 3D

axe1=1.0, axe2=1.0, axe3=0.1

Paramètre	Description
axe1	Dimension selon le premier axe du plan (m).
axe2	Dimension selon le deuxième axe du plan (m).
axe3	Épaisseur du plan (m) — utilisée pour le calcul des propriétés inertielles.

CYLND / DNLYC — Cylindre 3D

```
byrd=0.5, High=1.0
```

Paramètre	Description
byrd	Rayon du cylindre (m).
High	Hauteur (longueur axiale) du cylindre (m). Notez la majuscule.

POLYR — Polyèdre 3D

```
nb_vertices=8, vertices=[[-1., -1., -1.], [1., -1., -1.], [1., 1., -1.], [-1., 1., -1.],
[-1., -1., 1.], [1., -1., 1.], [1., 1., 1.], [-1., 1., 1.]]
```

Paramètre	Description
nb_vertices	Nombre de sommets du polyèdre.
vertices	Liste des coordonnées 3D de chaque sommet <code>[[x, y, z], ...]</code> . Coordonnées locales relatives au centre.

Pour un polyèdre convexe, les sommets peuvent être fournis dans n'importe quel ordre — pylmgc90 calcule l'enveloppe convexe. Pour un polyèdre non convexe, l'ordre des faces doit être cohérent.

PT2Dx / PT3Dx — Nœuds ponctuels

Aucun paramètre. Ces contacteurs représentent un point de contact en un nœud.

```
(champ Params vide)
```

Exemples complets

Avatar vide 2D — corps polygonal à 6 sommets

```
Mode : Avatar vide (emptyAvatar)
Dimension : 2
Centre : 0.0, 0.5
Matériau : BRIQx
Modèle : rigid
Couleur : REDxx

Contacteur 1 :
Forme : POLYG
Couleur: REDxx
Params : nb_vertices=6, vertices=[[-0.1, -0.05], [0.1, -0.05], [0.15, 0.0],
```

```
[0.1,0.05],[-0.1,0.05],[-0.15,0.0]]
```

Avatar vide 3D — corps avec sphère et cylindre

```
Mode : Avatar vide (emptyAvatar)
Dimension : 3
Centre : 0.0, 0.0, 0.5
Matériau : ACIER
Modèle : rig3D
Couleur : CYANx

Contacteur 1 :
Forme : SPHER
Couleur: CYANx
Params : byrd=0.2

Contacteur 2 :
Forme : CYLND
Couleur: GRAYx
Params : byrd=0.05, High=0.8
```

Ajout de contacteurs à un corps déformable

```
Mode : Corps déformable existant
Corps déformable : #3 – Rectangle (beton/MECAx)
Groupe (group=) : 102

Contacteur 1 :
Forme : CLxx
Couleur: BLUEx
Params : (vide)
```

Interface — liste des avatars vides

La liste en haut de l'onglet affiche uniquement les avatars de type **EMPTY_AVATAR**. Les colonnes sont :

Colonne	Description
#	Index de l'avatar dans la liste globale des avatars du projet.
Couleur	Code couleur LMGC90 du corps.
Centre	Coordonnées du centre de référence arrondies à 2 décimales.
Contacteurs	Nombre de contacteurs définis sur ce corps.

Menu contextuel (clic droit) :

- **Modifier** — charge l'avatar dans le formulaire pour édition.
- **Supprimer** — supprime l'avatar après confirmation. Refusé si l'avatar est référencé par une boucle ou un groupe.
- **Informations** — affiche une boîte de dialogue avec le détail de tous les contacteurs.

Remarques importantes

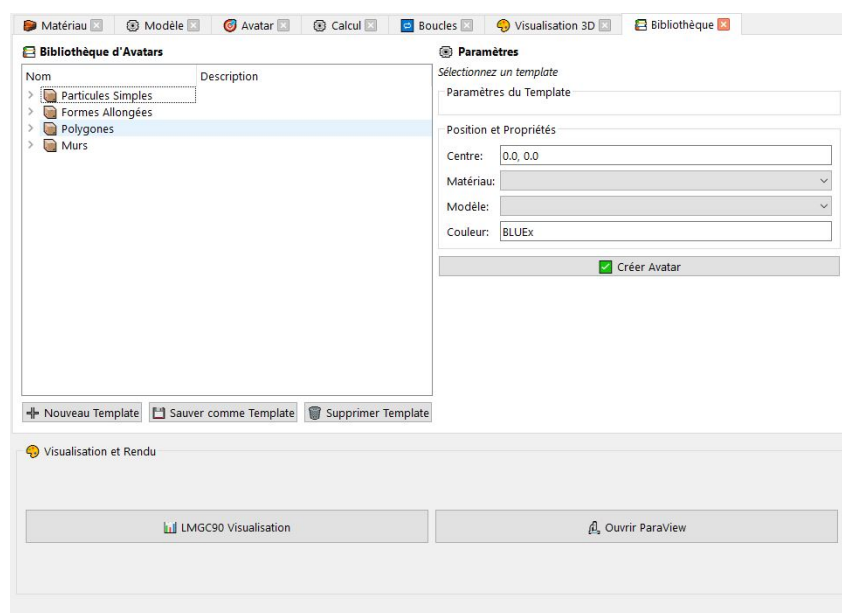
Un avatar vide nécessite au moins un contacteur. La création est refusée si la liste de contacteurs est vide.

Contacteurs multiples. Un avatar vide peut avoir autant de contacteurs que nécessaire, de formes différentes. Chaque contacteur génère une ligne `body.addContactors(...)` distincte dans le script.

Bibliothèque — Avatars préconfigurés

L'onglet **Bibliothèque** (■) met à disposition un catalogue d'avatars préconfigurés, organisés par catégorie. Il permet d'insérer rapidement des formes courantes dans le projet sans avoir à saisir manuellement les paramètres géométriques.

Cet onglet se **rafraîchit automatiquement** à chaque changement de dimension dans l'onglet Modèle : les templates disponibles s'adaptent à la dimension courante du projet (2D ou 3D).



Vue générale de l'onglet Bibliothèque

Organisation de l'interface

L'onglet est divisé en deux panneaux :

Panneau	Description
Gauche — Arbre des templates	Arborescence des avatars disponibles, organisés par catégorie. Cliquer sur une entrée pour afficher ses propriétés à droite.
Droite — Propriétés	Affiche le nom, le type d'avatar, la description et les paramètres par défaut du template sélectionné. Permet également de configurer le centre, le matériau, le modèle et la couleur avant insertion.

Templates disponibles

Les templates sont organisés par catégorie. La liste change selon que la dimension du projet est **2D** ou **3D**.

Templates 2D

Particules simples

Template	Type pylmgc90	Paramètres par défaut	Description
Petit Disque	<code>rigidDisk</code>	<code>radius = 0.05 m</code>	Disque rigide de petit rayon, typique des simulations granulaires fines.
Disque Moyen	<code>rigidDisk</code>	<code>radius = 0.10 m</code>	Disque rigide de rayon intermédiaire.
Grand Disque	<code>rigidDisk</code>	<code>radius = 0.20 m</code>	Disque rigide de grand rayon, pour les particules de taille importante.

Formes allongées

Template	Type pylmgc90	Paramètres par défaut	Description
Cylindre Horizontal	<code>rigidJonc</code>	<code>axe1 = 2.0 m, axe2 = 0.1 m</code>	Jonc elliptique allongé horizontalement, rapport d'aspect 2:1.
Cylindre Vertical	<code>rigidJonc</code>	<code>axe1 = 2.0 m, axe2 = 0.1 m</code>	Jonc elliptique allongé verticalement.

`axe1` est le demi-axe long, `axe2` le demi-axe court (mêmes paramètres que `rigidJonc` dans l'onglet Avatar).

Polygones réguliers

Template	Type pylmgc90	Paramètres par défaut	Description
Triangle	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=3, radius=0.1 m</code>	Triangle équilatéral inscrit dans un cercle de rayon 0,1 m.
Carré	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=4, radius=0.1 m</code>	Carré régulier inscrit dans un cercle de rayon 0,1 m.
Pentagone	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=5, radius=0.1 m</code>	Pentagone régulier.
Hexagone	<code>rigidPolygon</code>	<code>generation_type='regular', nb_vertices=6, radius=0.1 m</code>	Hexagone régulier — forme très utilisée en DEM pour les assemblages compacts.
Rectangle	<code>rigidPolygon</code>	<code>generation_type='full', sommets définis explicitement, radius=0.15 m</code>	Rectangle 0,30 m × 0,10 m défini par liste de sommets.

Murs

Template	Type pylmgc90	Paramètres par défaut	Description
Mur Horizontal	<code>fineWall</code>	<code>l=2.0 m, r=0.1 m, nb_polyg=20</code>	Mur horizontal fin de longueur 2 m, composé de 20 segments polygonaux.

Templates 3D

Particules simples

Template	Type pylmgc90	Paramètres par défaut	Description
Petite Sphère	<code>rigidSphere</code>	<code>radius = 0.05 m</code>	Sphère rigide de petit rayon.
Sphère Moyenne	<code>rigidSphere</code>	<code>radius = 0.10 m</code>	Sphère rigide de rayon intermédiaire.
Grande Sphère	<code>rigidSphere</code>	<code>radius = 0.20 m</code>	Sphère rigide de grand rayon.

Formes 3D

Template	Type pylmgc90	Paramètres par défaut	Description
Cylindre 3D	<code>rigidCylinder</code>	<code>radius=0.05 m, h=0.2 m</code>	Cylindre droit de rayon 0,05 m et de hauteur 0,2 m.
Plan Sol	<code>rigidPlan</code>	<code>axe1=2.0 m, axe2=2.0 m, axe3=0.1 m</code>	Plan horizontal rigide de 2 m × 2 m, utilisé comme sol ou plafond.

Assemblages complexes

En plus des templates unitaires, la bibliothèque propose des **assemblages de plusieurs avatars** générés en une seule action. Ces assemblages créent plusieurs corps coordonnés dans le projet.

Cluster de disques 2D

Crée un corps rigide 2D de type `rigidCluster` — une agrégation de disques liés rigidement, formant une particule non convexe.

Paramètre	Description	Valeur par défaut
Centre	Position du centre de référence du cluster.	<code>[0.0, 0.0]</code>
Matériau	Matériau de type <code>RIGID</code> .	—
Modèle	Modèle avec élément <code>Rxx2D</code> .	—
Rayon principal	Rayon de chaque disque composant le cluster (m).	<code>0.1 m</code>
Nombre de disques (<code>nb_disk</code>)	Nombre de disques dans le cluster.	<code>5</code>

Haltère 2D `_(dumbbell)_`

Crée un avatar vide (`emptyAvatar`) composé de trois contacteurs : deux disques (`DISKx`) aux extrémités et un jonc (`JONCx`) au centre, formant une forme en haltère.

Paramètre	Description	Valeur par défaut
Centre	Position du centre de l'haltère.	<code>[0.0, 0.0]</code>
Longueur totale	Distance entre les centres des deux disques (m).	<code>0.3 m</code>
Rayon des disques	Rayon des deux extrémités (m).	<code>0.05 m</code>

Contacteurs générés automatiquement :

Contacteur	Forme	Paramètres
Disque gauche	DISKx	byrd = rayon, positionné à $-longueur/2$
Disque droit	DISKx	byrd = rayon, positionné à $+longueur/2$
Corps central	JONCx	axe1 = longueur, axe2 = rayon $\times 0.3$

Boîte rectangulaire 2D (box container)

Crée un conteneur rectangulaire ouvert en haut, composé de **3 murs lisses** (`smoothWall`) : un mur inférieur et deux murs latéraux.

Paramètre	Description	Valeur par défaut
Largeur totale	Dimension horizontale intérieure de la boîte (m).	—
Hauteur	Dimension verticale de la boîte (m).	—
Épaisseur des murs	Épaisseur de chaque paroi (m).	—
Centre	Position du centre de la boîte.	—

Corps créés :

Corps	Type	Position
Mur bas	<code>smoothWall</code>	$centre_y - hauteur/2$
Mur gauche	<code>smoothWall</code>	$centre_x - largeur/2$
Mur droit	<code>smoothWall</code>	$centre_x + largeur/2$

Trémie en V 2D (hopper)

Crée une trémie conique composée de **2 parois inclinées** (`rigidPolygon` avec `generation_type='full'`), formant un entonnoir ouvert en haut et en bas.

Paramètre	Description
Largeur en haut	Ouverture supérieure de la trémie (m).
Largeur en bas	Ouverture inférieure de la trémie (m).
Hauteur	Hauteur totale de la trémie (m).
Centre	Position du centre géométrique.

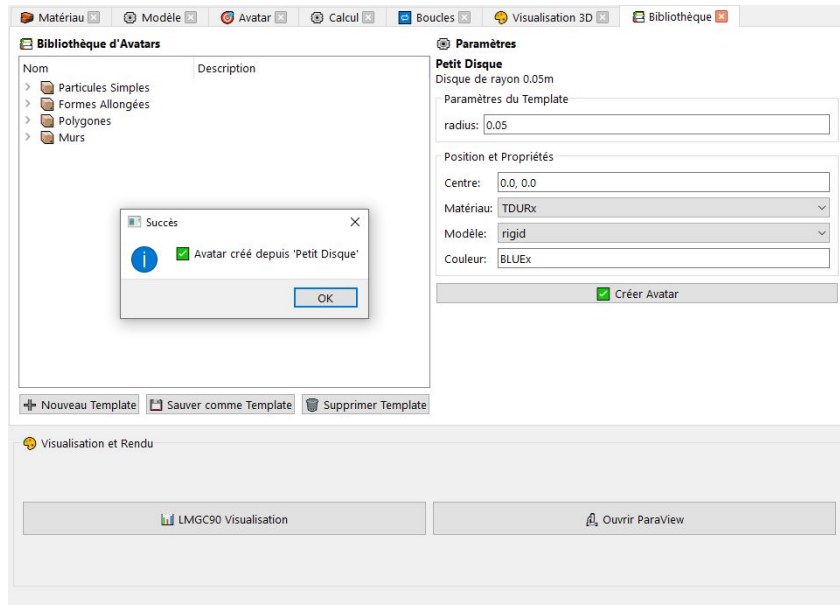
Corps créés : paroi gauche et paroi droite, chacune définie par 4 sommets calculés automatiquement à partir des dimensions.

Créer un avatar depuis un template

- Sélectionner un template dans l'arbre de gauche — ses propriétés s'affichent dans le panneau de droite.
- Configurer les champs dans le panneau droit :
 - Centre** : coordonnées d'insertion (x, y) en 2D ou (x, y, z) en 3D. Accepte les expressions Python.
 - Matériau** : sélectionner parmi les matériaux définis dans l'onglet Matériau.

- **Modèle** : sélectionner parmi les modèles définis dans l'onglet Modèle.
 - **Couleur** : code couleur LMGC90 à 5 caractères.
3. Cliquer sur **Créer Avatar**.

L'avatar est ajouté à la liste du projet et apparaît dans l'arbre du modèle.

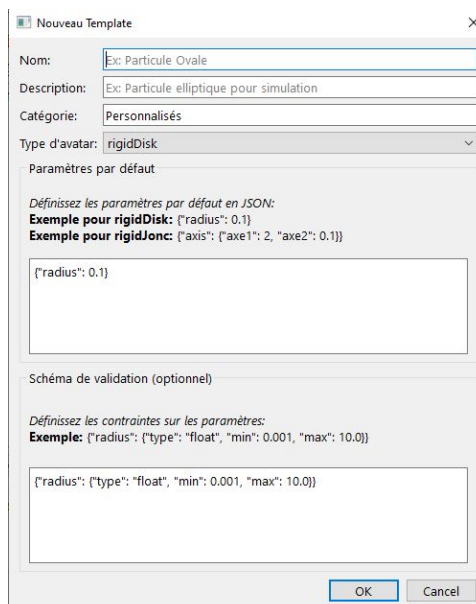


Création d'un avatar depuis un template

Créer un nouveau template

Il est possible d'enregistrer un avatar existant comme nouveau template pour le réutiliser dans d'autres projets.

1. Cliquer sur le bouton **Nouveau template**.
2. La boîte de dialogue de création de template s'ouvre.
3. Renseigner le nom et la description du template.
4. Les paramètres géométriques de l'avatar sélectionné sont repris automatiquement.
5. Valider pour ajouter le template à la bibliothèque.



Remarques importantes

Dimension courante : la liste des templates s'adapte automatiquement à la dimension du projet. Si la dimension est modifiée dans l'onglet Modèle, l'onglet Bibliothèque se rafraîchit et n'affiche que les templates compatibles (2D ou 3D).

Matériau et modèle obligatoires : un matériau et un modèle doivent être définis dans le projet avant de pouvoir créer un avatar depuis un template. Si aucun matériau ou modèle n'est disponible, un message d'erreur s'affiche.

Paramètres par défaut : les templates utilisent des valeurs par défaut raisonnables. Ces valeurs doivent être ajustées selon les unités et l'échelle du modèle. En particulier, les rayons et longueurs en mètres doivent être cohérents avec la granulométrie et les dimensions de la scène.

Assemblages : les assemblages complexes (boîte, trémie, haltère) créent plusieurs avatars simultanément dans le projet. Ils apparaissent tous dans l'arbre du modèle et peuvent être modifiés individuellement après leur création.

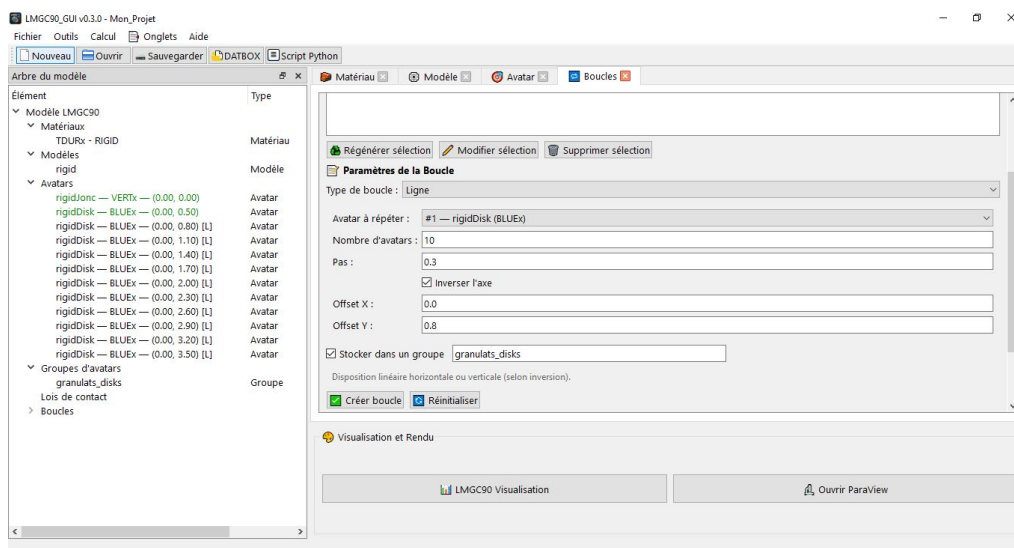
Expressions Python : le champ Centre accepte les expressions Python évaluées via `SafeEvaluator`, comme dans tous les autres onglets de l'interface : `avatar[0].x + 0.5`, `math.sqrt(2) * r`, etc.

Génération paramétrique d'avatars (boucles)

L'onglet **Boucles** (**Ctrl+6**) permet de créer automatiquement des séries d'avatars selon des motifs géométriques ou des expressions Python. Il propose deux mécanismes complémentaires :

- **Boucles géométriques** : positionnement automatique d'un avatar modèle selon un motif (Cercle, Grille, Ligne, Spirale).
- **Boucles for génériques** : génération d'avatars, matériaux, modèles, lois de contact, tables de visibilité ou opérations DOF via des expressions Python évaluées à chaque itération.

Les boucles créées sont sauvegardées dans le projet et reconstituées lors du rechargement. Elles sont également exportées dans le script Python généré.



captures/boucle_disk_ligne.JPG

Partie 1 — Boucles géométriques

Principe

Une boucle géométrique prend un **avatar modèle** déjà défini dans l'onglet Avatar, calcule une liste de positions selon le motif choisi, puis crée une copie identique de l'avatar à chaque position. Tous les attributs de l'avatar modèle sont copiés (type, matériau, modèle, rayon, couleur, etc.).

Champs du formulaire

Champ	Description
Avatar modèle	Index de l'avatar dont les propriétés seront copiées à chaque position. Sélectionner dans la liste déroulante des avatars existants.
Type de boucle	Motif géométrique de placement. Voir les 4 types ci-dessous.

Champ	Description
Nombre d'éléments	Nombre d'avatars à créer.
Offset X	Décalage du centre du motif en X (m).
Offset Y	Décalage du centre du motif en Y (m).
Groupe	Nom du groupe d'avatars dans lequel stocker les avatars générés. Laissé vide pour ne pas créer de groupe.

Types de boucles géométriques

Cercle

Répartit **count** avatars uniformément sur un cercle de rayon **radius**, centré en (**offset_x**, **offset_y**).

Paramètre	Description	Défaut
Rayon	Rayon du cercle de placement (m).	—
Offset X / Y	Centre du cercle.	0.0

Usage typique : particules en anneau, tambour rotatif, couronne de boulons, granulométrie circulaire.

Grille

Répartit **count** avatars sur une grille carrée de côté, avec un espacement **step** entre éléments.

Paramètre	Description	Défaut
Pas (step)	Distance entre deux avatars voisins sur la grille (m).	—
Offset X / Y	Coin inférieur gauche de la grille.	0.0

*La grille est toujours carrée. Si **count** n'est pas un carré parfait, le nombre réel d'avatars créés est n_side^2 . Exemple : **count=10** → grille 4×4 = 16 avatars.*

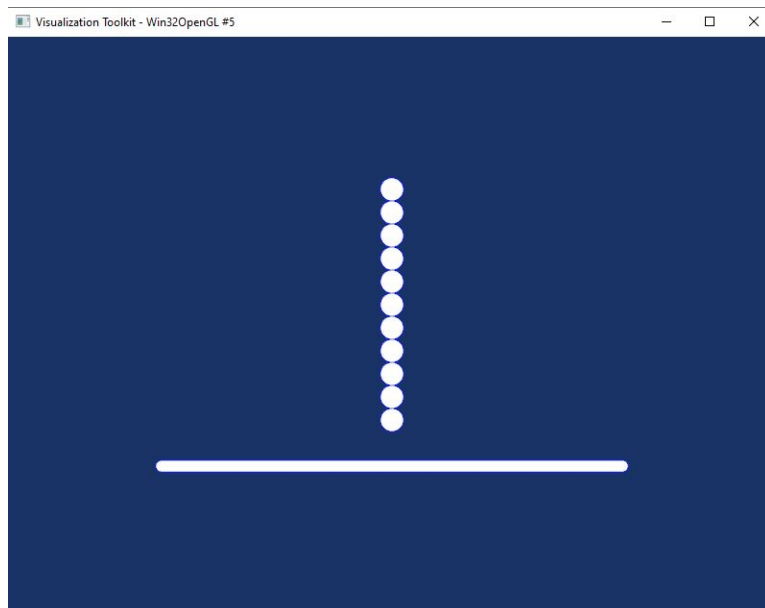
Usage typique : empilement régulier de particules, plaque perforée, réseau carré.

Ligne

Aligne **count** avatars en ligne droite, séparés d'un pas **step**, dans la direction X ou Y.

Paramètre	Description	Défaut
Pas (step)	Distance entre deux avatars consécutifs (m).	—
Offset X / Y	Position du premier avatar.	0.0
Inverser l'axe	Si coché, la ligne est orientée selon Y (verticale) au lieu de X (horizontale).	décoché

Usage typique : rangée de poteaux, file de particules, mur simple.



captures/rendu_boucle_disk_ligne.JPG

Spirale

Dispose **count** avatars sur une spirale d'Archimède, de rayon initial **radius** qui croît d'un facteur **spiral_factor** à chaque itération.

Paramètre	Description	Défaut
Rayon initial	Rayon de départ de la spirale (m).	—
Facteur de spirale	Incrément de rayon par élément (m).	—
Offset X / Y	Centre de la spirale.	0.0

Usage typique : mélanges, dépôts en spirale, arrangements décoratifs.

Gestion des boucles géométriques

Créer une boucle

Remplir le formulaire et cliquer sur **■ Générer la boucle**. Les avatars sont créés immédiatement dans le projet.

Modifier une boucle

Sélectionner une boucle dans la liste, modifier les paramètres et cliquer sur **■ Mettre à jour**. Les anciens avatars sont supprimés et recréés avec les nouveaux paramètres.

Supprimer une boucle

Sélectionner une boucle dans la liste et cliquer sur **■ Supprimer**. Tous les avatars générés par cette boucle sont supprimés automatiquement.

Suppression en cascade : la suppression d'une boucle supprime **tous** les avatars qu'elle a générés (indices enregistrés dans **loop.generated_indices**), dans l'ordre inverse pour ne pas décaler les autres indices.

Partie 2 — Boucles **for** génériques

Principe

Les boucles **for** génériques permettent de créer des séries d'éléments en faisant varier une **variable de boucle** sur un intervalle défini par des expressions Python. Le **type d'élément** à créer est choisi parmi : avatar, matériau, modèle, loi de contact, table de visibilité, ou opération DOF.



captures/for_generique.JPG

À chaque itération, le **template de configuration** est évalué avec la variable de boucle injectée dans le contexte d'évaluation.

Champs du formulaire

Champ	Description	Exemple
Variable de boucle	Nom de la variable Python disponible dans le template.	i, k, n
Début	Expression Python pour la valeur de départ.	0, 1, n_start
Fin	Expression Python pour la valeur de fin (exclue).	10, count, n_start + 5
Pas	Expression Python pour l'incrément.	1, 2, -1
Type d'élément	Type de l'objet à créer à chaque itération.	voir tableau ci-dessous
Template	Configuration de l'élément, avec la variable de boucle utilisable dans les valeurs.	voir sections par type
Groupe	Nom du groupe d'avatars (avatars uniquement).	ma_ligne

Contexte d'évaluation disponible

Les expressions dans le template et les bornes de la boucle ont accès aux éléments suivants :

Symbole	Description
i (ou la variable de boucle)	Valeur courante de l'itération
math	Module math Python complet
sqrt, pi, e	Raccourcis math

Symbole	Description
<code>abs, min, max, sum, len</code>	Fonctions Python standard
<code>str, int, float</code>	Conversions de type
Variables dynamiques	Toutes les variables définies dans le menu Outils → Variables dynamiques

Types d'éléments disponibles

Type `avatar` — Création d'avatars

Crée un avatar à chaque itération. Tous les paramètres de l'onglet Avatar sont disponibles dans le template.

Paramètres du template :

Clé	Description	Exemple
<code>avatar_type</code>	Type <code>pymgc90</code> de l'avatar.	<code>"rigidDisk"</code>
<code>center</code>	Expression Python retournant une liste <code>[x, y]</code> ou <code>[x, y, z]</code> . La variable de boucle peut être utilisée.	<code>[i * 0.1, 0.0]</code>
<code>material_name</code>	Nom du matériau (chaîne).	<code>"TDURx"</code>
<code>model_name</code>	Nom du modèle (chaîne).	<code>"rigid"</code>
<code>color</code>	Code couleur LMGC90 (5 caractères).	<code>"BLUEx"</code>
<code>radius</code>	Expression Python pour le rayon (m).	<code>"0.05 + i * 0.01"</code>
<code>axis</code>	Dict <code>{axe1: expr, axe2: expr}</code> pour les joncs / plans.	<code>{"axe1": "1.0", "axe2": "0.1"}</code>
<code>nb_vertices</code>	Expression pour le nombre de sommets (polygones).	<code>"6"</code>
<code>generation_type</code>	<code>"regular", "full" ou "bevel"</code> .	<code>"regular"</code>
<code>vertices</code>	Expression Python retournant la liste des sommets.	<code>"[[-0.1, -0.1], [0.1, -0.1], [0.1, 0.1], [-0.1, 0.1]]"</code>
<code>wall_params</code>	Dict des paramètres de mur.	<code>{"l": "2.0", "r": "0.05"}</code>
<code>contactors</code>	Liste de contacteurs (format identique à l'onglet Avatar vide).	
<code>is_hollow</code>	Booléen — crée un disque creux.	<code>false</code>

Script généré :

```
for i in range(0, 10, 1):
    center = [i * 0.1, 0.0]
    body = pre.rigidDisk(
        center=center,
        model=mods['rigid'],
        material=mats['TDURx'],
        color='BLUEx',
        r=0.05 + i * 0.01
    )
    bodies.addAvatar(body)
    bodies_list.append(body)
```

Usage typique : ligne de disques avec rayons croissants, grille d'avatars hétérogènes, série de corps avec des paramètres qui varient.

Type `material` — Création de matériaux

Crée un matériau à chaque itération. Utile pour générer une famille de matériaux avec des propriétés graduées.

Paramètres du template :

Clé	Description	Exemple
<code>name</code>	Expression Python pour le nom (chaîne, 5 car. max).	<code>"str('mat') + str(i)"</code>
<code>material_type</code>	Type de matériau LMGC90.	<code>"RIGID"</code>
<code>density</code>	Expression Python pour la densité (kg/m³).	<code>"2800 + i * 100"</code>
<code>properties</code>	Dict de propriétés supplémentaires.	<code>{"young": "1e9 + i * 1e8"}</code>

Script généré :

```
for i in range(0, 5, 1):
    mat_name = str('mat') + str(i)
    density_val = 2800 + i * 100
    mats[mat_name] = pre.material(
        name=mat_name,
        materialType='RIGID',
        density=density_val
    )
    materials.addMaterial(mats[mat_name])
```

Type `model` — Création de modèles

Crée un modèle éléments finis à chaque itération.

Paramètres du template :

Clé	Description	Exemple
<code>name</code>	Expression Python pour le nom.	<code>"'mod' + str(i)"</code>
<code>physics</code>	Physique.	<code>"MECAx"</code>
<code>element</code>	Élément fini.	<code>"Rxx2D"</code>
<code>dimension</code>	Dimension (2 ou 3).	<code>2</code>
<code>options</code>	Dict des options numériques.	<code>{}</code>

Type `contactLaw` — Création de lois de contact

Crée une loi de contact à chaque itération. Utile pour générer des lois avec des coefficients de friction différents.

Paramètres du template :

Clé	Description	Exemple
<code>name</code>	Expression Python pour le nom.	<code>"'law' + str(i)"</code>
<code>law_type</code>	Type de loi (<code>IQS_CLB</code> , etc.).	<code>"IQS_CLB"</code>
<code>friction</code>	Expression Python pour le coefficient de friction.	<code>"0.1 + i * 0.05"</code>

Script généré :

```
for i in range(0, 5, 1):
    law_name = 'law' + str(i)
    laws[law_name] = pre.tact_behav(
        name=law_name,
        law='IQS_CLB',
        fric=0.1 + i * 0.05
    )
    tacts.addBehav(laws[law_name])
```

Type visibility — Création de tables de visibilité

Crée une règle de visibilité (table de détection de contact) à chaque itération.

Paramètres du template :

Clé	Description	Exemple
candidate_body	Corps candidat (RBDY2, RBDY3).	"RBDY2"
candidate_contactor	Contacteur candidat (DISKx, etc.).	"DISKx"
candidate_color	Expression pour la couleur candidat.	"'BLUEx'"
antagonist_body	Corps antagoniste.	"RBDY2"
antagonist_contactor	Contacteur antagoniste.	"DISKx"
antagonist_color	Expression pour la couleur antagoniste.	"'REDxx'"
behavior_name	Nom de la loi de contact associée.	"'law' + str(i)"
alert	Distance d'alerte (m).	"0.05 + i * 0.01"

Type dof — Opérations DOF

Applique une condition aux limites (degré de liberté imposé) à un ou plusieurs avatars à chaque itération.

Paramètres du template :

Clé	Description	Exemple
operation_type	Type d'opération pylmgc90.	"imposeDrivenDof"
target_type	Toujours "avatar".	"avatar"
target_value	Expression Python pour l'index de l'avatar cible.	"i", "i + 10"
parameters	Dict des kwargs pylmgc90.	{"component": "[1, 2]", "dofty": "\"vlocy\""}"

Script généré :

```
for i in range(0, 5, 1):
    bodies_list[i].imposeDrivenDof(component=[1, 2], dofty="vlocy")
```

Gestion des boucles for

Les boucles **for** sont listées séparément des boucles géométriques. Les mêmes opérations sont disponibles : créer, modifier (régénère les éléments), supprimer (supprime les éléments générés).

Groupes d'avatars

Les deux types de boucles peuvent stocker les avatars générés dans un **groupe nommé**. Un groupe est une liste d'indices d'avatars accessible dans tout le projet sous le nom donné.

Les groupes permettent :

- d'appliquer une condition DOF à tous les avatars d'un groupe d'un seul coup
- de cibler un ensemble d'avatars dans les tables de visibilité
- de distinguer plusieurs populations d'avatars pour l'extraction de données (chipy `GetBodyVector`)

Si le champ Groupe est laissé vide, aucun groupe n'est créé et les avatars sont simplement ajoutés à la liste globale.

Récapitulatif des paramètres par type de boucle géométrique

Type	Paramètres requis	Paramètre optionnel	Formule
Cercle	count, radius	offset_x, offset_y	$x = \text{offset_x} + \text{radius} \times \cos(2\pi i / \text{count})$
Grille	count, step	offset_x, offset_y	$x = \text{offset_x} + (i \% n_side) \times \text{step}$
Ligne	count, step	offset_x, offset_y, invert_axis	$x = \text{offset_x} + i \times \text{step}$ (ou Y si inversé)
Spirale	count, radius, spiral_factor	offset_x, offset_y	$r = \text{radius} + i \times \text{spiral_factor}$

Remarques importantes

Avatar modèle non supprimable : un avatar utilisé comme modèle par une boucle géométrique ne peut pas être supprimé tant que la boucle existe. Un message d'avertissement est affiché.

Mise à jour en cascade : modifier une boucle (géométrique ou `for`) supprime et recrée tous ses avatars. Si d'autres éléments du projet référencent ces avatars (conditions DOF, tables de visibilité), ils peuvent être invalidés.

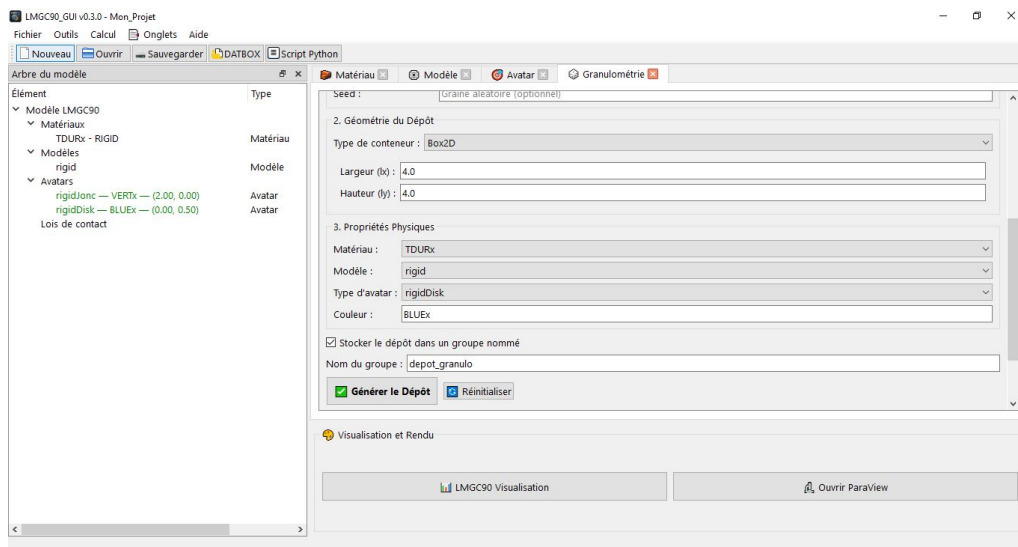
Expressions Python : dans les boucles `for`, les expressions sont évaluées via `SafeEvaluator`. Les expressions malformées génèrent un message d'erreur sans bloquer le projet. Les variables dynamiques définies dans **Outils** → **Variables dynamiques** (`Ctrl+V`) sont accessibles dans toutes les expressions.

Dimension 3D : les boucles géométriques génèrent des centres 2D `[x, y]` ou 3D `[x, y, z]` selon la dimension du projet. Les avatars 3D (sphères, polyèdres) peuvent être générés en boucle `for` avec des centres explicitement 3D dans le template.

Nombre d'éléments et performance : créer plus de quelques milliers d'avatars via une boucle peut ralentir l'interface. Pour les grands assemblages (> 5 000 avatars), utiliser le générateur de granulométrie ou les assistants dédiés.

Granulométrie

L'onglet **Granulométrie** (**Ctrl+7**) permet de créer, gérer et supprimer des distributions granulaires directement depuis l'interface, sans passer par l'assistant. Il est adapté pour des générations rapides à partir d'avatars et de matériaux déjà définis dans le projet.



Exemple de dépôt granulaire disques et joncs

Interface générale

L'onglet est divisé en deux zones :

- **Liste des distributions** (en haut) : tableau affichant toutes les générations granulométriques du projet avec leur type de conteneur, le nombre de particules, les rayons min/max et le groupe associé. Double-clic pour éditer. Clic droit pour accéder au menu contextuel.
- **Formulaire de création / modification** (en bas) : champs de configuration de la distribution.

Champs du formulaire

Avatar modèle

Champ	Description
Avatar modèle	Sélection d'un avatar rigide existant dans le projet (défini dans l'onglet Avatar). Tous ses attributs sont copiés sur chaque particule générée : type (rigidDisk ou rigidSphere), matériau, modèle, couleur.

L'avatar modèle doit être de type **rigidDisk** (2D) ou **rigidSphere** (3D). C'est lui qui définit indirectement le matériau, le modèle et la couleur de toutes les particules générées.

Distribution des particules

Champ	Description	Plage	Défaut
Nombre de particules	Nombre de particules demandé à l'algorithme de dépôt. Le nombre réellement placé peut être inférieur si le conteneur est saturé.	10 à 10 000	200
Rayon minimum (rmin)	Rayon minimal des particules (m).	0,001 à 10,0	0.05 m
Rayon maximum (rmax)	Rayon maximal des particules (m). Doit être strictement supérieur à rmin.	0,001 à 10,0	0.15 m

Performance : au-delà de ~1 500 avatars, le rafraîchissement de l'interface peut ralentir significativement. Pour les grands assemblages, utiliser l'**assistant granulométrie** (Ctrl+Shift+G) ou la **génération numpy** (menu Assistants).

Reproductibilité

Champ	Description	Défaut
Utiliser une graine	Case à cocher. Si activée, les rayons et positions sont générés de façon identique à chaque exécution.	décoché
Valeur de la graine	Entier de 0 à 999 999. Transmis à <code>pre.granulo_Random(seed=...)</code> .	12345

Conteneur de dépôt

Sélection du type de conteneur et de ses dimensions. Les conteneurs 2D sont disponibles quel que soit la dimension du projet — adapter en fonction du type d'avatar modèle choisi.

Conteneurs 2D

Conteneur	Paramètres	Fonction pylmgc90	Description
Box2D	lx (m), ly (m)	<code>pre.depositInBox2D(radii, lx, ly)</code>	Boîte rectangulaire. Dépôt gravitaire depuis le haut.
Disk2D	r (m)	<code>pre.depositInDisk2D(radii, r)</code>	Disque circulaire.
Couette2D	rint (m), rext (m)	<code>pre.depositInCouette2D(radii, rint, rext)</code>	Cellule de Couette — espace annulaire entre deux cylindres.
Drum2D	r (m)	<code>pre.depositInDrum2D(radii, r)</code>	Tambour rotatif circulaire.

Conteneurs 3D

Conteneur	Paramètres	Fonction pylmgc90	Description
Box3D	<code>lx, ly, lz</code> (m)	<code>pre.depositInBox3D(radii, lx, ly, lz)</code>	Boîte parallélépipédique.
Sphere3D	<code>r</code> (m)	<code>pre.depositInSphere3D(radii, r)</code>	Sphère 3D.
Cylinder3D	<code>r</code> (m)	<code>pre.depositInCylinder3D(radii, r)</code>	Cylindre 3D.

Options

Champ	Description	Défaut
Couleur	Code couleur LMGC90 à 5 caractères. Écrase la couleur de l'avatar modèle si renseigné.	BLUEx
Stocker dans un groupe	Si coché, tous les avatars générés sont enregistrés dans un groupe nommé dans <code>state.avatar_groups</code> .	coché
Nom du groupe	Identifiant du groupe. Par défaut automatique : <code>granulo_{type_conteneur}</code> (ex. <code>granulo_box2d</code>).	<code>granulo_box2d</code>

Générer une distribution

Cliquer sur **■ Générer le dépôt** pour lancer la génération. L'algorithme :

1. Tire aléatoirement `nb` rayons dans `[rmin, rmax]` via `pre.granulo_Random`.
2. Positionne les particules dans le conteneur choisi via `pre.depositInXxx`.
3. Crée un avatar `rigidDisk` ou `rigidSphere` pour chaque particule placée.
4. Enregistre la configuration dans `state.granulo_generations`.
5. Émet le signal `granulo_generated` pour rafraîchir l'interface.

Le signal `granulo_generated` déclenche un rafraîchissement complet de l'arbre du modèle et de tous les onglets.

Gestion des distributions

Modifier une distribution

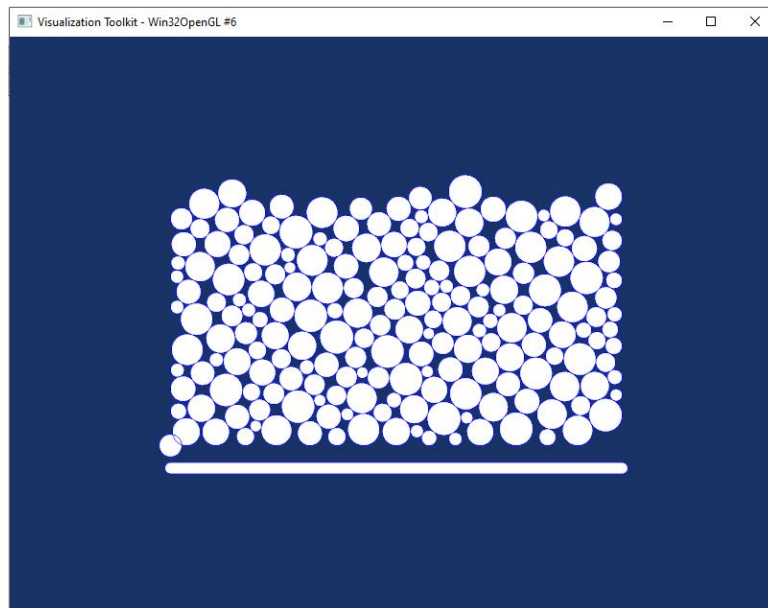
Sélectionner une distribution dans la liste et cliquer sur **—■ Modifier**. La configuration est chargée dans le formulaire en mode **Édition**. Modifier les paramètres et cliquer sur **■ Enregistrer** pour régénérer. Les anciens avatars sont supprimés et remplacés.

Supprimer une distribution

Sélectionner et cliquer sur **■ Supprimer**. Tous les avatars générés par cette distribution sont supprimés automatiquement (indices enregistrés dans `GranuloGeneration.generated_indices`). Le signal `granulo_deleted` est émis.

Exemple de configuration

Champ	Valeur exemple
Avatar modèle	#0 — rigidDisk / TDURx / rigid
Nombre de particules	400
Rayon minimum	0.05 m
Rayon maximum	0.075 m
Conteneur	Box2D — lx=4.0 m, ly=4.0 m
Couleur	BLUEx
Groupe	granulo_box2d



Rendu du dépôt granulaire

Remarques

Nombre réel vs demandé : le nombre de particules réellement placées (`_nb_remaining`) peut être inférieur au nombre demandé si le conteneur est trop petit ou la densité d'empilement est atteinte.

Limite de performance : l'onglet appelle `add_avatar()` pour chaque particule, ce qui émet un signal par avatar. Au-delà de ~1 500 avatars, l'interface ralentit. Pour des assemblages plus grands, utiliser l'**assistant granulométrie** qui insère les avatars directement dans les conteneurs `pymgc90` sans signal individuel.

Groupe automatique : si aucun nom de groupe n'est saisi, un nom est généré automatiquement sous la forme `granulo_{type_conteneur}`.

Assistant de granulométrie — `pymgc90`

L'**Assistant de distribution granulométrique** guide pas à pas la création d'un dépôt de particules rigides (disques 2D ou sphères 3D) via l'API officielle `pymgc90`. Il crée ou réutilise automatiquement le matériau et le modèle, configure la distribution des rayons et le conteneur de dépôt, puis génère tous les avatars d'un seul coup de façon optimisée.

Accès : menu **Assistants** → **Assistant de granulométrie** · raccourci **Ctrl+Shift+G**

Recommandé pour : jusqu'à environ **8 000 particules**. Au-delà, l'interface peut ralentir. Utiliser la génération numpy pour les très grands assemblages.

Vue d'ensemble des étapes

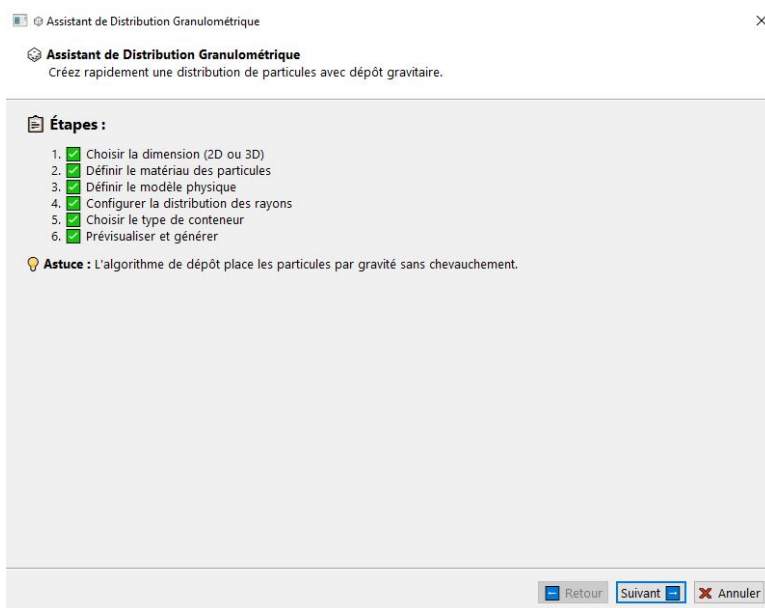
L'assistant est composé de **7 pages** actives (la page Aperçu est désactivée dans la version actuelle).

Page	Titre	Description
0	Introduction	Présentation de l'algorithme
1	Dimension	2D (disques) ou 3D (sphères)
2	Matériau	Créer ou réutiliser un matériau RIGID
3	Modèle	Créer ou réutiliser un modèle Rxx2D / Rxx3D
4	Distribution	Nombre de particules, rayons, graine
5	Conteneur	Géométrie et dimensions du dépôt
6	Récapitulatif	Vérification avant génération

Page 0 — Introduction

Présente le principe de l'algorithme en deux étapes :

1. `pre.granulo_Random(nb, r_min, r_max, seed)` — génère aléatoirement **nb** rayons dans `[r_min, r_max]` selon une distribution uniforme.
2. `pre.depositInxxx(radii, ...)` — effectue un dépôt gravitaire des particules dans le conteneur choisi, sans chevauchement.



captures/assistant_granulo_page1.JPG

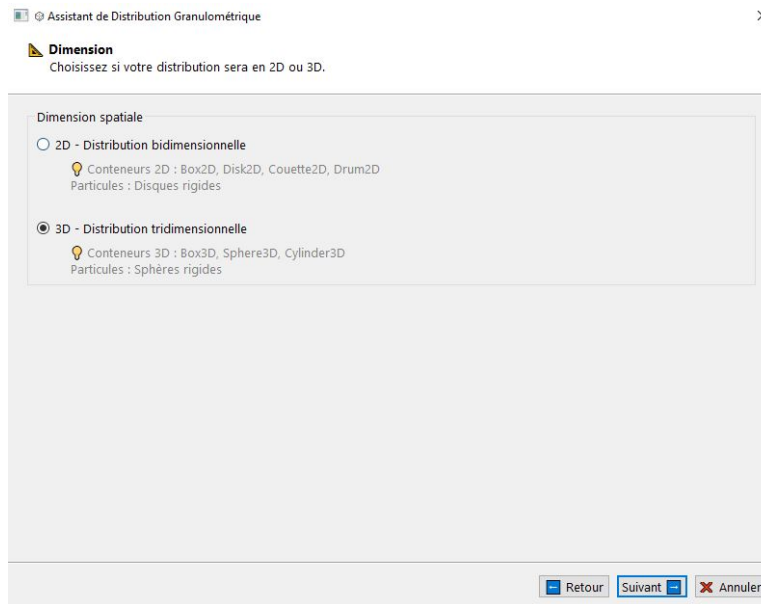
Cliquer sur **Suivant** ➡ pour commencer.

Page 1 — Dimension

Choix	Type de particule généré	Conteneurs disponibles
2D	rigidDisk	Box2D, Disk2D, Couette2D, Drum2D
3D	rigidSphere	Box3D, Sphere3D, Cylinder3D

La valeur **2D** est sélectionnée par défaut.

La dimension conditionne le type d'avatar créé, la liste des conteneurs disponibles à la page 5 et l'élément du modèle (Rxx2D ou Rxx3D).



Choix de la dimension

Page 2 — Matériau des particules

Deux modes :

Mode A — Utiliser un matériau existant _(proposé si des matériaux existent)_

Liste déroulante de tous les matériaux du projet.

Mode B — Créer un nouveau matériau _(coché automatiquement si aucun n'existe)_

Champ	Description	Défaut
Nom	Identifiant, 5 caractères maximum.	TDURx
Densité	Masse volumique (kg/m³). Plage : 100 à 20 000.	2500 kg/m³

*Le type est toujours **RIGID** — les particules granulaires sont des corps rigides.*

Assistant de Distribution Granulométrique

Matériau des Particules
Créez un nouveau matériau ou utilisez un existant.

Source du matériau
☒ Créer un nouveau matériau

Nouveau matériau
Nom : TDURx
Densité : 2500,00 kg/m³

Retour Suivant Annuler

Création du matériau

Page 3 — Modèle physique

Deux modes :

Mode A — Utiliser un modèle existant

Liste déroulante des modèles du projet.

Mode B — Créer un nouveau modèle

Champ	Valeur
Nom	5 caractères maximum. Défaut : rigid .
Physique	MECAx (automatique)
Élément	Rxx2D (2D) ou Rxx3D (3D) — adapté automatiquement

Assistant de Distribution Granulométrique

Modèle Physique
Créez un nouveau modèle ou utilisez un existant.

Source du modèle

☒ Créer un nouveau modèle

Nouveau modèle

Nom : rigid

Retour
Suivant
Annuler

Création du modèle

Page 4 — Distribution des particules

Nombre de particules

Champ	Description	Plage	Défaut
Nombre demandé	Nombre de particules à générer. Le nombre réellement placé peut être inférieur si le conteneur est saturé.	10 à 10 000	200

Un indicateur visuel signale la densité :

- **Orange** (< 100) : faible densité
- **Bleu** (100 à 499) : densité moyenne
- **Vert** (≥ 500) : densité élevée

Distribution des rayons

Champ	Description	Plage	Défaut
Rayon minimum	Rayon le plus petit des particules (m).	0,001 à 10,0	0.05 m
Rayon maximum	Rayon le plus grand (m). Doit être > Rmin.	0,001 à 10,0	0.15 m
Ratio Rmax/Rmin	Calculé automatiquement. Indique l'étendue de la polydispersité. Un ratio ≥ 3 produit une distribution très étalée.	—	3.00

Un **histogramme en temps réel** (20 classes, 200 rayons de prévisualisation) visualise la distribution des tailles avant génération.

Reproductibilité

Champ	Description	Défaut
Utiliser une graine	Case à cocher. Rend la génération reproductible.	décoché
Valeur de la graine	Entier de 0 à 999 999. Transmis à <code>pre.granulo_Random(seed=...)</code> .	12345

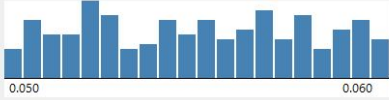
Assistant de Distribution Granulométrique

Distribution des Particules
Définissez le nombre et la taille des particules.

Nombre de particules
Nombre demandé : 500
Niveau : Élevé

Distribution des rayons
Rayon minimum : 0.05 m
Rayon maximum : 0.06 m
Ratio Rmax/Rmin : 1.20

Aperçu de la distribution :



Reproductibilité
☐ Utiliser une graine aléatoire 12345

Retour Suivant Annuler

Distribution des rayons

Page 5 — Conteneur de dépôt

La liste des conteneurs s'adapte automatiquement à la dimension choisie à la page 1.

Conteneurs 2D

Box2D — Boîte rectangulaire 2D

Paramètre	Description	Défaut
Largeur (lx)	Dimension horizontale de la boîte (m).	4.0 m
Hauteur (ly)	Dimension verticale de la boîte (m).	4.0 m

Fonction `pylmgc90` : `pre.depositInBox2D(radii, lx, ly)` **Usage :** essai de compression biaxiale, colonne de sol, cellule de cisaillement.

Disk2D — Disque circulaire 2D

Paramètre	Description	Défaut
Rayon (r)	Rayon du disque conteneur (m).	2.0 m

Fonction `pylmgc90` : `pre.depositInDisk2D(radii, r)` **Usage :** silo circulaire, tambour 2D.

Couette2D — Cellule de Couette 2D

Espace annulaire entre un cylindre intérieur de rayon `rint` et un cylindre extérieur de rayon `rext`.

Paramètre	Description	Défaut
Rayon intérieur (rint)	Rayon du cylindre intérieur (m).	2.0 m
Rayon extérieur (rext)	Rayon du cylindre extérieur (m). Doit être > rint.	4.0 m

Fonction `pylmgc90` : `pre.depositInCouette2D(radii, rint, rext)` Usage : rhéomètre de Couette, mesure de viscosité effective, écoulement annulaire.

Drum2D — Tambour rotatif 2D

Paramètre	Description	Défaut
Rayon (r)	Rayon du tambour (m).	2.0 m

Fonction `pylmgc90` : `pre.depositInDrum2D(radii, r)` Usage : mélangeur rotatif, broyeur à boulets, sécheur.

Conteneurs 3D

Box3D — Boîte parallélépipédique 3D

Paramètre	Description
Largeur (lx)	Dimension en X (m).
Profondeur (ly)	Dimension en Y (m).
Hauteur (lz)	Dimension en Z (m).

Fonction `pylmgc90` : `pre.depositInBox3D(radii, lx, ly, lz)` Usage : essai triaxial 3D, modèle de sol 3D.

Sphere3D — Sphère conteneur 3D

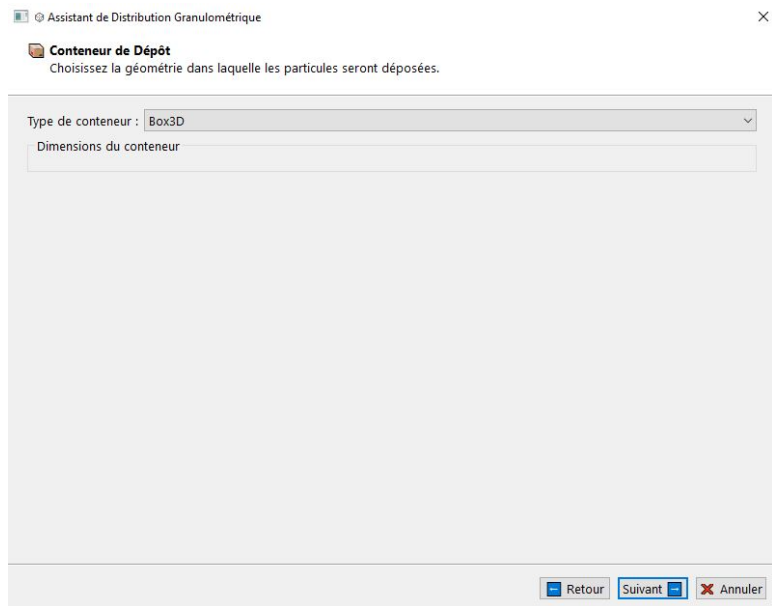
Paramètre	Description
Rayon (r)	Rayon de la sphère conteneur (m).

Fonction `pylmgc90` : `pre.depositInSphere3D(radii, r)`

Cylinder3D — Cylindre conteneur 3D

Paramètre	Description
Rayon (r)	Rayon du cylindre (m).

Fonction `pylmgc90` : `pre.depositInCylinder3D(radii, r)` Usage : colonne cylindrique, silo, essai œdométrique 3D.



Sélection du conteneur Box3D

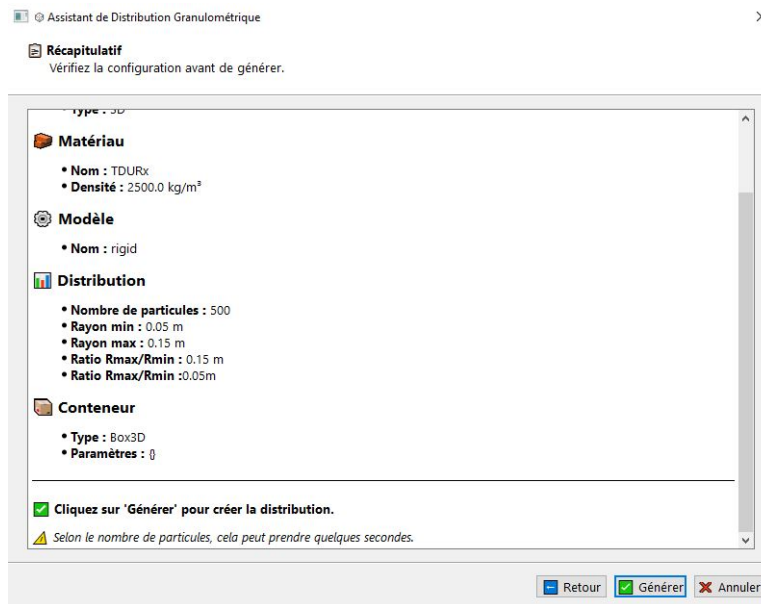
Page 6 — Récapitulatif

Affiche un tableau complet avant génération :

Section	Informations
Dimension	2D ou 3D
Matériau	Nom et densité (nouveau) ou matériau existant
Modèle	Nom (nouveau) ou modèle existant
Distribution	Nombre de particules, Rmin, Rmax, ratio Rmax/Rmin
Graine	Valeur si activée
Conteneur	Type, paramètres (lx, ly, r, rint, rext...)

Cliquer sur ■ **Générer** pour lancer. Un message de confirmation indique le succès.

En cas d'erreur : l'état du projet (nom, chemin, dimension) est entièrement restauré.



Récapitulatif

Résultat de la génération

Élément créé	Description
Matériau	Ajouté à l'onglet Matériau (si créé). Type RIGID .
Modèle	Ajouté à l'onglet Modèle (si créé). MECAx + Rxx2D / Rxx3D .
Avatars particules	Un rigidDisk (2D) ou rigidSphere (3D) par particule placée. Couleur BLUEx . Origine AvatarOrigin.GRANULO .
Groupe automatique	Groupe granulo_{conteneur} (ex. granulo_box3d).
GranuloGeneration	Configuration sauvegardée dans state.granulo_generations pour la reconstruction du script.

Optimisation : l'assistant insère les avatars directement dans les conteneurs `pymgc90 (_bodies_container, _pymgc_bodies)` sans émettre de signal par particule — ce qui permet de traiter plusieurs milliers de particules sans ralentir l'interface.

Remarques importantes

Reproductibilité : deux générations avec les mêmes paramètres produisent des générations différents. Fixer une distribution garantit l'identité exacte des résultats — indispensable pour les études paramétriques.

Ratio Rmax/Rmin : un ratio élevé (> 3) produit un assemblage polydisperse où les petites particules comblent les vides entre les grandes, augmentant la compacité. Un ratio proche de 1 donne un assemblage quasi-monodisperse avec une compacité maximale plus faible.

Limite 12 000 particules : au-delà, le rafraîchissement de l'arbre du modèle et des onglets peut prendre plusieurs secondes. Pour les très grands assemblages, utiliser la génération numpy.

Génération granulométrie numpy **_(bêta)_**

La **génération granulométrie numpy** est un mode de génération rapide adapté aux très grands assemblages (> 5 000 particules). Elle s'ouvre en tant que dialogue simple (**QDialog**) — sans étapes, sans page à parcourir — et utilise un **thread de calcul en arrière-plan** (**GranuloWorker**) pour ne pas bloquer l'interface pendant le dépôt.

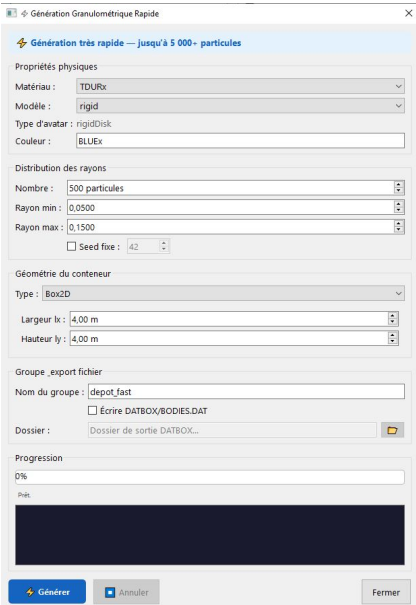
Accès : menu **Assistants** → ■ **Génération granulométrie numpy... (bêta)**

Recommandé pour : assemblages de **5 000 particules et plus**. En dessous de ce seuil, l'assistant **pylmgc90** (**Ctrl+Shift+G**) est préférable car il utilise directement les routines de dépôt officielles de **pylmgc90**.

Statut : fonctionnalité en version **bêta**. Les positions générées peuvent différer légèrement de celles produites par **pre.depositInBox2D** de **pylmgc90**, car l'algorithme de placement est distinct. Le résultat est toutefois physiquement cohérent (sans chevauchement).

Accès

Cliquer sur le menu **Assistants** puis ■ **Génération granulométrie numpy... (bêta)**. Le dialogue s'ouvre directement — aucun assistant par étapes.



Dialogue génération granulométrie numpy

Différences avec l'assistant pylmgc90

Caractéristique	Assistant pylmgc90 (Ctrl+Shift+G)	Génération numpy (bêta)
Interface	Assistant 7 pages	Dialogue unique
Algorithme de placement	pre.depositInXxx — dépôt gravitaire physique pylmgc90	GranuloGenerator via numpy — placement algorithmique
Calcul	Thread principal (peut bloquer l'UI)	Thread en arrière-plan (GranuloWorker) — UI non bloquée
Barre de progression	Non	Oui — progression en temps réel
Limite recommandée	~8 000 particules	> 5 000 particules, sans limite pratique
Matériau / modèle	Création ou réutilisation	Utilise matériaux et modèles existants

Caractéristique	Assistant pylmgc90 (Ctrl+Shift+G)	Génération numpy (bêta)
Conteneurs 3D	Box3D, Sphere3D, Cylinder3D	2D uniquement (Box2D, Disk2D, Couette2D, Drum2D)
Reproductibilité	Oui (graine)	Oui (graine)

Champs du dialogue

Paramètres de distribution

Champ	Description	Plage	Défaut
Nombre de particules	Nombre de particules demandé à l'algorithme de placement.	10 à 100 000+	1000
Rayon minimum (rmin)	Rayon minimal des particules (m).	0,001 à 10,0	0.05 m
Rayon maximum (rmax)	Rayon maximal des particules (m). Doit être > rmin.	0,001 à 10,0	0.10 m

Matériau et modèle

Champ	Description
Matériau	Liste déroulante des matériaux existants dans le projet. Sélectionner le matériau à appliquer aux particules.
Modèle	Liste déroulante des modèles existants dans le projet.

Contrairement à l'assistant pylmgc90, ce dialogue ne crée pas de nouveau matériau ni de nouveau modèle — ils doivent déjà exister dans le projet.

Conteneur de dépôt

Seuls les conteneurs **2D** sont disponibles dans la version actuelle.

Conteneur	Paramètres	Description
Box2D	lx (m), ly (m)	Boîte rectangulaire.
Disk2D	r (m)	Disque circulaire.
Couette2D	r _{int} (m), r _{ext} (m)	Cellule de Couette — espace annulaire.
Drum2D	r (m)	Tambour rotatif.

Reproductibilité

Champ	Description	Défaut
Utiliser une graine	Case à cocher. Rend la génération reproductible.	décoché
Valeur de la graine	Entier de 0 à 999 999.	12345

Options

Champ	Description	Défaut
Couleur	Code couleur LMGC90 à 5 caractères.	BLUEx
Groupe	Nom du groupe dans lequel stocker les avatars générés (<code>state.avatar_groups</code>).	granulo_numpy

Fonctionnement interne

Thread de calcul — `GranuloWorker`

La génération s'exécute dans un thread séparé (`GranuloWorker`, sous-classe de `QThread`) pour ne pas bloquer l'interface.

Signaux émis par `GranuloWorker` :

Signal	Paramètres	Déclenchement
progress_updated	(nb_fait, nb_total, message)	À chaque étape de calcul — met à jour la barre de progression
data_ready	list[{center, radius}]	Calcul terminé — envoie les données au thread principal
error_occurred	str	En cas d'exception — affiche un message d'erreur

Séquence d'exécution :

```
1. GranuloFastDialog.exec()
↓
2. GranuloWorker.start() ← thread secondaire
↓
3. GranuloGenerator.generate(config)
   ■■■ pre.granulo_Random(nb, rmin, rmax, seed)
   ■■■ pre.depositInXxx(radii, ...)
↓
4. GranuloWorker.data_ready.emit(particles_data)
↓
5. Thread principal : création des avatars
↓
6. GranuloFastDialog.granulo_generated.emit()
↓
7. MainWindow._refresh_all()
```

Données retournées par `GranuloWorker`

Chaque particule est transmise au thread principal sous forme de dictionnaire :

```
{
'center': [x, y], # coordonnées numpy → list Python
'radius': 0.075 # rayon en float
}
```

Algorithme de placement

`GranuloGenerator.generate()` appelle les mêmes routines `pymgc90` que l'onglet Granulométrie :

```
radii = pre.granulo_Random(config.nb_particles, config.radius_min, config.radius_max, config.seed)
nb_remaining, coor = pre.depositInBox2D(radii, params['lx'], params['ly'])
coor.shape = [coor.size // 2, 2]
radii = radii[:nb_remaining]
return nb_remaining, coor, radii
```

La différence par rapport à l'assistant est que le calcul est déporté dans un thread (`QThread`), ce qui empêche le gel de l'interface sur de très grands assemblages.

Progression et annulation

Pendant la génération, le dialogue affiche :

- Une **barre de progression** mise à jour par `progress_updated` (valeur courante / total).
- Un **message d'état** décrivant l'étape en cours (ex. : « Calcul du dépôt granulométrique... », « Dépôt calculé avec succès »).
- Un bouton ■ **Annuler** qui appelle `GranuloWorker.stop()` pour interrompre le calcul.

Résultat de la génération

Élément créé	Description
Avatars particules	Un <code>rigidDisk</code> par particule placée. Origine <code>AvatarOrigin.GRANULO</code> .
Groupe	Groupe nommé dans <code>state.avatar_groups</code> (défaut : <code>granulo_numpy</code>).
GranuloGeneration	Configuration sauvegardée dans <code>state.granulo_generations</code> .
Signal	<code>granulo_generated</code> émis → <code>MainWindow._refresh_all()</code>

*Les avatars sont créés dans le **thread principal** après réception du signal `data_ready` — la création reste synchrone mais le calcul lourd (dépôt physique) est asynchrone.*

Remarques importantes

Matériau et modèle obligatoires : contrairement à l'assistant `pymgc90`, le dialogue ne propose pas de créer un matériau ou un modèle. Créer au préalable dans l'onglet Matériau (ex. `TDURx`, `RIGID`, 2500 kg/m³) et dans l'onglet Modèle (ex. `rigid`, `MECAx`, `Rxx2D`).

Conteneurs 2D uniquement : la version bêta actuelle ne supporte pas les conteneurs 3D (Box3D, Sphere3D, Cylinder3D). Pour une génération 3D, utiliser l'assistant `pymgc90`.

Résultats identiques au rechargement : les positions et rayons générés sont sauvegardés via `GranuloGeneration` dans le projet. Lors du rechargement, le dépôt est **recalculé** à partir des paramètres — si aucune graine n'a été fixée, les positions seront différentes. Fixer une graine pour garantir l'identité exacte entre les sessions.

Thread et ressources : si le dialogue est fermé pendant le calcul, le thread `GranuloWorker` continue en arrière-plan jusqu'à son terme. Attendre la fin du calcul ou cliquer sur ■ Annuler avant de fermer.

Conditions aux limites (DOF)

Conditions aux limites (DOFs)

L'onglet **DOF** (**Ctrl+5**) permet d'appliquer des conditions aux limites mécaniques, thermiques ou initiales sur les avatars du projet. Chaque opération est enregistrée dans `state.operations`, appliquée immédiatement sur les objets `pymgc90` et exportée dans le script Python généré.

Principe de fonctionnement

Une **opération DOF** (`DOFOperation`) est composée de :

Champ	Description
<code>operation_type</code>	Nature de l'opération : <code>translate</code> , <code>rotate</code> , <code>imposeDrivenDof</code> , <code>imposeInitValue</code>
<code>target_type</code>	Cible : <code>'avatar'</code> (index unique) ou <code>'group'</code> (nom de groupe)
<code>target_value</code>	Index de l'avatar (int) ou nom du groupe (str)
<code>parameters</code>	Dictionnaire de paramètres passé directement à la méthode <code>pymgc90</code>

Interface de l'onglet

L'onglet est organisé en deux zones :

- **Liste des opérations** (en haut) : tableau de toutes les opérations enregistrées avec leur type, cible et paramètres principaux. Double-clic pour éditer. Clic droit pour accéder au menu contextuel.
- **Formulaire de création / modification** (en bas) : champs qui s'adaptent dynamiquement au type d'opération choisi.

Sélection de la cible

Champ	Description
Type de cible	<code>Avatar</code> (index numérique) ou <code>Groupe</code> (nom de groupe).
Cible	Si Avatar : index de l'avatar dans la liste (0-based). Si Groupe : liste déroulante de tous les groupes définis dans le projet (boucles, granulométrie, maçonnerie, etc.).

Tous les groupes créés dans les onglets Boucles, Granulométrie et Maçonnerie apparaissent automatiquement dans la liste des groupes disponibles.

Les quatre opérations

1. `translate` — Déplacement rigide

Déplace tous les nœuds de l'avatar d'un vecteur de translation. Opération purement géométrique — n'impose pas de condition cinématique pour la simulation.

Signature `pymgc90` :

```
avatar.translate(dx=0., dy=0., dz=0.)
```

Paramètres :

Paramètre	Type	Défaut	Description
<code>dx</code>	float	<code>0.0</code>	Translation selon l'axe X (m)
<code>dy</code>	float	<code>0.0</code>	Translation selon l'axe Y (m)
<code>dz</code>	float	<code>0.0</code>	Translation selon l'axe Z (m) — 3D uniquement

Exemple :

```
bodies[0].translate(dx=0.5, dy=0.0)
```

La position de l'avatar dans `state.avatars` est resynchronisée automatiquement après la translation (`_sync_avatar_position`). Cela met à jour l'affichage dans l'arbre du modèle et le viewer 3D.

2. `rotate` — Rotation rigide

Applique une rotation à l'avatar autour d'un centre donné. Deux modes de description sont disponibles : angles d'Euler ou axe-angle.

Signature `pymgc90` :

```
avatar.rotate(  
    description='Euler', # ou 'axis'  
    phi=0., theta=0., psi=0., # angles d'Euler (rad) – mode Euler  
    alpha=0., # angle (rad) – mode axis  
    axis=[0., 0., 1.], # axe de rotation – mode axis  
    center=[0., 0., 0.] # centre de rotation (m)  
)
```

Paramètres communs :

Paramètre	Type	Défaut	Description
<code>description</code>	str	<code>'Euler'</code>	Mode : <code>'Euler'</code> ou <code>'axis'</code>
<code>center</code>	list[3]	<code>[0., 0., 0.]</code>	Centre de rotation en coordonnées absolues (m)

Paramètres mode `'Euler'` :

Paramètre	Type	Défaut	Description
<code>phi</code>	float	<code>0.0</code>	1■ angle d'Euler — rotation autour de Z (rad)
<code>theta</code>	float	<code>0.0</code>	2■ angle d'Euler — rotation autour de X (rad)
<code>psi</code>	float	<code>0.0</code>	3■ angle d'Euler — rotation autour de Z (rad)

Les trois rotations sont appliquées successivement : d'abord **phi** autour de Z, puis **theta** autour de X, puis **psi** autour de Z.

Paramètres mode '**axis**' :

Paramètre	Type	Défaut	Description
alpha	float	0.0	Angle de rotation (rad)
axis	list[3]	[0., 0., 1.]	Vecteur directeur de l'axe de rotation

Exemples :

```
# Rotation de 90° autour de Z centré à l'origine (mode axis)
bodies[0].rotate(description='axis', alpha=1.5708, axis=[0., 0., 1.], center=[0., 0., 0.])

# Rotation d'Euler de 45° dans le plan XY
bodies[2].rotate(description='Euler', phi=0.7854, theta=0., psi=0., center=[1.0, 0.5, 0.])
```

Conseil : pour les murs maçonnés, la rotation '**axis**' avec **axis=[0,0,1]** permet de créer des angles de bâtiment. L'angle est saisi en degrés dans l'interface et converti en radians automatiquement.

3. **imposeDrivenDof** — DDL piloté

Impose un degré de liberté **piloté** sur les nœuds d'un groupe de l'avatar. La valeur imposée peut être constante, sinusoïdale avec rampe, ou définie par un fichier d'évolution temporelle.

Signature **pymgc90** :

```
avatar.imposeDrivenDof(
group='all',
component=1,
description='predefined',
dofty='vlocy',
ct=0.,
amp=0.,
omega=0.,
phi=0.,
rampi=1.,
ramp=0.,
evolutionFile=''
)
```

Paramètres :

Paramètre	Type	Défaut	Description
group	str	'all'	Groupe de nœuds de l'avatar. 'all' = tous les nœuds. Pour les corps déformables : 'down' , 'up' , 'left' , 'right' , 'front' , 'rear' .
component	int ou list	1	Composante(s) du DDL. Voir tableau ci-dessous.
description	str	'predefined'	Mode temporel : 'predefined' (formule analytique) ou 'evolution' (fichier).
dofty	str	'vlocy'	Type de DDL. Voir tableau ci-dessous.

Paramètre	Type	Défaut	Description
ct	float	0.0	Valeur constante.
amp	float	0.0	Amplitude du cosinus.
omega	float	0.0	Pulsation angulaire (rad/s).
phi	float	0.0	Phase du cosinus (rad).
rampi	float	1.0	Valeur initiale de la rampe multiplicatrice.
ramp	float	0.0	Pente de la rampe (s ⁻¹).
evolutionFile	str	' '	Chemin vers un fichier d'évolution temporelle (*.evol). Utilisé si descrip tion='evolution' .

Composantes (component)

Valeur	Signification
1	Translation selon X (ou DDL 1)
2	Translation selon Y (ou DDL 2)
3	Translation selon Z (ou DDL 3) — 3D uniquement
[1, 2]	X et Y simultanément (bloque le plan horizontal)
[1, 2, 3]	Blocage complet 3D

Types de DDL (dofty)

Physique	dofty	Description
MECAx	'vlocy'	Vitesse imposée (m/s)
MECAx	'force'	Force imposée (N)
THERx	'temp'	Température imposée (K ou °C selon convention)
THERx	'flux'	Flux thermique imposé (W/m²)
PORox	'vlocy'	Vitesse de filtration (m/s)
PORox	'force'	Pression imposée
MULTI	'prim_'	Variable primale (pression, température...)
MULTI	'dual_'	Variable duale (flux, force...)

Formule générale (mode 'predefined')

La valeur imposée à l'instant **t** est :

$$f(t) = [ct + amp \times \cos(\omega \times t + \phi)] \times \text{clamp}(rampi + ramp \times t)$$

où $\text{clamp}(x) = \text{sign}(x) \times \min(|x|, 1)$ — la rampe est bornée à 1 pour éviter les amplifications non physiques.

Scénario	Paramètres	Comportement
Bloqué (vitesse nulle)	ct=0, dofty='vlocy'	Nœuds fixes pendant toute la simulation
Déplacement constant	ct=v0, dofty='vlocy'	Vitesse constante v0 m/s
Oscillation sinusoïdale	amp=A, omega=ω, phi=φ	$v(t) = A \times \cos(\omega t + \phi)$

Scénario	Paramètres	Comportement
Démarrage progressif	<code>ct=v0, rampi=0, ramp=1/t_rampe</code>	Montée linéaire de 0 à <code>v0</code> sur <code>t_rampe</code> secondes
Évolution quelconque	<code>description='evolution', evolutionFile='monfichier.evol'</code>	Valeur interpolée depuis le fichier

Exemples

```
# Bloquer la base d'un maillage EF (vitesse nulle en X et Y)
mesh.imposeDrivenDof(group='down', component=[1, 2], dofty='vlocy', ct=0.)

# Imposer un déplacement vertical constant de 0.001 m/s
bodies[3].imposeDrivenDof(group='all', component=2, dofty='vlocy', ct=0.001)

# Oscillation sinusoïdale en X, fréquence 5 Hz, amplitude 0.01 m/s
bodies[5].imposeDrivenDof(
group='all', component=1, dofty='vlocy',
ct=0., amp=0.01, omega=31.416, phi=0.
)

# Chargement thermique depuis fichier d'évolution
mesh.imposeDrivenDof(
group='up', component=1, dofty='temp',
description='evolution', evolutionFile='temperature.evol'
)
```

4. `imposeInitValue` — Valeur initiale

Impose une **condition initiale** (position ou vitesse) sur les nœuds d'un groupe, uniquement à l'instant `t = 0`. N'impose aucune contrainte pendant le calcul.

Signature `pymg90` :

```
avatar.imposeInitValue(
group='all',
component=1,
value=0.
)
```

Paramètres :

Paramètre	Type	Défaut	Description
<code>group</code>	str	<code>'all'</code>	Groupe de nœuds.
<code>component</code>	int ou list	<code>1</code>	Composante(s) du DDL (même convention que <code>imposeDrivenDof</code>).
<code>value</code>	float	<code>0.0</code>	Valeur initiale à imposer (m/s, m, K selon le type).

Différence avec `imposeDrivenDof` : `imposeInitValue` n'est actif qu'à `t = 0`. L'avatar est libre de se déplacer ensuite. `imposeDrivenDof` maintient la contrainte tout au long de la simulation.

Gestion des opérations

Créer une opération

Remplir le formulaire et cliquer sur **■ Appliquer**. L'opération est appliquée immédiatement sur les objets pylmgc90 et sauvegardée dans `state.operations`. Le signal `operation_applied` est émis, déclenchant un rafraîchissement complet de l'interface et du viewer 3D.

Modifier une opération

Double-cliquer sur une opération dans la liste ou sélectionner et cliquer sur **—■ Modifier**. Le formulaire est chargé avec les valeurs de l'opération. Modifier et cliquer sur **■ Enregistrer** pour mettre à jour (`update_dof_operation`). L'opération est réappliquée.

Supprimer une opération

Sélectionner et cliquer sur **■■ Supprimer**. L'opération est retirée de `state.operations` (`remove_dof_operation`). Le signal `operation_deleted` est émis.

Attention : la suppression d'une opération n'annule pas son effet sur les avatars (les translations et rotations déjà appliquées restent en place). Pour annuler une translation, créer une translation inverse.

Support des groupes

La liste déroulante de groupes inclut automatiquement tous les groupes définis dans le projet :

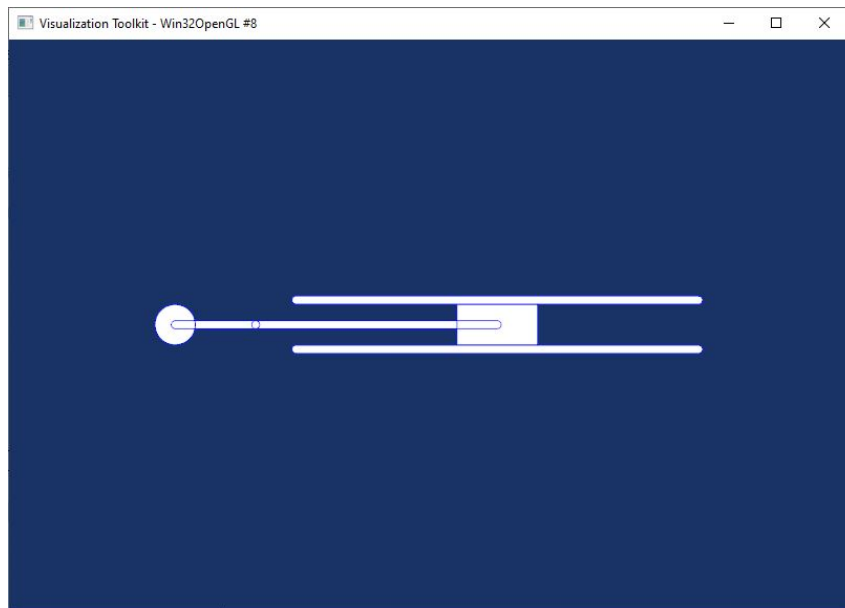
Source	Exemples de noms
Boucles géométriques	<code>ma_ligne</code> , <code>anneau_particules</code>
Boucles <code>for</code>	nom saisi dans le formulaire de la boucle
Granulométrie	<code>granulo_box2d</code> , <code>granulo_couette2d</code>
Maçonnerie	<code>mur_briques</code> , <code>mur_facade</code>
Groupes DOF maillage EF	<code>down</code> , <code>up</code> , <code>left</code> , <code>right</code> , <code>front</code> , <code>rear</code>

Appliquer une opération à un groupe exécute la même opération sur chaque avatar du groupe :

```
# Exemple script généré pour un groupe :
for av in group_granulo_box2d:
    av.imposeDrivenDof(group='all', component=1, dofty='vlocy', ct=0.0)
```

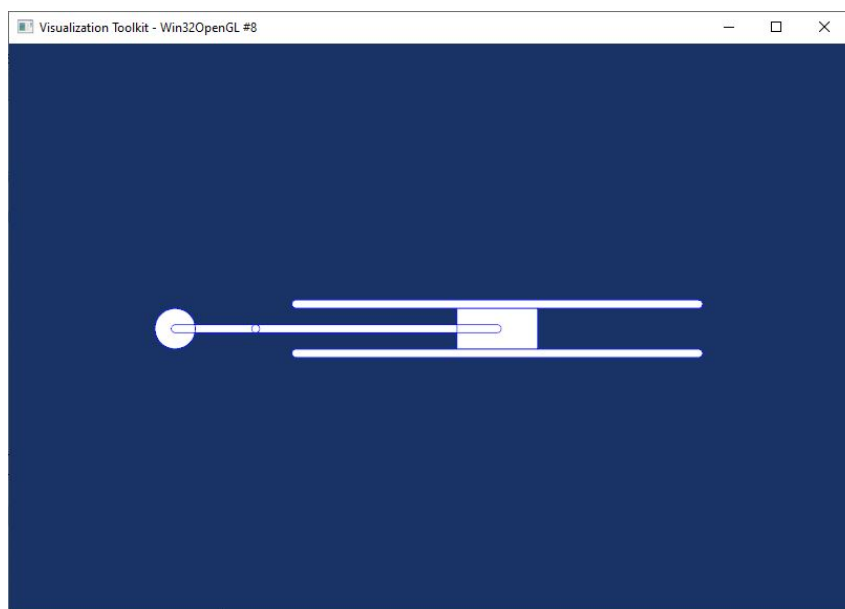
Exemple — Bielle-manivelle (`slider_crank.lmgc90`)

L'exemple fourni dans les exemples du projet illustre l'application de quatre conditions aux limites sur les avatars du mécanisme bielle-manivelle :



Exemple slider_crank

Opération	Type	Cible	Paramètres typiques
Rotation de la manivelle	<code>imposeDrivenDof</code>	Avatar manivelle	<code>component=3,</code> <code>dofty='vlocy', ct=ω</code>
Blocage du coulisseau	<code>imposeDrivenDof</code>	Avatar coulisseau	<code>component=2,</code> <code>dofty='vlocy', ct=0.</code>
Pivot bielle-manivelle	<code>imposeInitValue</code>	Avatar bielle	<code>component=[1,2],</code> <code>value=0.</code>
Position initiale	<code>translate</code>	Avatar manivelle	<code>dx=x0, dy=y0</code>



captures/exemple_slider_crank.JPG

Remarques importantes

Ordre des opérations : les opérations sont appliquées dans l'ordre où elles sont créées. Une translation suivie d'une rotation donne un résultat différent de la rotation suivie de la translation. L'ordre dans `state.operations` est respecté à la fois lors de l'application et dans le script généré.

Annulation impossible : `translate` et `rotate` modifient les coordonnées physiques des nœuds pylmgc90. Il n'existe pas de bouton Annuler — pour inverser, créer une opération opposée (translation inverse, rotation de signe opposé).

`imposeDrivenDof` vs `imposeInitValue` : utiliser `imposeDrivenDof` avec `ct=0` et `dofty='vlocy'` pour **bloquer** un DDL pendant la simulation. Utiliser `imposeInitValue` uniquement pour définir une **condition initiale** sans contrainte sur la durée.

Fichiers d'évolution : le fichier `.evol` doit être placé dans le répertoire `DATBOX/` du projet. Il contient deux colonnes : temps (s) et valeur, séparées par des espaces ou des tabulations. Le nombre de lignes doit couvrir toute la durée de simulation.

Composantes pour les corps déformables : pour les maillages EF, `component` accepte directement les entiers 1, 2, 3 (DDL de déplacement nodaux). Pour les corps rigides, `component=1` correspond à la translation en X, `component=2` en Y, `component=3` en Z. La composante de rotation est `component=4` (2D) ou 4-6 (3D) selon la physique.

Lois de contact

L'onglet **Contact** (**Ctrl+9**) permet de créer, modifier et supprimer les **lois de contact** (**tact_behav**) de pylmgc90. Chaque loi définit le comportement mécanique à la frontière entre deux corps en contact. Une loi doit être créée **avant** de pouvoir être référencée dans une table de visibilité (onglet Visibilité).

Principe de fonctionnement

Une **loi de contact** (**ContactLaw**) est un objet pylmgc90 créé par `pre.tact_behav(name, law, ...)`. Elle est identifiée par un nom unique dans le projet et caractérise la physique du contact : frottement, cohésion, rigidité, endommagement...

Appel pylmgc90 généré :

```
law_IQS_CLB = pre.tact_behav(
name='IQS_CLB',
law='IQS_CLB',
fric=0.3
)
tacts.addBehav(law_IQS_CLB)
```

Interface de l'onglet

L'onglet comprend trois zones :

- **Liste des lois** (en haut) : arbre avec colonnes Nom, Type, Friction, Propriétés. Les lois **référéncées** par au moins une table de visibilité apparaissent en **vert**. Clic droit pour Modifier, Supprimer ou afficher les Informations.
- **Formulaire de création / modification** (au milieu) : champs qui s'adaptent automatiquement au type choisi.
- **Aide contextuelle** (en bas) : description et liste des paramètres du type sélectionné.

Boutons

Bouton	Mode	Action
■ Créer Loi	Création	Valide le formulaire et crée la loi.
■ Enregistrer Modifications	Édition	Met à jour la loi sélectionnée.
■ Annuler	Édition	Revient au mode création sans sauvegarder.
■ Réinitialiser	Tous	Vide le formulaire et remet les valeurs par défaut.
⇒■ Modifier Sélection	Tous	Charge la loi sélectionnée dans le formulaire.
■● Supprimer Sélection	Tous	Supprime la loi (avec vérification de référence).

Champs du formulaire

Champ	Description
Nom	Identifiant unique de la loi (20 caractères max). Utilisé dans les tables de visibilité. Défaut : law01 .
Catégorie	Filtre le combo Type. Voir les 4 catégories ci-dessous.
Type	Type de loi pylmgc90. Les champs spécifiques s'affichent/masquent dynamiquement selon le type.

Les 4 catégories de lois

Catégorie 1 — Rigide / Rigide

Lois applicables entre deux corps rigides (**RBDY2** / **RBDY3**).

IQS_CLB — Coulomb Inégalité Quasi-Statique **_(la plus courante)_**

Loi de Coulomb standard. Contact unilatéral avec frottement de Coulomb sec. Approche non-lisse Inégalité Quasi-Statique (IQS).

Paramètre	Label interface	Description	Défaut
fric	Coefficient de friction	Coefficient de Coulomb μ . La force tangentielle ne peut pas dépasser $\mu \times$ force normale.	0.3

Valeurs typiques de **fric** :

Matériau	fric
Sable / billes de verre	0,3 – 0,5
Béton / roches	0,5 – 0,8
Métaux polis	0,1 – 0,3
Bois / plastique	0,3 – 0,6
Sans frottement (glissant)	0.0

Appel pylmgc90 :

```
law = pre.tact_behav(name='loi1', law='IQS_CLB', fric=0.3)
```

IQS_CLB_g0 — Coulomb avec jeu initial

Identique à **IQS_CLB** mais initialise le jeu géométrique initial entre contacteurs à **g0=0**. Utile lorsque les corps sont déjà en contact dès **t = 0** (sans interpénétration initiale).

Paramètre	Description	Défaut
fric	Coefficient de frottement de Coulomb.	0.3

Applications : assemblages mécaniques pré-serrés, compression progressive, contacts avec rugosité initiale.

IQS_DS_CLB — Coulomb avec rigidités statique/dynamique

Loi discrète avec deux rigidités de contact distinctes : une rigidité statique (avant glissement) et une dynamique (en glissement).

Paramètre	Label interface	Description	Défaut
fric	Coefficient de friction	Coefficient de Coulomb.	0.3
stfr	Rigidité de contact statique	Rigidité normale statique (N/m).	1e8
dyfr	Rigidité de contact dynamique	Rigidité normale dynamique (N/m).	1e8

Applications : systèmes de freinage ($\mu_{\text{statique}} > \mu_{\text{dynamique}}$), glissement de plaques tectoniques, mécanismes avec vibrations auto-induites.

IQS_MOHR_DS_CLB — Mohr-Coulomb avec cohésion

Critère de Mohr-Coulomb incluant une cohésion normale et tangentielle. Permet de modéliser des matériaux avec adhésion initiale (ciment, argile humide).

Paramètre	Label interface	Description	Défaut
fric	Coefficient de friction	Coefficient de Coulomb μ .	0.3
stfr	Rigidité statique	Rigidité avant rupture de cohésion (N/m).	1e8
dyfr	Rigidité dynamique	Rigidité après rupture (N/m).	1e8
cohn	Cohésion normale	Force d'adhésion en traction (Pa).	0.0
coht	Cohésion tangentielle	Résistance tangentielle additionnelle (Pa).	0.0

Applications : géomécanique (argiles, sols cohésifs), matériaux granulaires humides, poudres avec forces de van der Waals.

IQS_MAC_CZM — Zone cohésive MAC (rigide/rigide)

Modèle de zone cohésive Mohr-Coulomb-Allix-Corigliano. Simule la rupture progressive et le délaminage entre corps rigides.

Paramètre	Label interface	Description	Défaut
stfr	Rigidité statique	Rigidité tangentielle avant endommagement (N/m).	1e10
dyfr	Rigidité dynamique	Rigidité normale avant endommagement (N/m).	1e10
cn	Résistance normale	Résistance à la traction normale (Pa).	1e6
ct	Résistance tangentielle	Résistance au cisaillement (Pa).	1e6
b	Paramètre de mélange	Couplage modes I/II (0 = pur mode I, 1 = équipartition).	1.0
w	Énergie de rupture	Énergie de fracture critique (J/m^2).	0.01

Applications : rupture de liaisons, fissuration, délaminage composite.

RST_CLB — Restitution + Coulomb

Contact avec coefficient de restitution (chocs élastiques ou partiellement élastiques) et friction de Coulomb.

Paramètre	Description	Défaut
fric	Coefficient de frottement de Coulomb.	0.3

Applications : impact de billes, rebonds, chocs mécaniques.

Catégorie 2 — Rigide / Déformable (ou Déf / Déf)

Lois applicables entre un corps rigide (RBDY2/RBDY3) et un corps déformable EF (MAILx), ou entre deux corps déformables.

GAP_SGR_CLB — Contact jeu + Coulomb (rigide/déformable)

Loi standard pour contact rigide/déformable. Gère le jeu initial entre la surface rigide et le maillage EF.

Paramètre	Description	Défaut
fric	Coefficient de frottement de Coulomb.	0.3

Applications : contact outil/pièce, frappe, indentation, compression EF.

GAP_SGR_CLB_g0 — GAP avec initialisation à g0

Identique à GAP_SGR_CLB avec initialisation du jeu à zéro. Utiliser si les corps sont initialement en contact tangent.

Paramètre	Description	Défaut
fric	Coefficient de frottement de Coulomb.	0.3

GAP_MOHR_DS_CLB — Mohr-Coulomb jeu (rigide/déformable)

Critère de Mohr-Coulomb avec gestion du jeu pour contact rigide/déformable.

Paramètre	Description	Défaut
fric	Coefficient de Coulomb.	0.3
stfr	Rigidité statique (N/m).	1e8
dyfr	Rigidité dynamique (N/m).	1e8
cohn	Cohésion normale (Pa).	0.0
coht	Cohésion tangentielle (Pa).	0.0

MAC_CZM — Zone cohésive MAC (rigide/déformable ou déf/déf)

Modèle de zone cohésive MAC appliqué aux interfaces rigide/déformable. Mêmes paramètres que IQS_MAC_CZM.

Paramètre	Description	Défaut
stfr, dyfr	Rigidités (N/m).	1e10
cn, ct	Résistances normale et tangentielle (Pa).	1e6
b	Paramètre de mélange.	1.0
w	Énergie de rupture (J/m²).	0.01

MAL_CZM — Zone cohésive MAL (rigide/déformable ou déf/déf)

Variante du modèle CZM basée sur la formulation MAL (Mixed Augmented Lagrangian). Mêmes paramètres que MAC_CZM.

Catégorie 3 — Point / Point

Lois applicables entre contacteurs ponctuels (PT2Dx, PT3Dx, nœuds EF). Modélisent des liaisons filaires ou des barres discrètes.

ELASTIC_WIRE — Câble élastique

Lien actif uniquement en **traction** (câble inextensible dans un sens). Modèle de câble simple.

Paramètre	Label interface	Description	Défaut
stiffness	Rigidité axiale	Rigidité EA du câble (N).	1e6
prestrain	Pré-déformation	Pré-tension initiale (adimensionnel, ex. 0.01 = 1 %).	0.0

Applications : câbles de suspension, tirants d'ancrage, renforts fibreux.

BRITTLE_ELASTIC_WIRE — Câble élastique fragile

Câble élastique qui se rompt de façon fragile (sans déformation plastique) lorsque la contrainte dépasse sigc.

Paramètre	Label interface	Description	Défaut
stiffness	Rigidité axiale	Rigidité EA du câble (N).	1e6
prestrain	Pré-déformation	Pré-tension initiale.	0.0
sigc	Résistance à la rupture	Contrainte maximale avant rupture fragile (Pa).	1e6

Applications : fibres fragiles, fils de verre, rupture de renfort.

ELASTIC_ROD — Barre élastique

Barre rigide pouvant travailler en **traction et en compression** (contrairement au câble). Modèle de barre linéaire.

Paramètre	Description	Défaut
stiffness	Rigidité axiale EA (N).	1e6
prestrain	Pré-déformation initiale.	0.0

Applications : structures treillis, liaisons rigides point à point, raidisseurs.

VOIGT_ROD — Barre visco-élastique de Voigt

Barre avec comportement visco-élastique (ressort + amortisseur en parallèle — modèle de Kelvin-Voigt).

Paramètre	Label interface	Description	Défaut
stiffness	Rigidité axiale	Rigidité du ressort EA (N).	1e6
viscosity	Viscosité	Coefficient d'amortissement visqueux (N.s).	1e3
prestrain	Pré-déformation	Pré-tension initiale.	0.0

Applications : amortisseurs, structures avec dissipation visqueuse, modèles de sol visco-élastique.

Catégorie 4 — Toutes (any / any)

Lois universelles applicables quelle que soit la paire de contacteurs.

COUPLED_DOF — Couplage de degrés de liberté

Couplage parfait (saut de vitesse et de déplacement nul à l'interface). Liaison cinématique rigide.

Aucun paramètre requis.

Applications : liaison parfaite entre deux corps, couplage de DDL dans les assemblages multi-corps.

NORMAL_COUPLED_DOF — Couplage en direction normale

Couplage uniquement dans la direction normale au contact. Permet un glissement tangentiel libre.

Aucun paramètre requis.

ELASTIC_REPELL_CLB — Répulsion élastique + Coulomb

Contact mou par pénalisation élastique en direction normale, avec frottement de Coulomb. Méthode de régularisation (alternative aux lois IQS strictes).

Paramètre	Label interface	Description	Défaut
fric	Coefficient de friction	Coefficient de Coulomb.	0.3
Kn	Rigidité normale	Rigidité de pénalité normale (N/m).	1e8

Applications : contacts mous, pénalisation douce, modèles avec interpénétration contrôlée.

Tableau récapitulatif des 18 lois

Loi	Catégorie	Paramètres	Usage principal
IQS_CLB	Rig/Rig	fric	Contact granulaire standard
IQS_CLB_g0	Rig/Rig	fric	Contact avec jeu initial nul

Loi	Catégorie	Paramètres	Usage principal
<code>IQS_DS_CLB</code>	Rig/Rig	<code>fric, stfr, dyfr</code>	Frottement statique/dynamique
<code>IQS_MOHR_DS_CLB</code>	Rig/Rig	<code>fric, stfr, dyfr, cohn, coht</code>	Matériaux cohésifs
<code>IQS_MAC_CZM</code>	Rig/Rig	<code>stfr, dyfr, cn, ct, b, w</code>	Rupture, délaminage
<code>RST_CLB</code>	Rig/Rig	<code>fric</code>	Chocs avec restitution
<code>GAP_SGR_CLB</code>	Rig/Déf	<code>fric</code>	Contact outil/pièce EF
<code>GAP_SGR_CLB_g0</code>	Rig/Déf	<code>fric</code>	Contact EF avec jeu nul
<code>GAP_MOHR_DS_CLB</code>	Rig/Déf	<code>fric, stfr, dyfr, cohn, coht</code>	Interface cohésive EF
<code>MAC_CZM</code>	Rig/Déf	<code>stfr, dyfr, cn, ct, b, w</code>	Zone cohésive EF
<code>MAL_CZM</code>	Rig/Déf	<code>stfr, dyfr, cn, ct, b, w</code>	Zone cohésive MAL
<code>ELASTIC_WIRE</code>	Pt/Pt	<code>stiffness, prestrain</code>	Câble (traction seule)
<code>BRITTLE_ELASTIC_WIRE</code>	Pt/Pt	<code>stiffness, prestrain, sigc</code>	Câble fragile
<code>ELASTIC_ROD</code>	Pt/Pt	<code>stiffness, prestrain</code>	Barre élastique
<code>VOIGT_ROD</code>	Pt/Pt	<code>stiffness, viscosity, prestrain</code>	Barre visco-élastique
<code>COUPLED_DOF</code>	Any/Any	<code>_(aucun)_</code>	Liaison rigide parfaite
<code>NORMAL_COUPLED_DOF</code>	Any/Any	<code>_(aucun)_</code>	Couplage normal seul
<code>ELASTIC_REPELL_CLB</code>	Any/Any	<code>fric, Kn</code>	Contact mou par pénalité

Gestion des lois

Créer une loi

Remplir le formulaire et cliquer sur **■ Créer Loi**. La loi est créée via `add_contact_law()` qui appelle `LMGC90Bridge.create_contact_law() → pre.tact_behav(...) → tacts.addBehav(...)`. Le signal `law_created` est émis → `_refresh_all()`. Les lois créées sont stockées dans `state.contact_laws` (liste de `ContactLaw`) et classifiées par `ContactLawType` (enum) et `CONTACT_LAW_CATEGORIES` (dict catégorie → lois).

Modifier une loi

Double-cliquer dans la liste ou cliquer sur **—■ Modifier Sélection**. Le formulaire passe en mode édition (bouton **■** remplacé par **■**). Modifier les valeurs et cliquer sur **■ Enregistrer Modifications** (`update_contact_law()`). Appuyer sur **■ Annuler** pour revenir au mode création sans sauvegarder.

Renommage : renommer une loi met automatiquement à jour le champ `behavior_name` de toutes les tables de visibilité qui y font référence.

Supprimer une loi

Sélectionner et cliquer sur **■■ Supprimer Sélection** (`remove_contact_law()`). Si la loi est référencée par une ou plusieurs tables de visibilité, un avertissement liste les références et bloque la suppression. Supprimer d'abord les tables de visibilité concernées.

Informations sur une loi

Clic droit → **■ Informations** affiche une boîte de dialogue avec le type, tous les paramètres et la liste des tables de visibilité qui l'utilisent.

Indicateur visuel dans la liste

Couleur du nom	Signification
Vert	Loi référencée par au moins une table de visibilité
Noir (normal)	Loi non encore référencée

Remarques importantes

Cohérence avec la table de visibilité : la loi choisie doit être compatible avec les types de corps/contacteurs de la table de visibilité. Une loi **GAP_SGR_CLB** ne fonctionnera pas avec une paire **RBDY2/DISKx** — **RBDY2/DISKx** (rigide/rigide) — utiliser **IQS_CLB** dans ce cas.

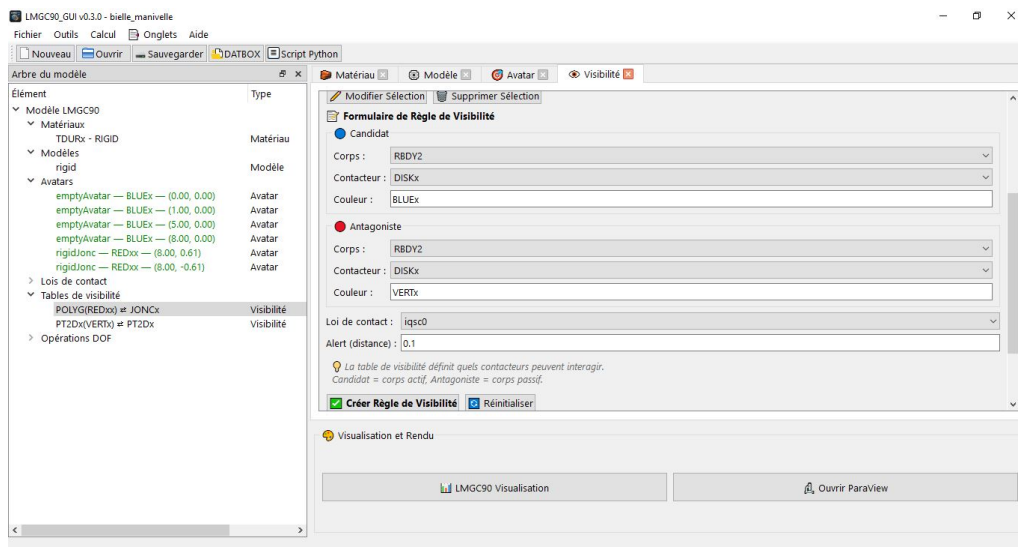
Cohérence avec le détecteur de contact : le type de loi doit correspondre au détecteur activé dans l'onglet Calcul. Par exemple, **ELASTIC_WIRE** nécessite le détecteur **PT2Lx** (point/ligne) ou similaire.

Valeurs numériques : les champs acceptent la notation scientifique Python (**1e8**, **1e-3**, **3.14e6**). Les valeurs sont évaluées au moment de la création de la loi.

Suppression protégée : une loi utilisée par au moins une table de visibilité ne peut pas être supprimée directement. Cette protection évite les erreurs de référence dans le projet.

Tables de visibilité

L'onglet **Visibilité** (**Ctrl+9**) permet de définir les **tables de visibilité** ([see_table](#)) de pylmgc90 : chaque règle déclare quels contacteurs peuvent se détecter mutuellement et avec quelle loi de contact. C'est une étape **obligatoire** avant de générer la DATBOX — sans table de visibilité, aucune interaction de contact n'est calculée.



captures/table_visibilite.JPG

Principe de fonctionnement

Une **table de visibilité** ([see_table](#)) est le mécanisme par lequel pylmgc90 sait quels corps doivent être testés pour la détection de contact. Elle met en relation un **corps candidat** (peut être pénétré) et un **corps antagoniste** (jamais pénétré), en précisant pour chaque paire la couleur des contacteurs impliqués et la loi de contact à appliquer.

Appel pylmgc90 généré :

```
see_0 = pre.see_table(
CorpsCandidat='RBDY2',
candidat='DISKx',
colorCandidat='BLUEx',
CorpsAntagoniste='RBDY2',
antagoniste='DISKx',
colorAntagoniste='BLUEx',
behav=law_IQS_CLB,
alert=0.1
)
sees.addSeeTable(see_0)
```

La structure **VisibilityRule** dans le projet stocke exactement ces 8 paramètres.

Interface de l'onglet

L'onglet est divisé en deux zones :

- **Liste des règles** (en haut) : tableau de toutes les tables de visibilité définies dans le projet, avec corps candidat, contacteur, couleurs, corps antagoniste, loi et distance d'alerte. Double-clic pour éditer.
- **Formulaire de création / modification** (en bas) : 8 champs correspondant aux paramètres de [pre.see_table](#).

Champs du formulaire

Corps candidat

Champ	Description	Valeurs courantes
Corps candidat (CorpsCandidat)	Type du corps portant le contacteur candidat.	RBDY2 (corps rigide 2D), RBDY3 (corps rigide 3D), MAILx (corps déformable EF)
Contacteur candidat (candidat)	Forme du contacteur sur le corps candidat. Doit correspondre au shape déclaré dans l'onglet Avatar vide.	Voir tableau des contacteurs ci-dessous
Couleur candidat (colorCandidat)	Code couleur LMGC90 à 5 caractères du contacteur candidat. Doit correspondre exactement à la couleur du contacteur déclaré sur le corps.	BLUEx , REDxx , VERTx , GRAYx...

Corps antagoniste

Champ	Description	Valeurs courantes
Corps antagoniste (CorpsAntagoniste)	Type du corps portant le contacteur antagoniste.	RBDY2 , RBDY3 , MAILx
Contacteur antagoniste (antagoniste)	Forme du contacteur antagoniste.	Voir tableau des contacteurs
Couleur antagoniste (colorAntagoniste)	Couleur du contacteur antagoniste. Doit correspondre à la couleur du contacteur déclaré sur le corps antagoniste.	BLUEx , REDxx...

Loi et alerte

Champ	Description	Défaut
Comportement (behav)	Nom de la loi de contact à appliquer pour cette paire. Sélectionner dans la liste déroulante des lois définies dans l'onglet Contact. La loi doit exister avant de créer la règle.	_(première loi disponible)_
Distance d'alerte (alert)	Distance de détection (m). Deux contacteurs distants de moins de alert sont considérés comme potentiellement en contact et sont testés. Plage : 0,001 à 10,0 m.	0.1 m

Types de corps disponibles

Type	Description	Usage
RBDY2	Corps rigide 2D (<code>rigidDisk</code> , <code>rigidJonc</code> , <code>rigidPolygon...</code>)	Simulations 2D — particules, maçonnerie, mécanismes
RBDY3	Corps rigide 3D (<code>rigidSphere</code> , <code>rigidPolyhedron...</code>)	Simulations 3D — sphères, polyèdres
MAILx	Corps déformable EF (<code>MESH_DEFORMABLE</code>)	Interactions rigide/déformable ou déformable/déformable

Types de contacteurs disponibles

Contacteurs 2D (pour RBDY2)

Contacteur	Description	Paramètre pylmgc90
DISKx	Disque circulaire	<code>r</code> (rayon)
xKSID	Disque orienté (anti-disque)	<code>r</code>
JONCx	Jonc elliptique / capsule	<code>axe1</code> , <code>axe2</code>
POLYG	Polygone convexe 2D	<code>nb_vertices</code> , <code>vertices</code>
CLxxx	Ligne de contact (maçonnerie)	longueur déterminée par la brique
PT2Dx	Point de contact 2D	—

Contacteurs 3D (pour RBDY3)

Contacteur	Description	Paramètre pylmgc90
SPHER	Sphère	<code>r</code> (rayon)
PLANx	Plan semi-infini	vecteur normal
CYLND	Cylindre	<code>r</code> , longueur
DNLYC	Demi-cylindre	<code>r</code> , longueur
POLYR	Polyèdre convexe 3D	<code>vertices</code>
PT3Dx	Point de contact 3D	—

Contacteurs pour corps déformables (pour MAILx)

Contacteur	Description
CLxxx	Ligne de contact sur arête EF (2D)
ALpxx	Ligne de polygone (interface EF 2D)

Couleurs et correspondance

La couleur (`colorCandidat`, `colorAntagoniste`) est le **critère de filtrage** principal. Seuls les contacteurs portant exactement la couleur déclarée dans la table de visibilité sont testés pour le contact. Cela permet de créer des règles sélectives :

- Faire interagir uniquement les particules bleues entre elles : `colorCandidat='BLUEx'`, `colorAntagoniste='BLUEx'`

- Faire interagir des particules rouges avec un mur gris : `colorCandidat='REDxx'`, `colorAntagoniste='GRAYx'`
- Contact entre disques et plan fixe : `RBDY2/DISKx/BLUEx` ↔ `RBDY3/PLANx/GRAYx`

Erreur courante : si aucune interaction n'est détectée pendant le calcul, vérifier que les couleurs dans la table de visibilité correspondent **exactement** (5 caractères, sensibles à la casse) aux couleurs déclarées sur les contacteurs des avatars.

Distance d'alerte (`alert`)

La distance `alert` détermine la zone de recherche de contact. Deux contacteurs sont mis en liste candidate pour le calcul si la distance entre eux est inférieure à `alert`.

Situation	Valeur recommandée
Particules granulaires ($r \approx 0,05$ à $0,15$ m)	0.1 à 0.3 m
Maçonnerie (briques standard)	0.02 à 0.05 m
Grandes structures	5 à 10 % du rayon maximal
Corps déformables EF	Taille caractéristique d'un élément

Trop petite : certains contacts réels ne sont pas détectés → interpénétration non gérée (très minim). **Trop grande** : trop de paires candidates → calcul ralenti inutilement.

Gestion des règles

Créer une règle

Remplir le formulaire et cliquer sur **■ Créer**. La règle sera créée :

1. Vérifie que la loi de contact référencée existe dans `_pymgc_laws`.
2. Crée l'objet `see_table` `pymgc90` via `LMGC90Bridge.create_visibility_rule()`.
3. L'ajoute au conteneur `_visibility_container` (collection `sees`).
4. Sauvegarde dans `state.visibility_rules`.
5. Émet le signal `rule_created` → `_refresh_all()`.

Modifier une règle

Double-cliquer dans la liste ou sélectionner et cliquer sur **—■ Modifier**. Après confirmation, `update_visibility_rule()` :

1. Reconstitue **entièrement** le conteneur `_visibility_container` (limitation `pymgc90` — pas de modification in-place d'une `see_table` existante).
2. Réinsère toutes les règles avec les nouvelles valeurs.
3. Émet le signal `rule_updated` → `_refresh_all()`.

Supprimer une règle

Sélectionner et cliquer sur **■■ Supprimer**. La règle est retirée de `state.visibility_rules` via `remove_visibility_rule()`. Le signal `rule_deleted` est émis.

Mise à jour en cascade : renommer une loi de contact dans l'onglet Contact met à jour automatiquement le champ `behavior_name` de toutes les règles de visibilité qui y font référence.

Règles courantes par type de simulation

Granulométrie 2D — disques rigides

```
CorpsCandidat : RBDY2 candidat : DISKx colorCandidat : BLUEx  
CorpsAntagoniste : RBDY2 antagoniste : DISKx colorAntagoniste : BLUEx  
behav : IQS_CLB alert : 0.1
```

Maçonnerie 2D — briques rigides

```
CorpsCandidat : RBDY2 candidat : CLxxx colorCandidat : BLUEx  
CorpsAntagoniste : RBDY2 antagoniste : CLxxx colorAntagoniste : BLUEx  
behav : IQS_CLB alert : 0.02
```

Granulométrie 3D — sphères rigides

```
CorpsCandidat : RBDY3 candidat : SPHER colorCandidat : BLUEx  
CorpsAntagoniste : RBDY3 antagoniste : SPHER colorAntagoniste : BLUEx  
behav : IQS_CLB alert : 0.1
```

Rigide / déformable — disque sur maillage EF

```
CorpsCandidat : RBDY2 candidat : DISKx colorCandidat : BLUEx  
CorpsAntagoniste : MAILx antagoniste : CLxxx colorAntagoniste : VERTx  
behav : GAP_SGR_CLB alert : 0.05
```

Corps de types différents — disques et joncs

```
CorpsCandidat : RBDY2 candidat : DISKx colorCandidat : BLUEx  
CorpsAntagoniste : RBDY2 antagoniste : JONCx colorAntagoniste : REDxx  
behav : IQS_CLB alert : 0.15
```

Lien avec l'assistant de projet

L'**Assistant de projet** (**Ctrl+Shift+N**) propose une page **Visibilité** qui pré-remplit automatiquement la table de visibilité en fonction des choix effectués aux étapes précédentes :

- **Corps** : **RBDY2** (2D) ou **RBDY3** (3D) selon la dimension du projet.
- **Contacteur** : **DISKx** (2D) ou **SPHER** (3D).
- **Couleurs** : synchronisées automatiquement avec la couleur de l'avatar créé à l'étape Avatar.
- **Loi** : la loi créée à l'étape Contact.

Pour des configurations plus complexes (plusieurs types de contacteurs, plusieurs lois), utiliser directement l'onglet Visibilité après la fin de l'assistant.

Remarques importantes

La loi doit exister avant la règle : `add_visibility_rule()` vérifie que la loi référencée est présente dans `_pylmgc_laws`. Si la loi est supprimée de l'onglet Contact après la création de la règle, la règle devient invalide et la génération du script échouera. L'onglet Contact avertit si l'on tente de supprimer une loi utilisée par une règle de visibilité.

Reconstruction complète lors de la modification : pylmgc90 ne permet pas de modifier une `see_table` existante. La moindre modification d'une règle entraîne la reconstruction complète du conteneur `sees` avec toutes les règles. Ce comportement est transparent pour l'utilisateur mais peut être lent si le projet contient de nombreuses règles.

Multiplicité des règles : un projet peut avoir autant de règles de visibilité que nécessaire. Pour un assemblage avec plusieurs populations de particules de couleurs différentes (ex. bleu, rouge, vert), créer une règle par paire de couleurs qui doit interagir. Des populations de couleurs différentes sans règle commune ne se voient pas.

Détecteur de contact vs table de visibilité : la table de visibilité déclare *qui* peut interagir. L'onglet Calcul (`Ctrl+8`) configure les *détecteurs de contact* (`DKDKx`, `SPSPx`...) qui déterminent *comment* le contact est résolu. Les deux doivent être cohérents — si une règle déclare `DISKx/DISKx`, le détecteur `DKDKx` doit être activé dans l'onglet Calcul.

Assistant de maçonnerie

Assistant de maçonnerie

L'**Assistant de maçonnerie** guide pas à pas la création de structures maçonnerie dans LMGC90_GUI. Il génère automatiquement des empilements de briques rigides via l'API pylmgc90 (**brick2D**, **brick3D**, **paneresse_simple**, **paneresse_double**), en gérant les fonctions de générations, les joints, les transformations et les groupes d'avatars.

Lancer l'assistant

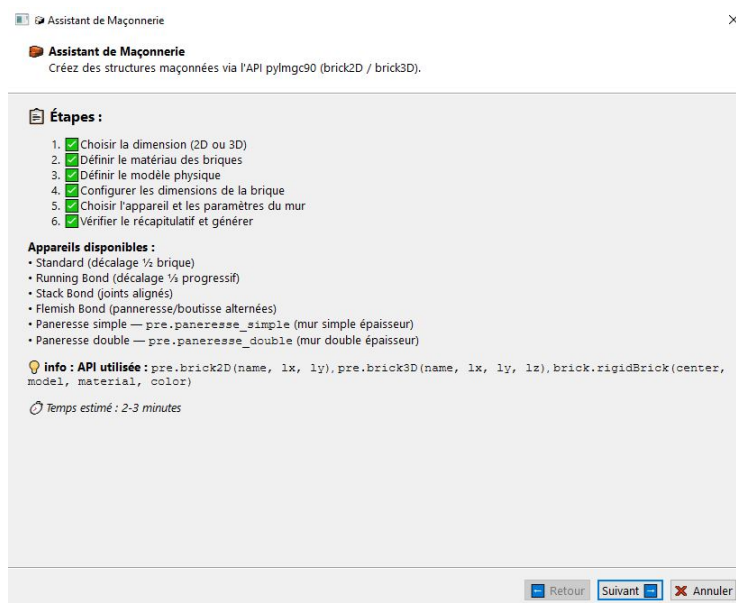
Méthode	Action
Menu	Assistants → Assistant de maçonnerie...
Raccourci clavier	Ctrl+Shift+M

Annulation possible à tout moment via le bouton **■ Annuler**, sans modification du projet.

Vue d'ensemble des étapes

L'assistant est composé de **8 pages** parcourues séquentiellement.

Page	Titre	Description
0	Introduction	Présentation des appareils et de l'API utilisée
1	Dimension	2D (brick2D) ou 3D (brick3D)
2	Matériau	Créer ou réutiliser un matériau de type RIGID
3	Modèle	Créer ou réutiliser un modèle Rxx2D / Rxx3D
4	Dimensions de la brique	lx , ly , lz et nom de la brique
5	Appareil de maçonnerie	Type d'appareil, dimensions du mur, position, couleur, groupe
6	Transformations	Translation, rotation, copie décalée post-génération
7	Récapitulatif	Vérification avant génération



captures/assistant_maçon_page1.JPG

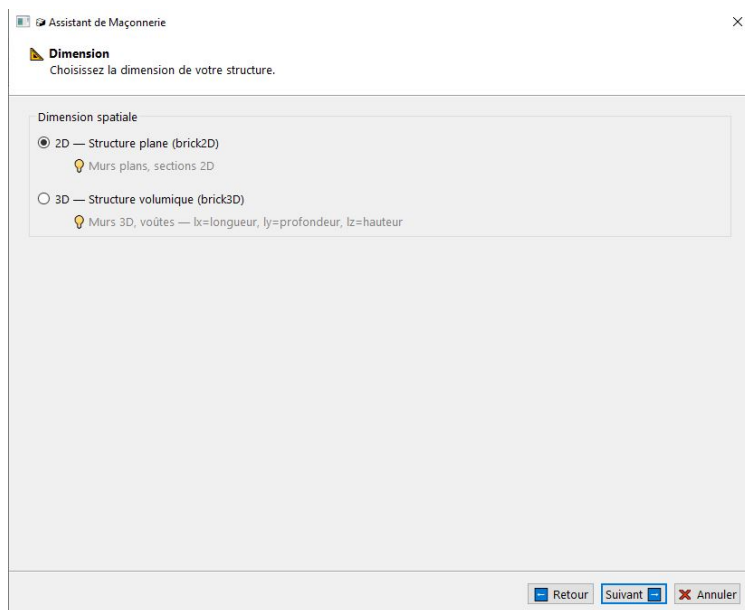
API pylmgc90 utilisée :

- `pre.brick2D(name, lx, ly)` — brique 2D (longueur × hauteur)
- `pre.brick3D(name, lx, ly, lz)` — brique 3D (longueur × profondeur × hauteur)
- `brick.rigidBrick(center, model, material, color)` — avatar pylmgc90 depuis la brique
- `pre.panerese_simple(brick_ref, disposition)` — mur simple épaisseur géré par pylmgc90
- `pre.panerese_double(brick_ref, disposition)` — mur double épaisseur géré par pylmgc90

Page 1 — Dimension

Choix	API pylmgc90	Description
2D — Structure plane	<code>pre.brick2D(name, lx, ly)</code>	Murs plans, sections 2D. ly est la hauteur de la brique.
3D — Structure volumique	<code>pre.brick3D(name, lx, ly, lz)</code>	Murs 3D, voûtes. ly est la profondeur , lz est la hauteur .

La valeur **2D** est sélectionnée par défaut.



captures/assistant_maçon_page2.JPG

Effet sur les étapes suivantes : la dimension conditionne le label de **ly** (hauteur en 2D / profondeur en 3D), la visibilité de **lz** et de l'offset Z, l'élément du modèle (**Rxx2D** ou **Rxx3D**), et les axes disponibles pour la rotation.

Page 2 — Matériau des briques

Deux modes disponibles :

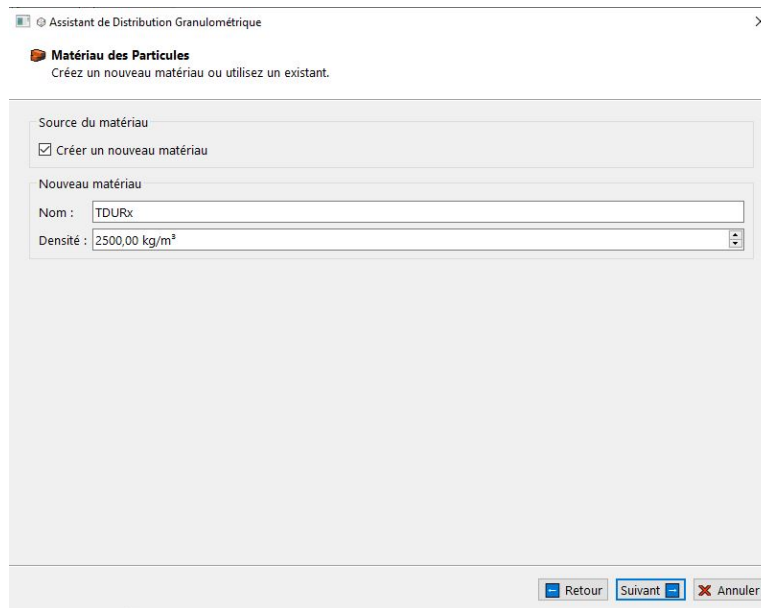
Mode A — Créer un nouveau matériau (coché par défaut)

Champ	Description	Valeur par défaut
Nom	Identifiant du matériau. 5 caractères maximum (contrainte LMGC90).	brick
Densité	Masse volumique (kg/m³). Plage : 100 à 20 000.	1800 kg/m³ (maçonnerie typique)

Le type est toujours **RIGID** — les briques sont des corps rigides. Il n'est pas nécessaire de spécifier des propriétés élastiques.

Mode B — Utiliser un matériau existant

Liste déroulante de tous les matériaux déjà définis dans l'onglet Matériau. Tous les types sont acceptés, mais **RIGID** est recommandé pour les briques.



captures/assistant_granulo_page3.JPG

Validation : le bouton Suivant est bloqué si le nom est vide (mode A) ou si aucun matériau valide n'est disponible (mode B).

Page 3 — Modèle physique

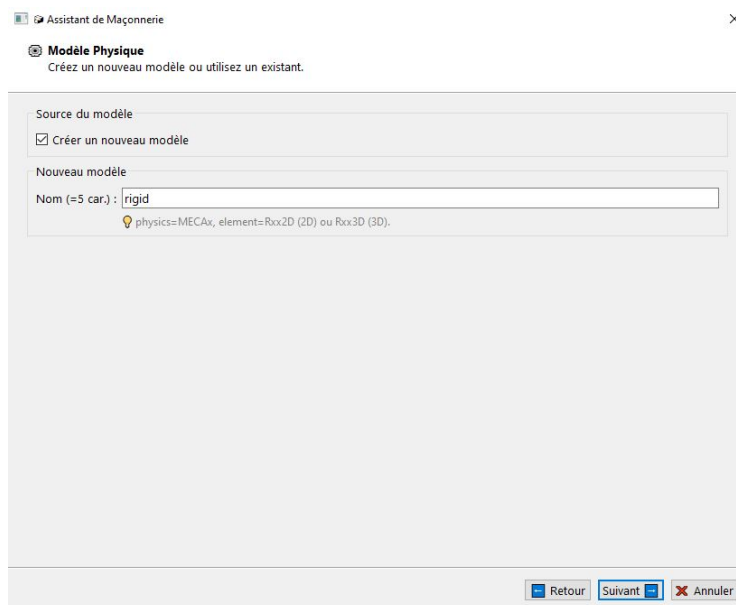
Deux modes disponibles :

Mode A — Créer un nouveau modèle _(coché par défaut)_

Champ	Description	Valeur par défaut
Nom	Identifiant du modèle. 5 caractères maximum.	rigid
Physique	Toujours MECAx pour les briques rigides.	MECAx (automatique)
Élément	Adapté automatiquement à la dimension.	Rxx2D (2D) ou Rxx3D (3D)

Mode B — Utiliser un modèle existant

Liste déroulante des modèles définis dans l'onglet Modèle.



captures/assistant_maçon_page4.JPG

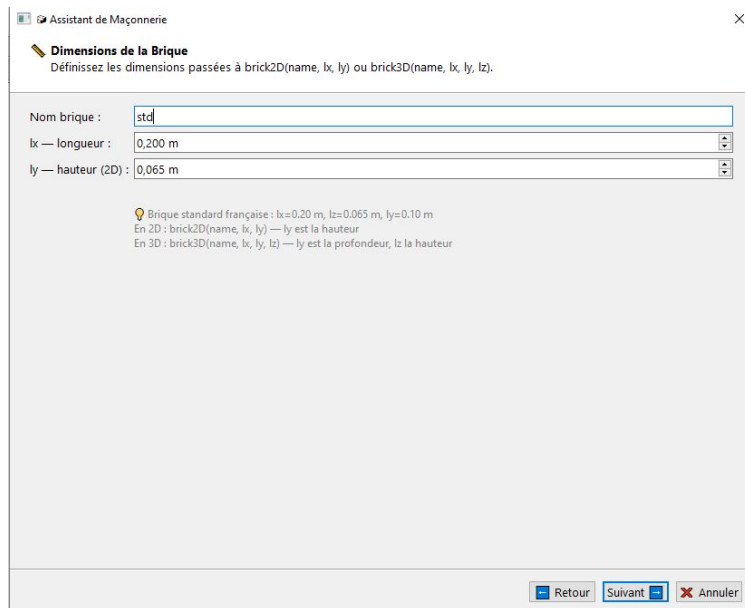
Validation : le bouton Suivant est bloqué si le nom est vide (mode A) ou si aucun modèle valide n'est disponible (mode B).

Page 4 — Dimensions de la brique

Définit la géométrie de la brique de référence passée à `brick2D()` ou `brick3D()`.

Champ	Description	2D	3D	Valeur par défaut
Nom brique	Identifiant interne pylmgc90 (8 caractères max). Utilisé comme premier argument de <code>brick2D</code> / <code>brick3D</code> .	■	■	std
lx — longueur	Longueur de la brique dans la direction X (m).	■	■	0.200 m
ly — hauteur (2D)	En 2D : hauteur de la brique. Argument <code>ly</code> de <code>brick2D</code> .	■	—	0.065 m
ly — profondeur (3D)	En 3D : profondeur (épaisseur du mur). Argument <code>ly</code> de <code>brick3D</code> .	—	■	0.100 m
lz — hauteur (3D)	En 3D uniquement : hauteur de la brique. Argument <code>lz</code> de <code>brick3D</code> .	—	■	0.065 m

Brique standard française : $lx = 0,20\text{ m} \cdot ly = 0,10\text{ m} \cdot lz = 0,065\text{ m}$ (format NF EN 771-1). **Convention pylmgc90 :** `brick2D(name, lx, ly)` avec `ly` = hauteur. `brick3D(name, lx, ly, lz)` avec `ly` = profondeur, `lz` = hauteur.



captures/assistant_maçon_page5.JPG

Page 5 — Appareil de maçonnerie

Page principale de configuration. Définit le type d'empilement, les dimensions du mur, la position, la couleur et le groupe d'avatars.

Types d'appareils disponibles

Standard — Décalage demi-brique

Chaque rang impair est décalé d'une demi-brique ($1x/2$) par rapport au rang pair. Appareil le plus courant, très résistant aux efforts verticaux.

```
Rang 2 : [=====][=====][=====]
Rang 1 : [=====][=====][=====]
Rang 0 : [=====][=====][=====]
```

Calcul des centres :

```
cx = offset_x + col × (lx + joint) + (lx/2 si rang impair) + lx/2
cy = offset_y + rang × (ly + joint) + ly/2
```

Running Bond — Décalage progressif d'un tiers

Le décalage augmente d'un tiers de brique ($1x/3$) à chaque rang, puis revient à zéro tous les trois rangs.

```
Rang 3 : [=====][=====][=====]
Rang 2 : [=====][=====][=====]
Rang 1 : [=====][=====][=====]
Rang 0 : [=====][=====][=====]
```

Calcul des centres :

```
cx = offset_x + col × (lx + joint) + (rang % 3) × (lx/3) + lx/2
```

Stack Bond — Joints alignés

Tous les joints verticaux sont parfaitement alignés. Aucun décalage entre les rangs. Esthétique, mais mécaniquement moins résistant (absence d'accrochage entre rangs).

```
Rang 2 : [=====] [=====] [=====]
Rang 1 : [=====] [=====] [=====]
Rang 0 : [=====] [=====] [=====]
```

Calcul des centres :

```
cx = offset_x + col * (lx + joint) + lx/2
cy = offset_y + rang * (ly + joint) + ly/2
```

Flemish Bond — Panneresse / Boutisse alternées

Chaque rang alterne des briques en panneresse (longueur lx) et des briques en boutisse (longueur $lx/2$). Le motif s'inverse d'un rang à l'autre.

```
Rang 1 : [1/2][==lx==][1/2][==lx==]
Rang 0 : [==lx==][1/2][==lx==][1/2]
```

Calcul : si $(rang + col) \% 2 == 0 \rightarrow$ panneresse ($brick_lx = lx$), sinon boutisse ($brick_lx = lx/2$). La position X est calculée avec un curseur cumulatif.

Fonctions seulement en 3D

Panneresse simple — `pre.panneresse_simple`

Mur de **simple épaisseur** généré par l'API `pymgc90` avec gestion automatique des demi-briques aux extrémités, des joints et de la hauteur.

Options spécifiques à cet appareil :

Option	Description	Valeurs
Disposition	Orientation des briques dans le rang.	<code>paneresse</code> · <code>boutisse</code> · <code>chant</code>
Première brique	Type de brique en début de premier rang.	<code>1</code> · <code>1/2</code> · <code>1/4</code> · <code>3/4</code>
Dimensionnement	Méthode pour définir la largeur du mur.	Nombre de briques · Longueur totale (m)
Longueur totale	Visible uniquement si Dimensionnement = « Longueur totale ». Appelle <code>setFirstRowByLength()</code> .	Ex : <code>3.0 m</code>
Sans demi-briques	Si coché, appelle <code>buildRigidWallWithHalfBricks()</code> au lieu de <code>buildRigidWall()</code> .	case à cocher

Dispositions :

Disposition	Description
<code>paneresse</code>	Briques posées dans le sens de la longueur (face visible = grand côté).
<code>boutisse</code>	Briques posées en travers (face visible = petit côté, $lx/2$).
<code>chant</code>	Briques posées sur le chant (hauteur = largeur de la brique).

Paneresse double — `pre.paneresse_double`

Mur de **double épaisseur** (deux rangées de briques côte à côte). Toutes les options de la paneresse simple sont disponibles. L'API `pymgc90` gère automatiquement le décalage entre les deux épaisseurs.

Dimensions du mur

Champ	Description	Défaut
Nombre de rangs	Hauteur du mur en nombre de rangées de briques. Plage : 1 à 200.	10
Nombre de colonnes	Largeur du mur en nombre de briques par rang. Plage : 1 à 200.	15
Épaisseur du joint	Épaisseur du mortier entre briques (m). Plage : 0 à 0,05.	0.010 m

Estimation automatique dans le récapitulatif : $\text{Largeur du mur} \approx \text{nb_colonnes} \times (\text{lx} + \text{joint})$ $\text{Hauteur du mur} \approx \text{nb_rangs} \times (\text{ly} + \text{joint})$

Avertissement au-delà de 5 000 briques : une boîte de confirmation demande validation avant de continuer, car la génération peut prendre plusieurs secondes.

Position initiale

Coordonnées du **coin inférieur gauche** du mur (en mètres). Les centres des briques sont calculés depuis ce point.

Champ	Description	Défaut
Offset X	Abscisse du coin inférieur gauche.	0.0 m
Offset Y	Ordonnée du coin inférieur gauche.	0.0 m
Offset Z	Cote Z du mur (3D uniquement).	0.0 m

Options

Option	Description	Défaut
Couleur LMGC90	Code couleur à 5 caractères.	BLUEx
Stocker dans un groupe	Si coché, enregistre tous les avatars générés dans un groupe nommé (<code>state.avatar_groups</code>). Le groupe est aussi sauvegardé dans <code>masonry_patterns</code> pour la reconstruction du script.	coché
Nom du groupe	Identifiant du groupe d'avatars.	mur_briques

Assistant de Maçonnerie
Appareil de Maçonnerie
Configurez la disposition des briques.
Type d'appareil
Appareil : Standard
Décalage d'une demi-brique (lx/2) entre rangs consécutifs.
Dimensions du mur
Nombre de rangs : 10
Nombre de colonnes : 15
Épaisseur joint : 0,010 m
Position initiale (coin inférieur gauche)
Offset X : 0,000 m
Offset Y : 0,000 m
Options
Couleur LMGC90 : BLUEx
☒ Stocker dans un groupe mur_briques
Retour Suivant Annuler

captures/assistant_maçon_page6.JPG

Page 6 — Transformations post-génération

Les transformations s'appliquent **après** la génération de toutes les briques, sur l'ensemble des corps pylmgc90 de la session courante. Elles correspondent aux appels pylmgc90 `bodies.translate()`, `bodies.rotate()` et `copy.deepcopy(bodies)`.

Toutes les transformations sont **désactivées par défaut** (cases décochées). Les champs sont grisés tant que la case correspondante n'est pas cochée.

Translation — `bodies.translate(dx, dy, dz)`

Déplace l'ensemble du mur d'un vecteur (dx, dy, dz).

Champ	Description	Défaut
Activer la translation	case à cocher	non coché
dx	Déplacement en X (m).	0.0 m
dy	Déplacement en Y (m).	0.0 m
dz	Déplacement en Z (m). 3D uniquement.	0.0 m

Appel pylmgc90 généré :

```
# En 2D
bodies.translate(dx=dx, dy=dy)
# En 3D
bodies.translate(dx=dx, dy=dy, dz=dz)
```

Rotation — `bodies.rotate(description='axis', center, axis, alpha)`

Fait pivoter l'ensemble du mur autour d'un axe passant par un centre donné. L'angle est saisi en degrés et converti automatiquement en radians.

Champ	Description	Valeurs	Défaut
Activer la rotation	case à cocher	—	non coché
Centre x	Coordonnée X du centre de rotation (m).	—	0.0 m
Centre y	Coordonnée Y du centre de rotation (m).	—	0.0 m
Centre z	Coordonnée Z du centre de rotation (m). 3D uniquement.	—	0.0 m
Axe	Axe de rotation. En 2D, seul Z est pertinent (fixé automatiquement).	Z · X · Y	Z
Angle α	Angle de rotation en degrés. Converti en radians avant l'appel pylmgc90.	-360° à +360°	90.0°

Appel pylmgc90 généré :

```
import math, numpy as np
alpha = math.radians(90.0)
axis = [0., 0., 1.] # axe Z
center = np.array([cx, cy, cz])
bodies.rotate(description='axis', center=center, axis=axis, alpha=alpha)
```

En 2D : l'axe est toujours Z (rotation dans le plan XY). Le champ Axe est grisé et fixé à Z.

Copie décalée — `copy.deepcopy(bodies)` + `translate`

Duplique le mur complet (après translation et rotation éventuelles) et lui applique un décalage. Crée ainsi un deuxième mur indépendant dans le projet.

Champ	Description	Défaut
Créer une copie décalée	case à cocher	non coché
Décalage dx	Décalage X de la copie (m).	0.0 m
Décalage dy	Décalage Y de la copie (m).	0.0 m
Décalage dz	Décalage Z de la copie (m). 3D uniquement.	0.0 m

Appel pylmgc90 généré :

```
import copy
bodies_copy = copy.deepcopy(bodies)
bodies_copy.translate(dx=dx_copy, dy=dy_copy) # ou dz en 3D
```

Usage typique : créer deux murs parallèles d'un seul coup (dx = épaisseur de la pièce), ou, combinée avec la rotation, créer l'angle d'un bâtiment.

Assistant de Maçonnerie

Transformations
Appliquez des transformations au mur après génération : translation, rotation, copie décalée.

Ces transformations s'appliquent sur le bodies_container pylmgc90 après buildRigidWall(). Elles correspondent à : bodies.translate(), bodies.rotate() et copy.deepcopy(bodies).

Translation — bodies.translate(dx, dy, dz)
☐ Activer la translation
dx : 0,0000 m
dy : 0,0000 m

Rotation — bodies.rotate(description='axis', center, axis, alpha)
☐ Activer la rotation
Centre x : 0,0000 m
Centre y : 0,0000 m
Centre z :
Axe : Z
Angle α : 90,00 °

Copie décalée — copy.deepcopy(bodies) + translate
☐ Créer une copie décalée du mur
Décalage dx : 0,0000 m
Décalage dy : 0,0000 m

Retour Suivant Annuler

captures/assistant_maçon_page7.JPG

Page 7 — Récapitulatif

Affiche un tableau HTML complet avant génération, incluant :

Section	Informations
Dimension	2D ou 3D
API pylmgc90	Appel <code>brick2D(...)</code> ou <code>brick3D(...)</code> avec les valeurs exactes
Matériau	Nom et densité (nouveau) ou nom existant
Modèle	Nom (nouveau) ou nom existant
Dimensions brique	lx, ly, lz
Appareil	Nom du pattern sélectionné
Rangs × colonnes	Nombre total estimé de briques
Épaisseur joint	Valeur en mètres
Taille mur estimée	Largeur × Hauteur calculées depuis les paramètres
Position	Offset X, Y (et Z en 3D)
Couleur	Code couleur LMGC90
Groupe	Nom du groupe (si activé)
Translation	dx, dy, dz (si activée)
Rotation	Axe, angle α , centre (si activée)
Copie décalée	dx, dy, dz de la copie (si activée)
Options panerese	Disposition, première brique, mode dimensionnement, sans demi-briques (si panerese)

Avertissement automatique : si le nombre total de briques dépasse 1 000, un message orange signale que la génération peut être lente.

Cliquer sur **■ Générer** pour créer la structure. Un message de confirmation indique le nombre de briques créées.

Résultat de la génération

À la fin de l'assistant, les éléments suivants sont créés dans le projet :

Élément	Description
Matériau	Ajouté à l'onglet Matériau (si créé). Type RIGID , densité configurée.
Modèle	Ajouté à l'onglet Modèle (si créé). MECAx + Rxx2D ou Rxx3D .
Avatars briques	Un avatar EMPTY_AVATAR par brique, avec wall_params stockés pour reconstruction. Couleur configurée.
Groupe	Groupe nommé dans state.avatar_groups regroupant tous les indices des avatars générés (si l'option est activée).
masonry_patterns	Dictionnaire sauvegardé dans le state pour la reconstruction du script de génération par ScriptGenerator .

Reconstruction du script : les paramètres complets du pattern (appareil, dimensions, joints, transformations, etc.) sont sauvegardés dans **state.masonry_patterns[group_name]**. Le générateur de script (**script_generator.py**) utilise ces données pour reproduire fidèlement la génération **pylmgc90** lors de l'export.

Exemples d'usage

Mur standard 2D simple

```
Dimension : 2D
Matériau : brick - RIGID - 1800 kg/m³
Modèle : rigid - MECAX - Rxx2D
Brique : std - lx=0.200 m, ly=0.065 m
Appareil : Standard
Rangs x col. : 10 x 15 (150 briques)
Joint : 0.010 m
Position : (0.0, 0.0)
Couleur : BLUEx
Groupe : mur_nord
Transformations: (aucune)
```

Mur en paneressse simple avec longueur fixe

```
Dimension : 3D
Matériau : stone - RIGID - 2200 kg/m³
Modèle : rigid - MECAX - Rxx3D
Brique : std - lx=0.200 m, ly=0.100 m, lz=0.065 m
Appareil : Paneressse simple (pylmgc90)
Disposition : paneressse
Première brique : 1/2
Dimensionnement : Longueur totale - 4.000 m
Sans demi-briques : Non
Rangs : 15
Joint : 0.010 m
Groupe : mur_facade
```

Deux murs parallèles via copie décalée

```
Appareil : Standard – 8 rangs × 12 colonnes
Translation : désactivée
Rotation : désactivée
Copie décalée : activée – dx=0.0, dy=3.5 m (largeur de la pièce)
```

Résultat : deux murs identiques distants de 3,5 m, créés en une seule exécution de l'assistant.

Remarques importantes

Nom de brique limité à 8 caractères : le champ Nom brique accepte jusqu'à 8 caractères, mais LMGC90 peut tronquer les identifiants. Utiliser des noms courts (**std**, **half**, **custom**).

Performance : au-delà de 5 000 briques, la génération et l'interface peuvent ralentir significativement. Pour les grands assemblages (> 10 000 briques), préférer la génération directe depuis un script Python.

Couleur par défaut : si le champ Couleur est laissé vide, la valeur **BLUEx** est utilisée automatiquement.

Transformations et centres : après une translation ou une rotation, les centres des avatars dans **state.avatars** sont mis à jour depuis les coordonnées réelles des nœuds pylmgc90 (**body.nodes[1].coor**). Les avatars de la copie décalée sont ajoutés à la suite avec leur propre centre calculé.

Réutilisabilité : l'assistant peut être lancé plusieurs fois sur le même projet. Chaque exécution ajoute un nouveau groupe de briques sans effacer les précédents, à condition d'utiliser des noms d'éléments et de groupes différents.

Corps déformables (EF)

Assistant Corps Déformable — Maillage EF

L'**Assistant Corps Déformable** guide pas à pas la création d'un corps déformable maillé par éléments finis dans LMGC90_GUI. Il génère automatiquement le maillage, crée le matériau et le modèle associés, applique les conditions aux limites, puis stocke l'avatar de type `MESH_DEFORMABLE` directement dans le projet.

Le maillage est construit via les fonctions pylmgc90 `pre.buildMesh2D()` ou `pre.buildMeshH8()` pour les géométries structurées, et via `gms` pour les géométries courbes (disque, sphère, cylindre). Dans tous les cas, le maillage brut est converti en avatar pylmgc90 via `pre.buildMeshedAvatar()`, qui produit un corps de type **MAILx** (maillage déformable). Les géométries complexes peuvent également être importées depuis un fichier externe via `pre.readMesh()`.

Lancer l'assistant

Méthode	Action
Menu	Assistants → Assistant de déformable...
Raccourci clavier	Ctrl+Shift+D

Annulation : cliquer sur **■ Annuler** à n'importe quelle étape ferme l'assistant sans modifier le projet. L'état du projet est entièrement restauré.

Vue d'ensemble des étapes

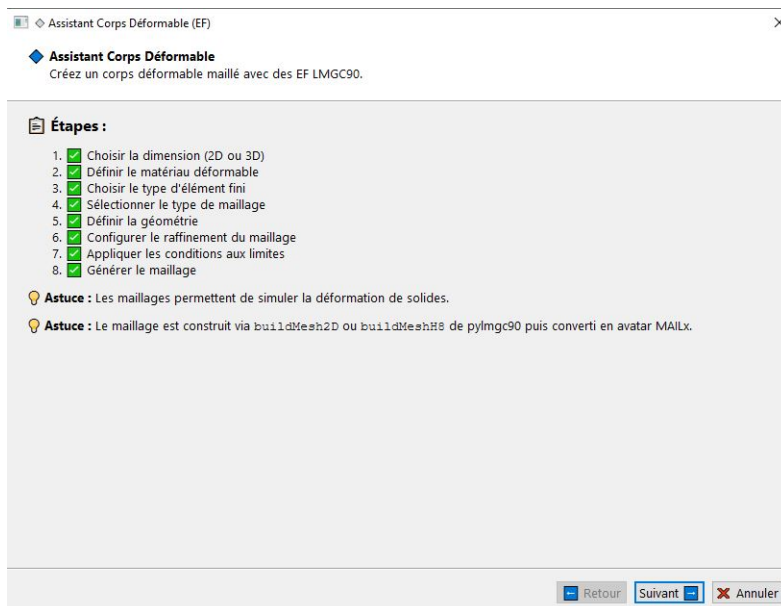
L'assistant est composé de **8 pages** parcourues séquentiellement. Les boutons **■■ Retour** et **Suivant ➡■** permettent de naviguer librement.

Page	Titre	Description
0	Introduction	Présentation de l'assistant et des étapes
1	Dimension	2D ou 3D
2	Matériau	Créer ou réutiliser un matériau élastique
3	Modèle EF	Créer ou réutiliser un modèle éléments finis
4	Géométrie	Forme et dimensions du corps
5	Raffinement	Finesse du maillage
6	Conditions aux limites (DOF)	Conditions de Dirichlet et de chargement
7	Récapitulatif	Vérification avant génération

Page 0 — Introduction

Présentation des 8 étapes à venir. Aucune saisie requise. Cliquer sur **Suivant ➡■** pour commencer.

Le maillage généré est un avatar **MESH_DEFORMABLE** de couleur **CYANx** ajouté automatiquement à la liste des avatars du projet. Ses paramètres de reconstruction sont sauvegardés dans **mesh_params** pour permettre le rechargement du projet.



captures/assistant_defor_page1.JPG

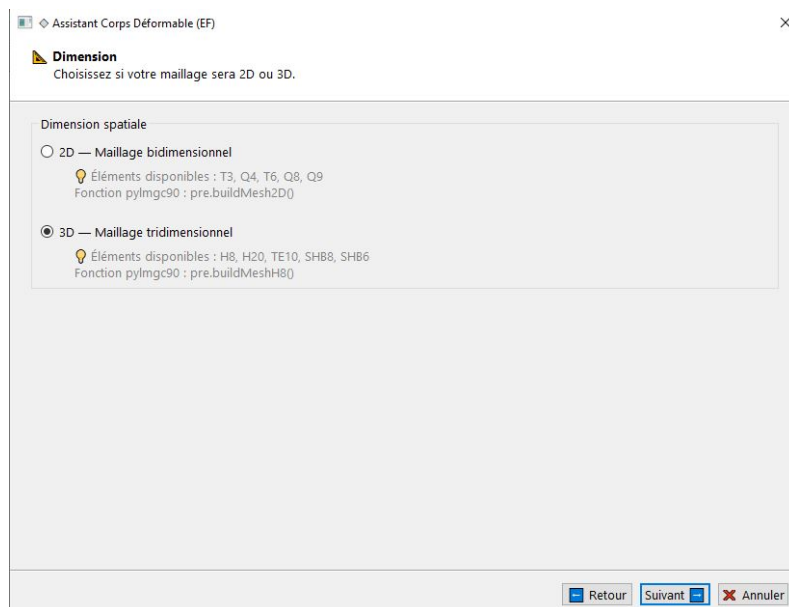
Page 1 — Dimension

Choisir entre deux options exclusives :

Choix	Fonction pylmgc90 utilisée	Géométries disponibles
2D	<code>pre.buildMesh2D()</code> + gmsh	Rectangle, Disque, Fichier externe
3D	<code>pre.buildMeshH8()</code> + gmsh	Boîte (H8), Sphère, Cylindre, Fichier externe

La valeur **2D** est sélectionnée par défaut.

Effet sur les étapes suivantes : la dimension conditionne les types d'éléments disponibles à la page *Modèle*, les formes disponibles à la page *Géométrie*, les paramètres de raffinement, et les groupes de surface (**down/up/left/right** en 2D, avec **front/rear** en plus en 3D).



captures/assistant_defor_page2.JPG

Page 2 — Matériau déformable

Cette page propose deux modes :

Mode A — Utiliser un matériau existant (si des matériaux élastiques existent dans le projet)

Seuls les matériaux de types élastiques sont listés : **ELAS**, **ELAS_DILA**, **VISCO_ELAS**, **ELAS_PLAS**, **THERMO_ELAS**, **PORO_ELAS**. Les matériaux **RIGID** ne sont pas proposés car incompatibles avec un corps déformable.

Mode B — Créer un nouveau matériau

Cocher **Créer un nouveau matériau à la place** pour afficher le formulaire. Les champs conditionnels s'affichent ou se masquent automatiquement selon le type choisi.

Champs communs à tous les types élastiques

Champ	Description	Valeur par défaut
Nom	Identifiant du matériau. 5 caractères maximum.	ELAS1
Type	Type de comportement mécanique. Voir tableau ci-dessous.	ELAS
Densité	Masse volumique (kg/m³). Plage : 10 à 25 000.	2700 kg/m³ (aluminium)
Module de Young	Rigidité du matériau (Pa). Plage : 10³ à 10¹².	70 × 10⁹ Pa
Coefficient de Poisson (ν)	Compressibilité latérale (sans dimension). Plage : 0 à 0,4999.	0.3

Types de matériaux disponibles et champs conditionnels

Type	Champs supplémentaires	Description
ELAS	_(aucun)_	Élasticité linéaire standard.

Type	Champs supplémentaires	Description
ELAS_DILA	Dilatation thermique (K^{-1} , défaut $1e-5$), T_{ref_meca} ($^{\circ}C$, défaut 20.0)	Élasticité avec couplage thermique unilatéral.
VISCO_ELAS	Young visqueux (Pa, défaut 1.17×10^9), Poisson visqueux (défaut 0.35)	Visco-élasticité de Kelvin-Voigt.
ELAS_PLAS	Limite élastique iso_hard (Pa, défaut 4×10^9), Module d'écrouissage isoh_coeff (Pa, défaut 10^9)	Élasto-plasticité J2 avec écrouissage isotrope linéaire, critère Von-Mises.
THERMO_ELAS	_(young et nu mis à 0 — définis par le modèle)_	Thermo-élasticité couplée. <code>conductivity='field'</code> et <code>specific_capacity='field'</code> .
PORO_ELAS	_(young et nu mis à 0 — définis par le modèle)_	Poro-élasticité selon la théorie de Biot. <code>hydro_cpl=0.0</code> par défaut.

Propriétés générées automatiquement : `elas='standard'` et `anisotropy='isotropic'` sont toujours ajoutés. Pour ELAS_PLAS, le critère est toujours Von-Mises, l'écrouissage `isoh='linear'`, et les paramètres `cinh='none'`, `visc='none'`.

captures/assistant_defor_page3.JPG

Page 3 — Modèle éléments finis

Même logique que la page Matériau : réutiliser un modèle existant compatible avec la dimension, ou en créer un nouveau.

Mode A — Utiliser un modèle existant

Seuls les modèles dont la dimension correspond à celle choisie à la page 1 sont listés.

Mode B — Créer un nouveau modèle

Champ	Description	Valeur par défaut
Nom	Identifiant du modèle. 5 caractères maximum.	femxx
Physique	Physique résolue.	MECAx , THERx ou HYDRx
Élément fini	Type d'élément. Adapté automatiquement à la dimension.	Voir tableau ci-dessous
Anisotropie	iso__ (isotrope) ou ortho (orthotrope).	iso__
Cinématique	small (petites déformations) ou large (grandes déformations).	small
Formulation	UpdtL (lagrangien actualisé) ou Total (lagrangien total).	UpdtL
Stockage masse	lump_ (masse concentrée) ou coher (masse cohérente).	lump_

Une description de l'élément sélectionné s'affiche en italique sous la liste déroulante.

Éléments disponibles par dimension

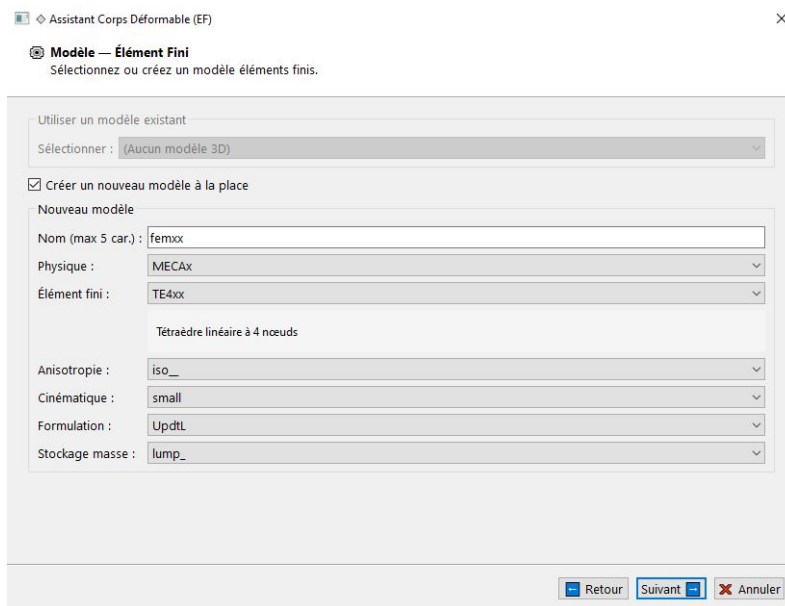
En 2D :

Élément	Description
T3xxx	Triangle linéaire à 3 nœuds
Q4xxx	Quadrangle bilinéaire à 4 nœuds
T6xxx	Triangle quadratique à 6 nœuds
Q8xxx	Quadrangle serendipity à 8 nœuds
Q9xxx	Quadrangle biquadratique complet à 9 nœuds

En 3D :

Élément	Description
H8xxx	Hexaèdre trilinéaire à 8 nœuds
H20xx	Hexaèdre triquadratique à 20 nœuds
TE10x	Tétraèdre quadratique à 10 nœuds
SHB8x	Solide-coque hexaédrique SHB8 à 8 nœuds
SHB6x	Solide-coque prismatique SHB6 à 6 nœuds

*Les options **material='elas_'** et **external_model='no__'** sont ajoutées automatiquement dans les options du modèle lors de la génération.*



captures/assistant_defor_page4.JPG

Page 4 — Géométrie

Définit la forme et les dimensions du corps. La liste des formes disponibles dépend de la dimension choisie.

Formes 2D

Rectangle (maillage structuré natif pylmgc90)

Génère un maillage rectangulaire via `pre.buildMesh2D()`.

Champ	Description	Défaut
Centre X, Y	Position du centre du rectangle (m).	0.0, 0.0
Longueur X (lx)	Dimension horizontale (m).	1.0 m
Longueur Y (ly)	Dimension verticale (m).	1.0 m

Le coin inférieur gauche est calculé automatiquement : $x_0 = cx - lx/2$, $y_0 = cy - ly/2$.

Disque (via gmsh)

Génère un maillage de disque circulaire plein via **gmsh** (`addDisk`), algorithme Frontal-Delaunay, puis importe le fichier `.msh` v2.2.

Champ	Description	Défaut
Centre X, Y	Position du centre du disque (m).	0.0, 0.0
Rayon (r)	Rayon du disque (m).	0.5 m

Nécessite que **gmsh** soit installé et accessible en Python (`import gmsh`).

Fichier externe

Importe un maillage depuis un fichier existant via `pre.readMesh(filepath, 2)`.

Champ	Description
Fichier maillage	Chemin vers le fichier. Formats acceptés : <code>.msh</code> , <code>.vtk</code> , <code>.mesh</code> . Utiliser le bouton  Parcourir pour naviguer.

Formes 3D

Boîte (H8) (maillage structuré natif pylmgc90)

Génère un maillage hexaédrique via `pre.buildMeshH8()`.

Champ	Description	Défaut
Centre X, Y, Z	Position du centre de la boîte (m).	<code>0.0, 0.0, 0.0</code>
Longueur X (lx)	Dimension en X (m).	<code>1.0 m</code>
Longueur Y (ly)	Dimension en Y (m).	<code>1.0 m</code>
Longueur Z (lz)	Dimension en Z (m).	<code>1.0 m</code>

Sphère (via gmsh)

Génère un maillage de sphère pleine via **gmsh** (`addSphere`), algorithme Frontal 3D.

Champ	Description	Défaut
Centre X, Y, Z	Position du centre de la sphère (m).	<code>0.0, 0.0, 0.0</code>
Rayon (r)	Rayon de la sphère (m).	<code>0.5 m</code>

Cylindre (via gmsh)

Génère un maillage de cylindre plein via **gmsh** (`addCylinder`), d'axe Z, centré en (cx, cy, cz).

Champ	Description	Défaut
Centre X, Y, Z	Position du centre géométrique du cylindre (m).	<code>0.0, 0.0, 0.0</code>
Rayon (r)	Rayon du cylindre (m).	<code>0.5 m</code>
Hauteur (h)	Longueur axiale du cylindre (m).	<code>1.0 m</code>

Fichier externe (3D)

Importe un maillage 3D via `pre.readMesh(filepath, 3)`. Formats acceptés : `.msh`, `.vtk`, `.mesh`.

Assistant Corps Déformable (EF)

Géométrie
Définissez la forme et les dimensions de votre corps.

Forme : Fichier externe

Centre X (m) : 0,0000

Centre Y (m) : 0,0000

Centre Z (m) : 0,0000

Fichier maillage : C:/lmgc90_user_2023/examples/Tutorials/deformable/pre/gmsh_files/Donut.msh

Parcourir

Retour Suivant Annuler

captures/assistant_defor_page5.JPG

Page 5 — Raffinement du maillage

Définit la finesse du maillage. Les paramètres affichés dépendent de la géométrie sélectionnée à la page précédente.

Type de maillage structuré (Rectangle uniquement)

Type	Description
Q4	Quadrangles bilinéaires à 4 nœuds — rapide et robuste. Recommandé par défaut.
2T3	Triangles à 3 nœuds obtenus en coupant chaque Q4 en 2.
4T3	Triangles à 3 nœuds obtenus en coupant chaque Q4 en 4. Maillage plus isotrope.
Q8	Quadrangles serendipity à 8 nœuds. Plus précis, plus coûteux.

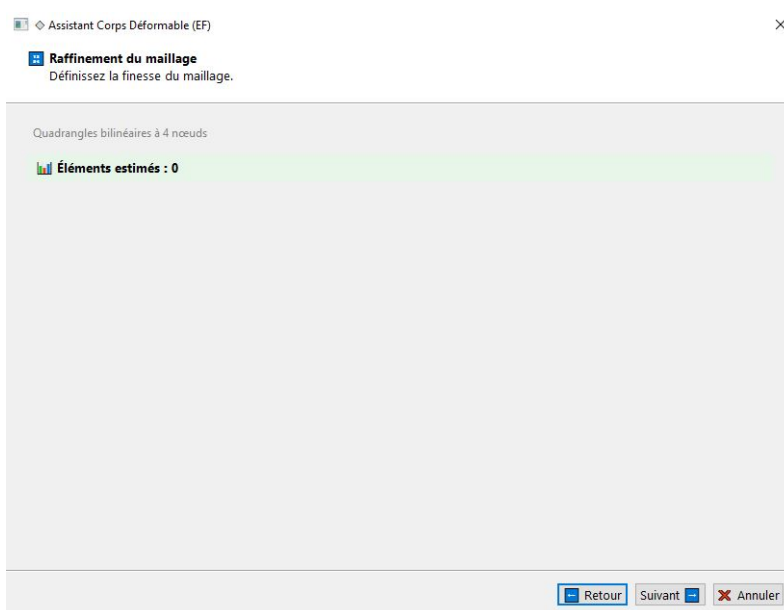
Paramètres de raffinement par géométrie

Géométrie	Paramètres	Estimation
Rectangle	$nx \times ny$ (défaut 10×10)	$nx \times ny$ éléments
Disque	nr (défaut 5) $\times$ $ntheta$ (défaut 16)	$nr \times ntheta$ éléments
Boîte (H8)	$nx \times ny \times nz$ (défaut $10 \times 10 \times 5$)	$nx \times ny \times nz$ éléments
Sphère	$nr \times ntheta \times nphi$ (défaut $5 \times 16 \times 8$)	$nr \times ntheta \times nphi$ éléments
Cylindre	$nr \times ntheta \times nz$ (défaut $5 \times 16 \times 5$)	$nr \times ntheta \times nz$ éléments
Fichier externe	_(pas de paramètre — maillage défini dans le fichier)_	—

Un compteur d'éléments estimés se met à jour en temps réel lors de la modification des paramètres.

Signification des paramètres gmsh :

Paramètre	Signification
nr	Nombre de couches d'éléments dans la direction radiale.
ntheta	Nombre d'éléments dans la direction angulaire (circonférence).
nphi	Nombre d'éléments dans la direction polaire (latitude, sphère uniquement).
nz	Nombre d'éléments dans la direction axiale Z.
nx, ny, nz	Nombre d'éléments dans les directions cartésiennes X, Y, Z.



captures/assistant_defor_page6.JPG

Conseil : commencer avec les valeurs par défaut pour vérifier la géométrie, puis augmenter **nx/ny/nr/ntheta** pour améliorer la précision du calcul.

Page 6 — Conditions aux limites (DOF)

Définit les conditions aux limites mécaniques appliquées sur les **groupes de surface** du maillage. Ces groupes sont créés automatiquement par **buildMesh2D** et **buildMeshH8**.

Groupes de surface disponibles

Groupe	Description	Disponible
down	Bord inférieur ($y = y_{\min}$ ou $z = z_{\min}$)	2D et 3D
up	Bord supérieur ($y = y_{\max}$ ou $z = z_{\max}$)	2D et 3D
left	Bord gauche ($x = x_{\min}$)	2D et 3D
right	Bord droit ($x = x_{\max}$)	2D et 3D
front	Face avant ($z = z_{\min}$)	3D uniquement
rear	Face arrière ($z = z_{\max}$)	3D uniquement

Note : pour les géométries courbes (disque, sphère, cylindre) importées depuis **gmsh**, les groupes de surface dépendent des entités physiques définies dans le fichier **.msh**. Les noms peuvent différer de ceux ci-dessus.

Créer une condition DOF

Cliquer sur + **Ajouter une condition DOF** pour créer une ligne. Chaque ligne contient trois colonnes :

Colonne	Description	Valeurs
Type DOF	Nature de la condition.	<code>imposeDrivenDof</code> ou <code>imposeInitValue</code>
Groupe	Groupe de surface sur lequel appliquer la condition.	<code>down</code> , <code>up</code> , <code>left</code> , <code>right</code> , <code>front</code> , <code>rear</code>
Paramètres	Arguments pylmgc90 au format <code>cle=valeur</code> , <code>cle=valeur</code> . Accepte les expressions Python.	Voir exemples ci-dessous

Types de conditions DOF

Type	Description
<code>imposeDrivenDof</code>	Degré de liberté piloté : impose un déplacement ou une vitesse sur la durée du calcul.
<code>imposeInitValue</code>	Valeur initiale : impose une position ou une vitesse à l'instant $t = 0$ uniquement.

Exemples de paramètres

Usage	Paramètres
Blocage de la translation (2D)	<code>component=[1,2]</code> , <code>dofty="vlocy"</code>
Blocage de la translation (3D)	<code>component=[1,2,3]</code> , <code>dofty="vlocy"</code>
Déplacement imposé en Y	<code>component=[2]</code> , <code>ct=0.001</code>
Valeur initiale nulle	<code>component=[1,2]</code> , <code>value=0.0</code>
Blocage en X uniquement	<code>component=[1]</code> , <code>dofty="vlocy"</code>

Assistant Corps Déformable (EF)

Conditions aux Limites
Notez les conditions souhaitées — à appliquer via l'onglet DOF.

Fixer :

- ☐ Bord inférieur (down)
- ☐ Bord supérieur (up)
- ☐ Bord gauche (left)
- ☐ Bord droit (right)
- ☐ Face avant (front) — 3D
- ☐ Face arrière (rear) — 3D

Chargement : ☐ Activer

Groupe cible : down

Valeur : 0,00 N/m²

Direction : X

Retour Suivant Annuler

captures/assistant_defor_page7.JPG

Traitement des paramètres : le champ Paramètres est analysé via `SafeEvaluator.eval_dict()` — les expressions Python simples sont autorisées (`[1,2]`, `0.001`, `"vlocy"`). Chaque condition est convertie en `DOFOperation` et transmise

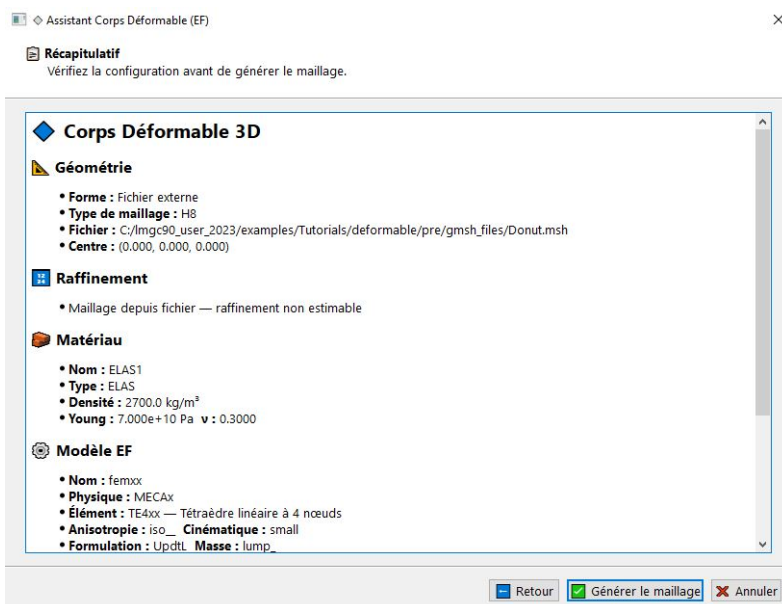
à `controller.add_dof_operation()`, qui applique la condition ET la sauvegarde dans `state.operations` (visible dans l'onglet DOF pour modification ultérieure).

Pour supprimer une ligne, cliquer sur le bouton **x** à droite de la ligne.

Page 7 — Récapitulatif

Affiche un résumé complet de la configuration avant génération.

Section	Informations affichées
Géométrie	Forme, type de maillage structuré (2D), dimensions, centre
Raffinement	Paramètres de discrétisation, nombre d'éléments estimé
Matériau	Nom, type, densité, Young, ν — ou matériau existant
Modèle EF	Nom, physique, élément, anisotropie, cinématique, formulation, stockage masse
Conditions DOF	Liste des conditions : type, groupe, paramètres



captures/assistant_defor_page8.JPG

Cliquer sur **■ Générer le maillage** pour lancer la génération. Un message de confirmation indique le nombre de nœuds et d'éléments créés.

En cas d'erreur : l'état du projet est entièrement restauré. Le message d'erreur détaille la cause (matériau introuvable, fichier manquant, paramètre invalide, gmsh non disponible, etc.).

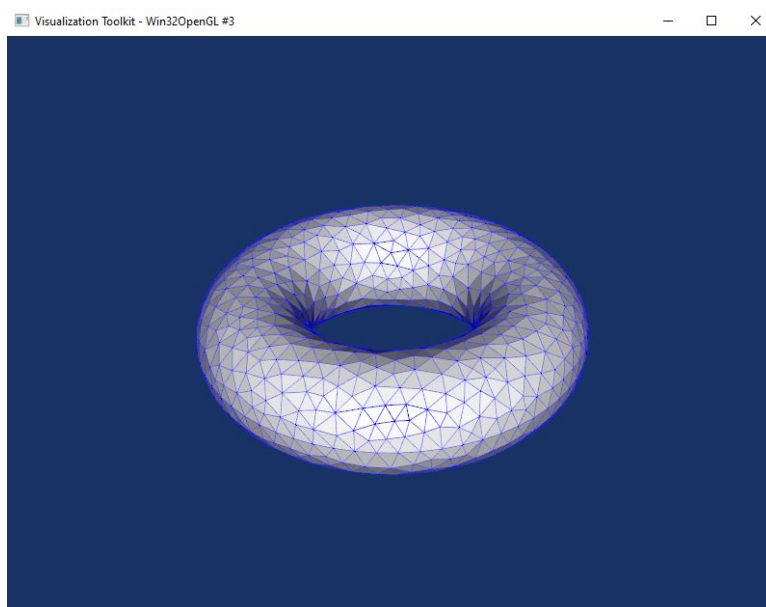
Résultat de la génération

À la fin de l'assistant, les éléments suivants sont créés dans le projet :

Élément	Description
Matériau	Ajouté à l'onglet Matériau (si créé).
Modèle EF	Ajouté à l'onglet Modèle (si créé).

Élément	Description
Avatar MESH_DEFORMABLE	Corps déformable de couleur CYANx , ajouté à la liste des avatars.
Conditions DOF	Sauvegardées dans state.operations , visibles dans l'onglet DOF.

Le corps déformable peut ensuite être enrichi via l'onglet **Avatar vide** (mode « Corps déformable existant ») pour y ajouter des contacteurs de surface permettant les interactions avec des corps rigides.



captures/assistant_defor_page9.JPG

Remarques importantes

Fichiers externes : les formats **.msh** (gmsh v2), **.vtk** et **.mesh** sont acceptés par **pre.readMesh()**. Le fichier doit être compatible avec la dimension choisie (2D ou 3D). Pour les fichiers gmsh, s'assurer que les entités physiques sont définies (surfaces en 2D, volumes en 3D).

Conditions DOF : les conditions aux limites créées dans l'assistant sont sauvegardées et visibles dans l'onglet DOF. Elles peuvent être modifiées ou supprimées après la génération sans avoir à relancer l'assistant.

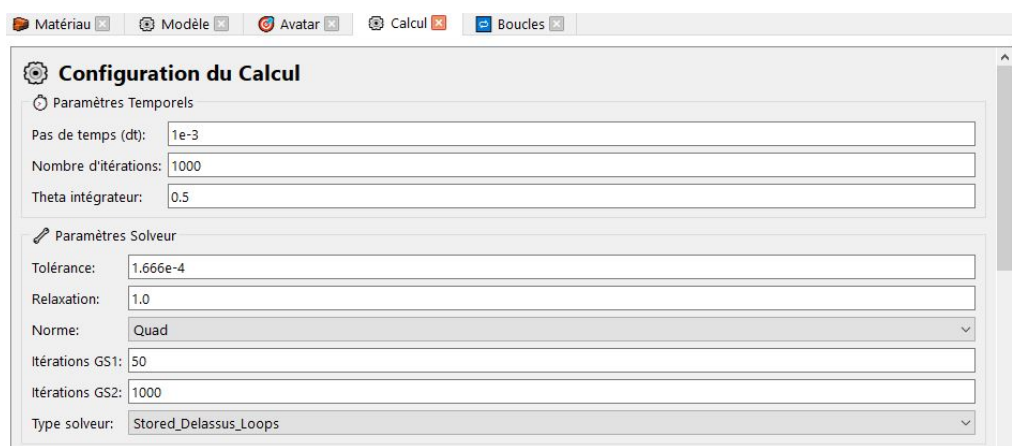
Estimation du nombre d'éléments : le compteur à la page Raffinement est une **estimation** basée sur le produit des paramètres de discrétisation. Le nombre réel peut différer légèrement pour les géométries courbes générées par gmsh.

Calcul

Vue d'ensemble

L'onglet **calcul** gère la configuration et le lancement des simulations LMGC90. Il génère automatiquement le DATBOX et le script **command.py**, puis exécute ce script.

Paramètres de calcul



captures/calcul.JPG

Paramètre	Champ UI	Valeur par défaut	Description
dt	Pas de temps	1e-3	Incrément temporel
nb_steps	Nombre d'itérations	1000	Nombre de pas de calcul
theta	Theta intégrateur	0.5	Paramètre du schéma d'intégration
tol	Tolérance	1.666e-4	Tolérance de convergence du solveur
relax	Relaxation	1.0	Facteur de relaxation
norm	Norme	Quad	Norme de convergence (Quad , QM , Maxim)
gs_it1	Itérations GS1	50	Nombre d'itérations Gauss-Seidel (boucle externe)
gs_it2	Itérations GS2	100	Nombre d'itérations Gauss-Seidel (boucle interne)
solver_type	Solveur	NLGS	Type de solveur (Stored_Delassus_Loops , Exchange_Local_Global , Exchange_Local_Global)
freq_write	Fréquence écriture	50	Écriture des résultats tous les N pas

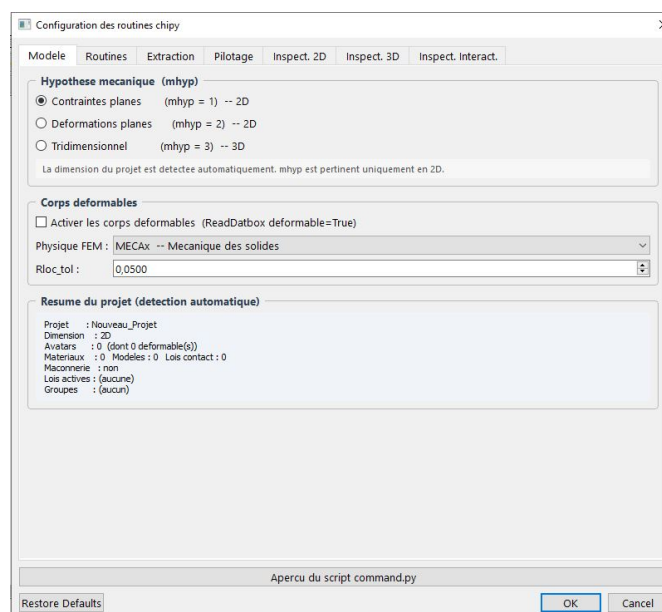
Paramètre	Champ UI	Valeur par défaut	Description
freq_display	Fréquence affichage	50	Mise à jour affichage tous les N pas

Fichiers générés au lancement d'un calcul

Fichier	Emplacement	Description
DATBOX/	Dossier du projet	Données d'entrée LMGC90
command.py	Dossier du projet	Script de calcul chipy
OUTBOX/	Dossier du projet	Résultats (créé par LMGC90)
Display/	Dossier du projet	Fichiers d'affichage
Postpro/	Dossier du projet	Fichiers post-traitement

Paramétrer vos calculs

Il est possible de paramétrer vos scripts calculs via au bouton "Configurer les routines chipy", une boite de dialogue s'ouvrira sur votre interface,



captures/config_calculs.JPG

1. Onglet Modèle

Le premier onglet fait l'objet d'une détection automatique sur l'hypothèse de votre modèle, et charge tous les paramètres selon vos avatars,

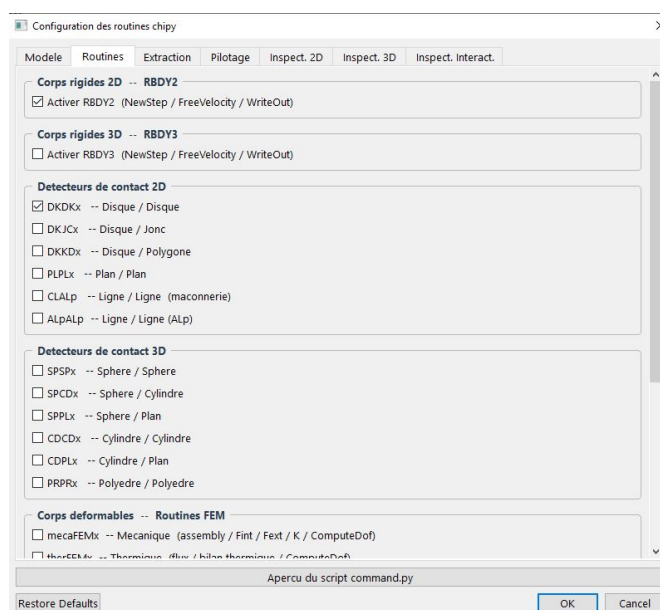
Contraintes planes (mhyp = 1)	Calcul 2D en état plan de contraintes. Pertinent pour les structures minces.
Déformations planes (mhyp = 2)	Calcul 2D en état plan de déformations. Pertinent pour les structures infiniment longues en z.
Tridimensionnel (mhyp = 3)	Calcul 3D complet.

Corps déformables

Activer les corps déformables	Génère ReadDatbox(deformable=True) dans le script. Active automatiquement mecaFEMx si la physique est MECAx.
Physique FEM	Détermine le solveur éléments finis utilisé : MECAx (mécanique des solides), THERx (thermique), HYDRx (hydraulique), ou THMx (thermo-hydraulique-mécanique couplé).
Rloc_tol	Tolérance sur les efforts de contact pour la reprise de Rloc. Valeur typique : 5×10^{-2} . Utilisée dans chipy.SetRlocTol().

2.Routines

Cet onglet sélectionne les routines chipy à inclure dans la boucle de calcul. Chaque case à cocher correspond à un appel de la famille NewStep / ComputeStep / WriteOut dans le script généré.



captures/config_calculs_routines.JPG

Corps rigides 2D — RBDY2

– RBDY2 (NewStep / FreeVelocity / WriteOut) — coché par défaut. Routines obligatoires pour tout corps rigide 2D. Génère RBDY2_NewStep(), RBDY2_FreeVelocity(), RBDY2_WriteOut() dans la boucle.

Corps rigides 3D — RBDY3

– RBDY3 (NewStep / FreeVelocity / WriteOut) — déclenche les routines équivalentes en 3D.

Détecteurs de contact 2D

Chaque case active la détection de contact entre une paire de types de contacteurs. Le script génère XXX_SelectProxTactors() et XXX_RecupRloc() / XXX_StockRloc() correspondants.

Détecteur	Description
DKDKx	Disque / Disque (coché par défaut) — granulométrie 2D, milieux granulaires
DKJCx	Disque / Jonc — particules avec ellipses
DKKDX	Disque / Polygone (Corde) — interaction disque-paroi polygonale
PLPLx	Plan / Plan — parois planes entre elles
CLALp	Ligne / Ligne maçonnerie — interfaces entre briques (CLALp)

Détecteur	Description
ALpALp	Ligne / Ligne ALp — variante pour contacteurs polygonaux

Détecteurs de contact 3D

Détecteur	Description
SPSPx	Sphère / Sphère — granulométrie 3D
SPCDx	Sphère / Cylindre
SPPLx	Sphère / Plan
CDCDx	Cylindre / Cylindre
CDPLx	Cylindre / Plan
PRPRx	Polyèdre / Polyèdre

Corps déformables — Routines FEM

- mecaFEMx — Mécanique des solides : assemblage, calcul des forces internes (Fint), forces externes (Fext), matrice de rigidité (K) et résolution (ComputeDof).
- therFEMx — Thermique : flux thermique, bilan d'énergie thermique, résolution ComputeDof.
- hydrFEMx — Hydraulique : pression hydraulique, flux fluide, résolution ComputeDof.

Contacteurs mixtes — Rigide / Déformable

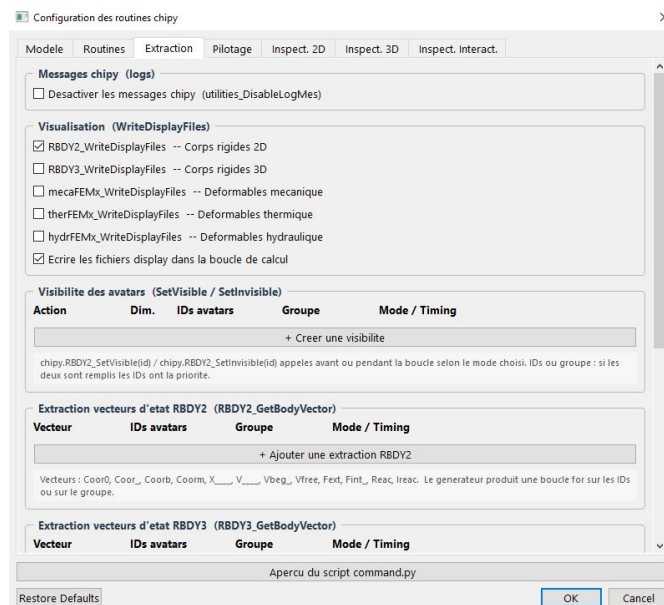
- DKMECAx — Interaction entre disques rigides (2D) et maillages mécaniques (MECAx FEM 2D).
- ALpMECAx — Interaction entre interfaces de maçonnerie (CLALp) et maillages mécaniques FEM 2D. Utile pour les simulations de structures maçonnées avec blocs déformables.
- SPMECAx — Interaction entre sphères rigides (3D) et maillages mécaniques FEM 3D.

Routines spéciales

- PT2Dx — Nœuds ponctuels 2D : interaction point/point pour les éléments câbles (ELASTIC_WIRE) et barres élastiques (ELASTIC_ROD).
- PT3Dx — Équivalent 3D de PT2Dx.
- NODES — Nœuds couplés : routines de couplage de degrés de liberté (COUPLED_DOOF, NORMAL_COUPLED_DOOF).
- UpdateBulkBehav — Lois de comportement volumique : génère chipy.UpdateBulkBehav() pour les modèles avec plasticité, endommagement ou variables d'histoire.

3.Extraction

Cet onglet configure toutes les sorties du calcul : fichiers de visualisation, visibilité des avatars, extraction de vecteurs d'état, forces de contact, énergie et champs FEM.



captures/config_calculs_extraction.JPG

Messages chipy (logs)

– Désactiver les messages chipy (utilities_DisableLogMes) — Génère chipy.utilities_DisableLogMes() immédiatement après chipy.Initialize(). Supprime tous les messages de progression dans la console. Recommandé pour les calculs en production ou de longue durée.

Visualisation (WriteDisplayFiles)

Contrôle l'écriture des fichiers de visualisation vers le répertoire DISPLAY/. Chaque case correspond à une famille d'avatars : – RBDY2_WriteDisplayFiles — Corps rigides 2D (coché par défaut). – RBDY3_WriteDisplayFiles — Corps rigides 3D. – mecaFEMx_WriteDisplayFiles — Maillages déformables mécaniques. – therFEMx_WriteDisplayFiles — Maillages déformables thermiques. – hydrFEMx_WriteDisplayFiles — Maillages déformables hydrauliques. – Écrire les fichiers display dans la boucle — Si coché, les fichiers sont écrits à chaque pas (ou à la fréquence définie). Si décoché, un seul fichier est écrit à la fin du calcul.

Visibilité des avatars (SetVisible / SetInvisible)

Permet d'afficher ou de masquer des avatars individuellement ou par groupe à des moments précis de la simulation. Chaque ligne de la liste correspond à une règle de visibilité. Pour créer une règle, cliquer sur « + Créer une visibilité ». Chaque ligne contient :

Action	SetVisible ou SetInvisible — rend l'avatar visible ou invisible dans chipy.
Dim.	2D (RBDY2) ou 3D (RBDY3) — détermine le préfixe de la fonction générée.
IDs avatars	Liste d'identifiants séparés par des virgules (ex. : 1, 3, 5). Prioritaire sur le groupe si les deux sont renseignés.
Groupe	Nom d'un groupe d'avatars défini dans le projet. Résolu en liste d'IDs à la génération.
Mode / Timing	Détermine quand l'appel est généré (voir tableau des modes ci-dessous).

Modes de temporisation (Timing)

Modes de temporisation (Timing)

Mode	Condition générée	Usage typique
Tous les pas	Aucune condition (appel direct)	Extraction systématique, bilan énergétique continu

Tous les N pas |if k % N == 0: |Réduire la fréquence d'écriture pour alléger les sorties|

Au pas k =	if k == K:	Événement ponctuel : changer la visibilité à un pas précis
Après boucle	Hors de la boucle (après for k ...)	État final uniquement, post-traitement en fin de calcul

Extraction de vecteurs d'état RBDY2 (RBDY2_GetBodyVector)

Génère des appels à `chipy.RBDY2_GetBodyVector(vecteur, id)` dans la boucle de calcul. Pour chaque ligne ajoutée via « + Ajouter une extraction RBDY2 », configurer :

Vecteur	Nom du vecteur d'état à extraire (voir liste complète ci-dessous).
IDs avatars	Liste d'IDs séparés par des virgules. Si renseigné, génère une boucle for sur ces IDs.
Groupe	Groupe d'avatars à parcourir. Priorité inférieure aux IDs si les deux sont renseignés.
Mode / Timing	Un des quatre modes de temporisation décrits ci-dessus.

Vecteurs disponibles :

Nom	Description
Coor0	Position de référence (configuration initiale)
Coor_	Position courante
Coorb	Position au pas précédent
Coorm	Position moyenne entre deux pas
X____	Déplacement total accumulé
V____	Vitesse courante (linéaire et angulaire)
Vbeg_	Vitesse en début de pas
Vfree	Vitesse libre (avant résolution du contact)
Fext_	Forces et moments extérieurs appliqués
Fint_	Forces et moments internes
Reac_	Résultante des réactions de contact
Ireac	Impulsions de réaction de contact

Forces et réactions de contact

– Forces nodales (inter_handler_Rnod) — Extrait les forces nodales aux points de contact et les écrit dans POSTPRO/. – Vitesses locales (inter_handler_Vloc) — Extrait les vitesses relatives dans le repère local de contact. – Forces locales (inter_handler_Rloc) — Extrait les impulsions/forces dans le repère local de contact.

Énergie

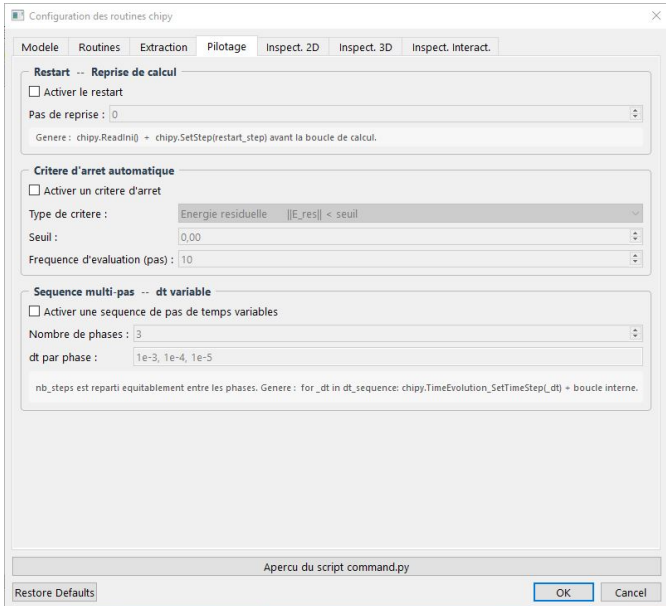
– Bilan énergétique global (ComputeEnergy + WriteEnergy) — Calcule et écrit l'énergie cinétique, potentielle et dissipée par frottement. – Énergie cinétique RBDY2 (RBDY2_KineticEnergy) — Écrit l'énergie cinétique de chaque corps rigide 2D séparément.

Champs FEM (contraintes, déformations, température...)

– Champs par élément (mecaFEMx_WriteBodies) — Écrit les champs par élément : contraintes et déformations (MECAx), température (THERx), pression (HYDRx). – Variables internes (mecaFEMx_WriteInternalVariables) — Écrit les variables internes aux points de Gauss : plasticité, endommagement, variables d'histoire.

4.Pilotage

Cet onglet contrôle des fonctions avancées du déroulement du calcul : reprise depuis un état sauvegardé, arrêt anticipé sur critère de convergence et séquençage multi-pas.



captures/config_pilotage.JPG

Restart — Reprise de calcul Permet de reprendre un calcul à partir d'un état précédemment sauvegardé dans les fichiers .dat.last.

Activer le restart	Génère chipy.ReadLine() puis chipy.SetStep(restart_step) avant la boucle de calcul.
Pas de reprise	Numéro du pas de temps à partir duquel reprendre (entier, de 0 à 9 999 999).

Critère d'arrêt automatique

Interrompt la boucle de calcul avant la fin du nombre de pas prévu si un critère de convergence est satisfait.

Activer un critère d'arrêt	Active le mécanisme d'arrêt anticipé. Génère une condition break dans la boucle for k.		
Type de critère	Trois types disponibles : résidu d'énergie (E_res < seuil), déplacement maximum (max	u	< seuil), résidu de force (F_res < seuil).
Seuil	Valeur numérique du critère d'arrêt (de 10 ⁻¹ à 1,0). Valeur par défaut : 10 ⁻³ .		
Fréquence d'évaluation	Évaluer le critère tous les N pas. Évite un calcul du critère à chaque itération, ce qui peut être coûteux.		

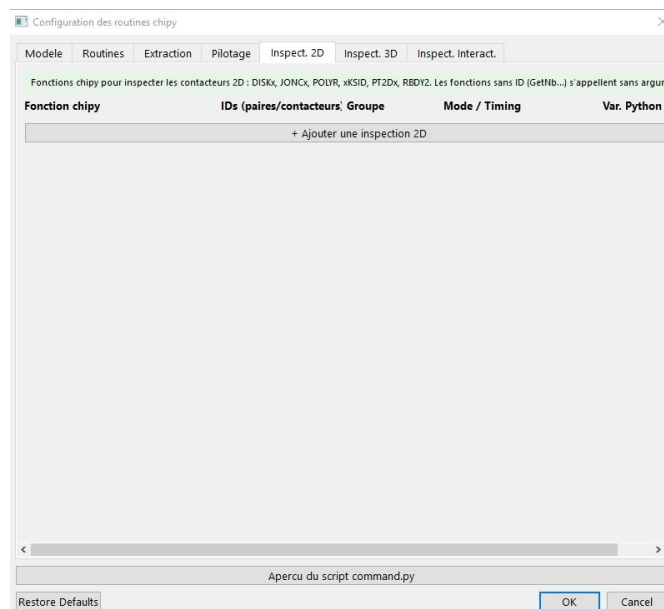
Séquence multi-pas — dt variable

Permet de définir plusieurs phases de calcul avec des pas de temps différents. Utile pour les calculs avec chargements progressifs ou pour affiner le pas de temps à l'approche d'un événement critique.

Activer une séquence multi-pas	Génère une boucle externe sur les phases : <code>for _dt in dt_sequence: chipy.TimeEvolution_SetTimeStep(_dt) +</code> boucle interne.
Nombre de phases	Entre 2 et 20 phases. Le nombre total de pas (nb_steps) est réparti équitablement entre les phases.
dt par phase	Liste de valeurs de dt séparées par des virgules, une par phase (ex. : 1e-3, 1e-4, 1e-5).

5. Inspection 2D

Cet onglet permet d'ajouter des appels d'inspection sur les contacteurs 2D du modèle. Chaque ligne correspond à un appel `chipy.XXXX_GetYYYY()` inséré dans la boucle de calcul ou après celle-ci.



captures/config_inspect2D.JPG

Cliquer sur « + Ajouter une inspection 2D » pour créer une ligne. Chaque ligne contient cinq colonnes :

Fonction chipy	Sélection dans la liste déroulante des fonctions disponibles pour les contacteurs 2D. La description s'affiche dans l'infobulle.
IDs (contacteurs)	Liste d'identifiants chipy séparés par des virgules. Laissé vide pour les fonctions de type GetNb... qui ne prennent pas d'argument.
Groupe	Nom d'un groupe d'avatars. Résolu en IDs à la génération si les IDs sont vides.
Mode / Timing	Un des quatre modes de temporisation (Tous les pas, Tous les N pas, Au pas k =, Après boucle).
Var. Python	Nom de la variable Python dans laquelle stocker le résultat (ex. : vel_disk). Laissé vide si le résultat n'est pas réutilisé.

Fonctions disponibles — Contacteurs 2D

Les fonctions sont regroupées par type de contacteur :

DISKx – Disques rigides 2D – DISKx_GetNbDISKx — Nombre total de contacteurs DISKx (pas d'ID requis). – DISKx_GetBodyId(i) — ID du corps RBDY2 auquel appartient le contacteur i. – DISKx_GetPtrDISKx2BDYTY(i) — Index local du contacteur dans son corps RBDY2. – DISKx_GetPtrTactBehav(i) — Loi de comportement de contact associée au contacteur i. – DISKx_GetRadius(i) — Rayon du disque i. – DISKx_GetCoor(i) — Coordonnées du centre du disque i. – DISKx_GetVelocity(i) — Vitesse du centre du disque i.

JONCx – Joncs / Ellipses 2D – JONCx_GetNbJONCx — Nombre total de contacteurs JONCx. – JONCx_GetBodyId(i), JONCx_GetPtrJONCx2BDYTY(i), JONCx_GetPtrTactBehav(i) — Identification. – JONCx_GetAxes(i) — Demi-axes (a, b) du jonc i. – JONCx_GetCoor(i) — Coordonnées du centre du jonc i.

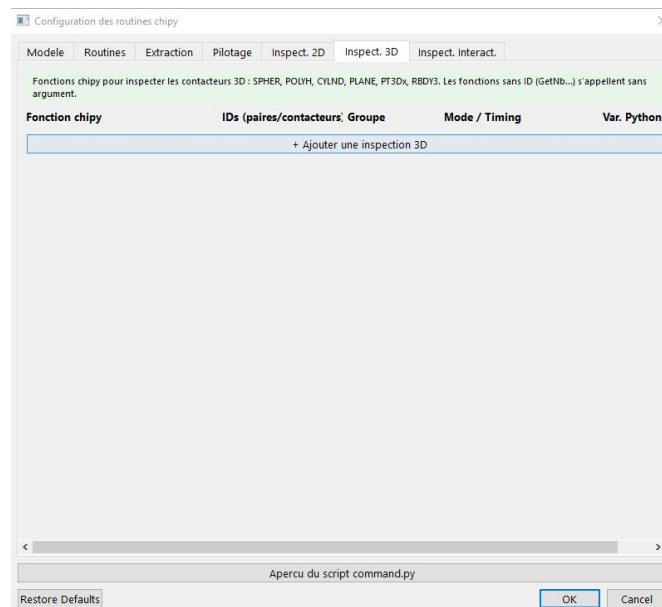
POLYR – Polygones rigides 2D – POLYR_GetNbPOLYR — Nombre total de contacteurs POLYR. – POLYR_GetBodyId(i), POLYR_GetPtrPOLYR2BDYTY(i), POLYR_GetPtrTactBehav(i) — Identification. – POLYR_GetNbVerti(i) — Nombre de sommets du polygone i. – POLYR_GetVerti(i) — Coordonnées de tous les sommets du polygone i. – POLYR_GetCoor(i) — Coordonnées du centre de référence du polygone i.

xKSID – Clusters de disques discrets 2D – xKSID_GetNbXKSID, xKSID_GetBodyId(i), xKSID_GetPtrXKSID2BDYTY(i), xKSID_GetRadius(i).

RBDY2 – Corps rigides 2D (synthèse) – RBDY2_GetNbRBDY2 — Nombre total de corps rigides 2D. – RBDY2_KineticEnergy — Énergie cinétique totale de tous les corps RBDY2.

PT2Dx – Nœuds contacteurs FEM 2D – PT2Dx_GetNbPT2Dx — Nombre de nœuds contacteurs FEM 2D. – PT2Dx_GetBodyId(i) — ID du corps FEM parent. – PT2Dx_GetCoor(i) — Coordonnées du nœud contacteur i.

6. Inspection 3D



captures/config_inspect3D.JPG

Fonctionnement identique à l'onglet Inspection 2D, mais pour les contacteurs 3D. Les familles disponibles sont :

SPHER – Sphères rigides 3D – SPHER_GetNbSPHER, SPHER_GetBodyId(i), SPHER_GetPtrSPHER2BDYTY(i), SPHER_GetPtrTactBehav(i). – SPHER_GetRadius(i) — Rayon de la sphère i. – SPHER_GetCoor(i), SPHER_GetVelocity(i) — Position et vitesse.

POLYH – Polyèdres rigides 3D – POLYH_GetNbPOLYH, POLYH_GetBodyId(i), POLYH_GetPtrPOLYH2BDYTY(i), POLYH_GetPtrTactBehav(i). – POLYH_GetNbFaces(i), POLYH_GetNbVerti(i), POLYH_GetVerti(i) — Géométrie. – POLYH_GetCoor(i) — Coordonnées du centre de référence.

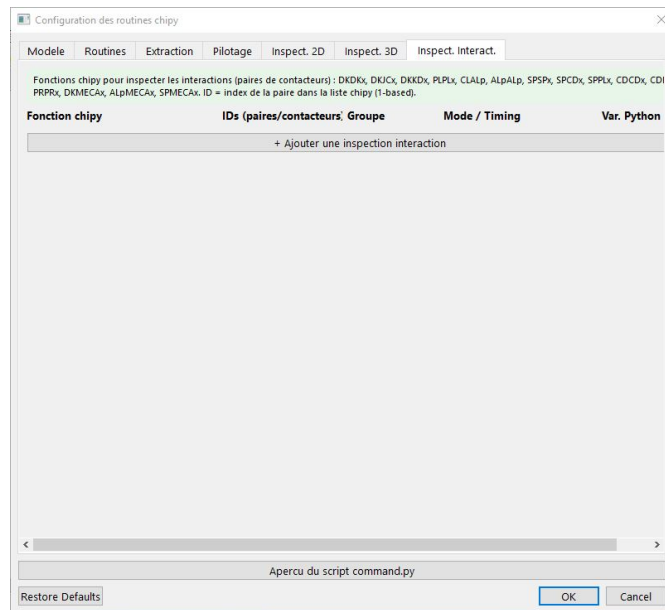
CYLND – Cylindres rigides 3D – CYLND_GetNbCYLND, CYLND_GetBodyId(i), CYLND_GetPtrCYLND2BDYTY(i), CYLND_GetPtrTactBehav(i). – CYLND_GetRadius(i), CYLND_GetLength(i), CYLND_GetCoor(i).

PLANE – Plans rigides 3D – PLANE_GetNbPLANE, PLANE_GetBodyId(i), PLANE_GetNormal(i), PLANE_GetCoor(i).

RBDY3 et PT3Dx – RBDY3_GetNbRBDY3 — Nombre total de corps rigides 3D. – PT3Dx_GetNbPT3Dx, PT3Dx_GetBodyId(i), PT3Dx_GetCoor(i) — Nœuds contacteurs FEM 3D.

7. Inspection des interactions

Cet onglet inspecte les paires de contacteurs actives (interactions en cours de simulation). L'ID utilisé est l'index de la paire dans la liste chipy (numérotation 1-based).



captures/config_interac.JPG

Les fonctions sont regroupées par type d'interaction :

`DKDKx – Disque / Disque` – DKDKx_GetNbDKDKx — Nombre de paires actives. – DKDKx_GetBodyIds(i) — IDs RBDY2 des deux corps de la paire i. – DKDKx_GetTactors(i) — IDs des deux contacteurs DISKx de la paire i. – DKDKx_GetGapTT(i) — Jeu (gap) de la paire i. – DKDKx_GetStatusTT(i) — Statut de contact : 0 = pas de contact, 1 = contact actif. – DKDKx_GetRlocTT(i) — Réaction locale (Rn, Rt) dans le repère local de contact. – DKDKx_GetVlocTT(i) — Vitesse locale relative (Vn, Vt).

`DKJCx – Disque / Jonc` – DKJCx_GetNbDKJCx, DKJCx_GetBodyIds(i), DKJCx_GetTactors(i), DKJCx_GetGapTT(i), DKJCx_GetStatusTT(i), DKJCx_GetRlocTT(i).

`DKKDx – Disque / Corde (Polygone)` – DKKDx_GetNbDKKDx, DKKDx_GetBodyIds(i), DKKDx_GetGapTT(i), DKKDx_GetRlocTT(i).

`PLPLx – Polygone / Polygone` – PLPLx_GetNbPLPLx, PLPLx_GetBodyIds(i), PLPLx_GetTactors(i), PLPLx_GetGapTT(i), PLPLx_GetStatusTT(i), PLPLx_GetRlocTT(i), PLPLx_GetVlocTT(i).

`CLALp – Brique / Brique (maçonnerie)` – CLALp_GetNbCLALp, CLALp_GetBodyIds(i), CLALp_GetGapTT(i), CLALp_GetStatusTT(i), CLALp_GetRlocTT(i). ● Utiliser CLALp avec le mode « Tous les pas » pour suivre l'évolution des forces de contact dans les joints de maçonnerie au cours de la simulation.

`ALpALp – ALp / ALp` – ALpALp_GetNbALpALp, ALpALp_GetBodyIds(i), ALpALp_GetGapTT(i), ALpALp_GetRlocTT(i).

`SPSPx — Sphère / Sphère (3D)` — SPSPx_GetNbSPSPx, SPSPx_GetBodyIds(i), SPSPx_GetTactors(i), SPSPx_GetGapTT(i), SPSPx_GetStatusTT(i), SPSPx_GetRlocTT(i) (Rn, Rt, Rs), SPSPx_GetVlocTT(i) (Vn, Vt, Vs).

`SPCDx — Sphère / Cylindre (3D)` — SPCDx_GetNbSPCDx, SPCDx_GetBodyIds(i), SPCDx_GetTactors(i), SPCDx_GetGapTT(i), SPCDx_GetRlocTT(i).

`SPPLx — Sphère / Plan (3D)` — SPPLx_GetNbSPPLx, SPPLx_GetBodyIds(i), SPPLx_GetGapTT(i), SPPLx_GetRlocTT(i).

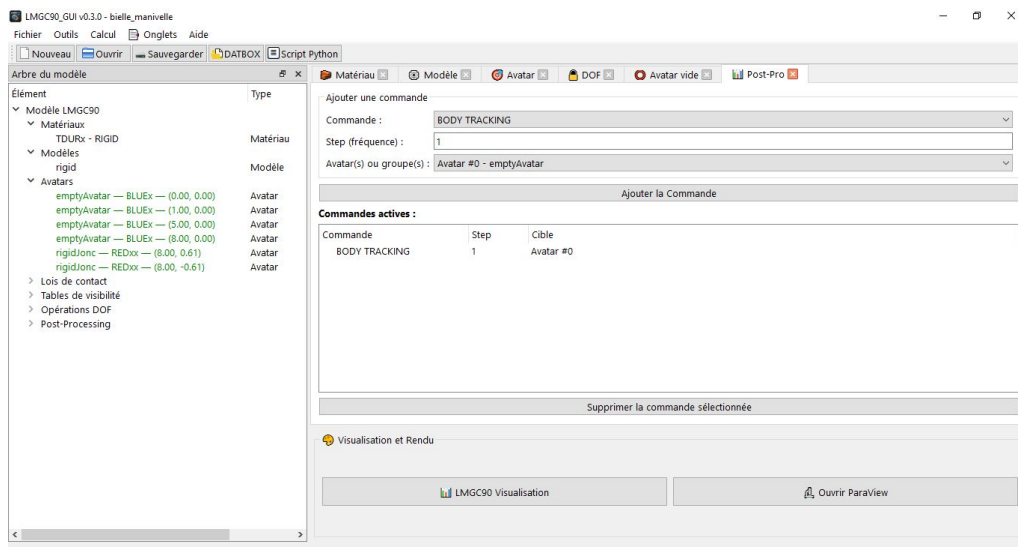
`CDCDx — Cylindre / Cylindre (3D)` — CDCDx_GetNbCDCDx, CDCDx_GetBodyIds(i), CDCDx_GetGapTT(i), CDCDx_GetRlocTT(i). CDPLx — Cylindre / Plan (3D) — CDPLx_GetNbCDPLx, CDPLx_GetBodyIds(i), CDPLx_GetGapTT(i), CDPLx_GetRlocTT(i).

`PRPRx — Polyèdre / Polyèdre (3D)` — PRPRx_GetNbPRPRx, PRPRx_GetBodyIds(i), PRPRx_GetTactors(i), PRPRx_GetGapTT(i), PRPRx_GetStatusTT(i), PRPRx_GetRlocTT(i) (Rn, Rt, Rs).

`Contacteurs mixtes Rigide / Déformable` — DKMECAx_GetNbDKMECAx, DKMECAx_GetBodyIds(i), DKMECAx_GetGapTT(i), DKMECAx_GetRlocTT(i) — Disque / MECAx FEM 2D. — ALpMECAx_GetNbALpMECAx, ALpMECAx_GetBodyIds(i), ALpMECAx_GetRlocTT(i) — ALp / MECAx FEM 2D. — SPMECAx_GetNbSPMECAx, SPMECAx_GetBodyIds(i), SPMECAx_GetRlocTT(i) — Sphère / MECAx FEM 3D.

Post-traitement (PostPro)

L'onglet **Post-Pro** définit les **commandes de post-traitement** qui seront incluses dans la DATBOX et exécutées par LMGC90 pendant le calcul. Chaque commande demande à LMGC90 d'écrire des fichiers de sortie à une fréquence donnée, permettant ensuite l'analyse et la visualisation des résultats.



captures/postpro.JPG

Principe de fonctionnement

Une **commande PostPro** (**PostProCommand**) associe un **nom de commande** pylmgc90 à une **fréquence d'écriture** (**step**) et, optionnellement, à une **cible** (avatar unique ou groupe). L'appel pylmgc90 correspondant est :

```
# Sans cible (commande globale) :
posts.addCommand(pre.postpro_command(name='SOLVER INFORMATIONS', step=1))

# Avec cible avatar :
posts.addCommand(pre.postpro_command(
name='BODY TRACKING',
step=10,
rigid_set=[bodies[3]]
))

# Avec cible groupe :
posts.addCommand(pre.postpro_command(
name='TORQUE EVOLUTION',
step=5,
rigid_set=[bodies[i] for i in group_granulo_box2d]
))
```

La structure **PostProCommand** dans le projet stocke trois champs : **name**, **step** et optionnellement **target_type** / **target_value**.

Interface de l'onglet

L'onglet est organisé en deux zones :

- **Liste des commandes** (en haut) : tableau de toutes les commandes enregistrées. Chaque ligne affiche le nom de la commande, la fréquence (**step=N**) et la cible (**Global**, **Avatar #N** ou **Groupe: nom**). Double-clic pour éditer.
- **Formulaire de création / modification** (en bas) : trois champs simples.

Champs du formulaire

Champ	Description	Valeur par défaut
Commande	Nom de la commande de post-traitement pylmgc90. Sélectionner dans la liste déroulante.	SOLVER INFORMATION
Fréquence (step)	Écriture tous les step pas de calcul. step=1 = écriture à chaque pas, step=100 = tous les 100 pas.	1
Cible	Global (toute la simulation), Avatar (index numérique) ou Groupe (nom de groupe).	Global

Commandes disponibles

Qualité numérique — pas de cible requise

Ces commandes sont **globales** et ne nécessitent pas de **rigid_set**. Elles s'appliquent à l'ensemble de la simulation.

Commande	Description	Fichiers de sortie
SOLVER INFORMATION	Informations du solveur de contact à chaque pas : nombre d'itérations, résidu, temps de calcul. Indispensable pour vérifier la convergence du schéma de contact.	OUTBOX/solver_information.dat
VIOLATION EVOLUTION	Évolution de la violation (interpénétration résiduelle) moyenne et maximale entre corps. Mesure l' erreur numérique de non-pénétration.	OUTBOX/violation_evolution.dat
KINETIC ENERGY	Énergie cinétique totale du système à chaque pas. Utile pour suivre la dissipation d'énergie et détecter les instabilités.	OUTBOX/kinetic_energy.dat
CONTACT ENERGY	Énergie dissipée par les contacts (frottement + restitution).	OUTBOX/contact_energy.dat
STRAIN ENERGY	Énergie de déformation stockée dans le système. Pour les corps déformables EF.	OUTBOX/strain_energy.dat

etc,

Suivi de corps — cible requise (**rigid_set**)

Ces commandes écrivent des informations sur un **corps spécifique** ou un **groupe d'avatars**. Une cible (`target_type` + `target_value`) est obligatoire.

Commande	Description	Données extraites	Fichiers de sortie
BODY TRACKING	Suivi complet de la position, vitesse et accélération d'un corps au cours du temps. La commande la plus utilisée pour analyser la trajectoire d'un avatar.	Position (x, y, z), vitesse (vx, vy, vz), accélération, angle et vitesse angulaire	OUTBOX/body_tracking.dat
TORQUE EVOLUTION	Évolution du moment (couple) appliqué sur le corps ou le groupe.	Composantes du couple selon X, Y, Z	OUTBOX/torque_evolution.dat
MOMENTUM EVOLUTION	Évolution de la quantité de mouvement du corps ou du groupe.	Quantité de mouvement (px, py, pz)	OUTBOX/momentum_evolution.dat

Commandes additionnelles

Commande	Description
WORK EVOLUTION	Travail des forces extérieures appliquées au cours du temps.
DISSIPATED ENERGY	Énergie dissipée totale (contact + amortissement).

Cible des commandes

Global `_`(pas de `rigid_set`)

La commande s'applique à l'ensemble du système. Utiliser pour `SOLVER INFORMATION`, `VIOLATION EVOLUTION`, `KINETIC ENERGY`.

```
pre.postpro_command(name='SOLVER INFORMATION', step=1)
```

Avatar (index unique)

La commande surveille un corps spécifique identifié par son index dans la liste des avatars (0-based).

```
pre.postpro_command(name='BODY TRACKING', step=10, rigid_set=[bodies[3]])
```

Dans l'interface : sélectionner **Avatar** comme type de cible, puis saisir l'index (ex : 3).

Groupe (ensemble d'avatars)

La commande surveille tous les corps d'un groupe nommé (boucle, granulométrie, maçonnerie...).

```
pre.postpro_command(
name='TORQUE EVOLUTION',
step=5,
rigid_set=[bodies[i] for i in group_granulo_box2d]
)
```

Dans l'interface : sélectionner **Groupe** comme type de cible, puis choisir le groupe dans la liste déroulante.

Tous les groupes définis dans le projet (boucles, granulométrie, maçonnerie) apparaissent automatiquement dans la liste.

Fréquence d'écriture (`step`)

Le paramètre `step` contrôle la fréquence d'écriture des fichiers de résultats.

Valeur <code>step</code>	Comportement	Usage
1	Écriture à chaque pas de calcul	Analyse fine, débogage, petits modèles
10	Écriture tous les 10 pas	Bon compromis précision / taille fichier
100	Écriture tous les 100 pas	Grandes simulations longues durées
<code>step_total / 1000</code>	~1000 points dans le fichier	Règle empirique pour des courbes lisses

Impact sur les performances : un `step=1` avec `BODY TRACKING` sur un grand groupe peut générer des fichiers de plusieurs gigaoctets et ralentir le calcul. Adapter la fréquence à la durée et à la précision requise.

Gestion des commandes

Créer une commande

Remplir le formulaire et cliquer sur ■ **Ajouter la Commande**. La commande est créée via `add_postpro_command()` qui :

1. Résout la cible (`rigid_set`) en listes d'objets `pymgc90`.
2. Crée l'objet `pre.postpro_command(name, step, rigid_set)`.
3. L'ajoute au conteneur `_postpro_container` via `addCommand()`.
4. Sauvegarde dans `state.postpro_commands`.
5. Émet le signal `command_added` → `_refresh_all()`.

Modifier une commande

Double-cliquer dans la liste pour charger les valeurs dans le formulaire. Modifier et cliquer sur ■ **Mettre à jour**. `update_postpro_command()` reconstruit l'ensemble du conteneur `postpro` (même comportement que pour les visibilités — limitation `pymgc90`).

Supprimer une commande

Sélectionner et cliquer sur ■■ **Supprimer**. La commande est retirée de `state.postpro_commands` via `remove_postpro_command()`. Le signal `command_deleted` est émis.

Visualisation dans l'arbre du modèle

Les commandes `postpro` sont affichées dans l'arbre du modèle (panneau gauche) sous le nœud **Post-Processing**, avec pour chaque commande :

- Son nom (ex. `BODY TRACKING`)
- Sa fréquence (`step=10`)
- Sa cible (`Global`, `Avatar #3` ou `Groupe: granulo_box2d`)

Double-cliquer sur une commande dans l'arbre ouvre directement l'onglet PostPro en mode édition (`load_for_edit(postpro)`).

Script Python généré

```
# Post-traitement

# Commande globale
post_cmd_0 = pre.postpro_command(
name='SOLVER INFORMATIONS',
step=1
)
posts.addCommand(post_cmd_0)

# Commande avec cible avatar
post_cmd_1 = pre.postpro_command(
name='BODY TRACKING',
step=10,
rigid_set=[bodies[3]]
)
posts.addCommand(post_cmd_1)

# Commande avec cible groupe
post_cmd_2 = pre.postpro_command(
name='TORQUE EVOLUTION',
step=5,
rigid_set=[bodies[i] for i in group_granulo_box2d]
)
posts.addCommand(post_cmd_2)
```

Le conteneur `posts` est passé à `pre.writeDatbox(post=posts, ...)` lors de la génération de la DATBOX.

Lecture des résultats

Les fichiers de sortie sont écrits dans le répertoire `OUTBOX/` du projet LMGC90 pendant le calcul. Chaque fichier est un fichier texte (colonnes séparées par des espaces) dont le format dépend de la commande :

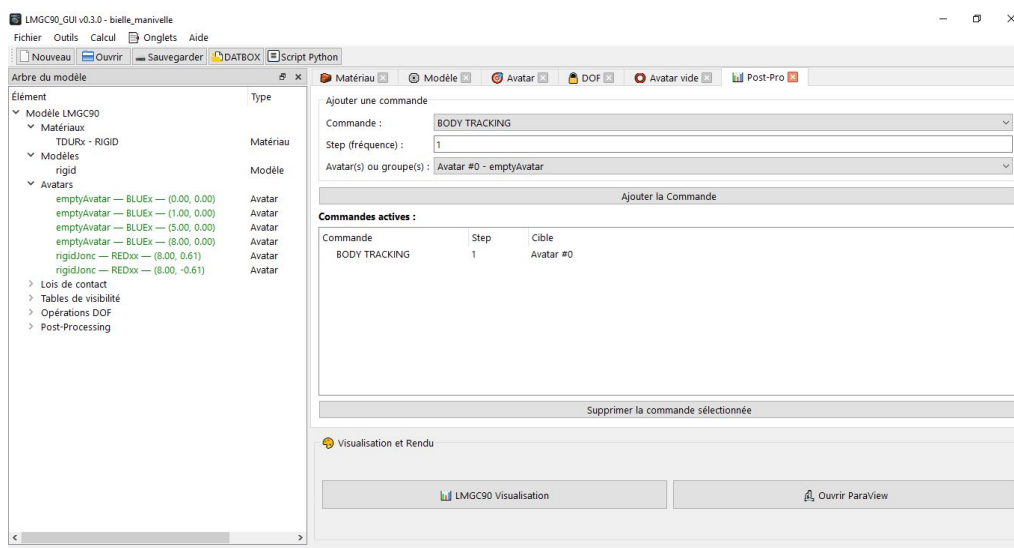
Commande	Format typique	Colonnes
<code>BODY TRACKING</code>	Texte, N colonnes	<code>t, x, y, z, vx, vy, vz, theta, omega</code>
<code>SOLVER INFORMATIONS</code>	Texte	<code>t, iter, residual, cpu_time</code>
<code>VIOLATION EVOLUTION</code>	Texte	<code>t, mean_violation, max_violation</code>
<code>KINETIC ENERGY</code>	Texte	<code>t, Ec</code>
<code>TORQUE EVOLUTION</code>	Texte	<code>t, Mx, My, Mz</code>

Ces fichiers peuvent être lus et tracés directement avec Python (numpy, matplotlib) ou avec l'outil de visualisation intégré de LMGC90.

Exemple d'utilisation — bielle-manivelle

Pour une simulation de bielle-manivelle, configurer les commandes suivantes :

#	Commande	Step	Cible	Objectif
0	SOLVER INFORMATIONS	1	Global	Vérifier la convergence
1	VIOLATION EVOLUTION	1	Global	Contrôler l'interpénétration
2	BODY TRACKING	10	Avatar #0 (manivelle)	Trajectoire angulaire
3	BODY TRACKING	10	Avatar #2 (coulisseau)	Déplacement linéaire
4	KINETIC ENERGY	1	Global	Bilan énergétique



Exemple onglet PostPro

Remarques importantes

Reconstruction lors de la modification : comme pour les tables de visibilité, toute modification d'une commande entraîne la reconstruction complète du conteneur `_postpro_container`. Ceci est transparent pour l'utilisateur.

Les fichiers de sortie ne sont écrits que pendant le calcul. La génération de la DATBOX et du script Python n'écrit pas de résultats — le calcul doit être lancé depuis l'onglet Calcul (F5) pour que les fichiers `OUTBOX/` soient créés.

Step et durée de simulation : s'assurer que `step` est inférieur au nombre total de pas de calcul. Une commande avec `step=100` dans une simulation de 50 pas ne produira aucune sortie.

Cohérence des indices d'avatars : les indices dans `target_value` référencent la position dans `state.avatars` au moment de la création. Si des avatars sont supprimés ou réordonnés après la création d'une commande, les indices peuvent devenir incorrects. Préférer les groupes nommés pour les commandes sur plusieurs corps.

Post-traitement

Définition des sorties pour LMGC90 qui vont à extraire et analyser vos calculs.

Commandes disponibles

1. Vérifier la qualité numérique

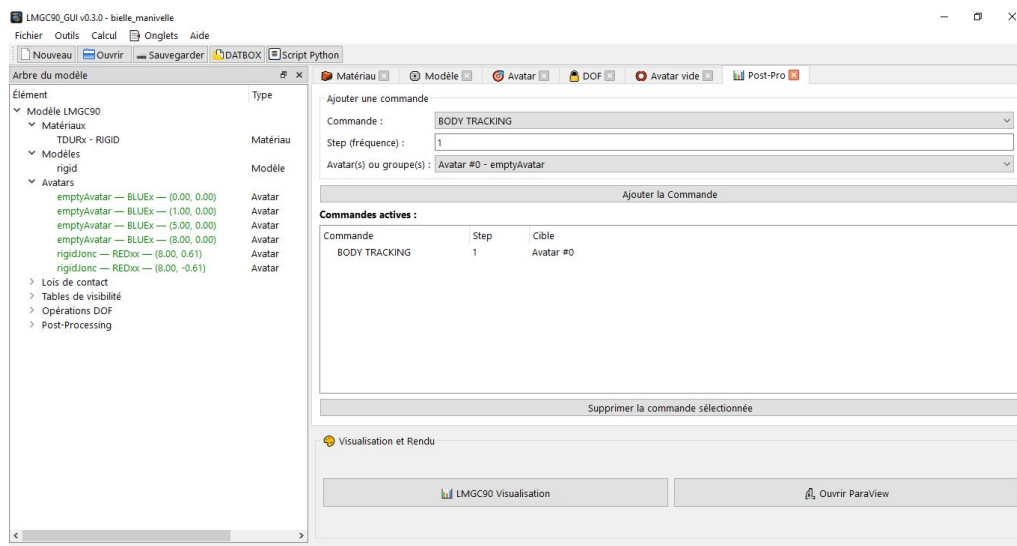
- SOLVER INFORMATION : pour s'assurer la convergence
- VIOLATION EVOLUTION : mesure "l'erreur" d'interpénétration moyenne
- TORQUE EVOLUTION (sur avatar/groupe) :
- BODY TRACKING (suivi de corps)
- KINETIC ENERGY, etc.

Fonctionnalités

- step : étape
- rigid_set : avatar ou group d'avatars

Exemple :

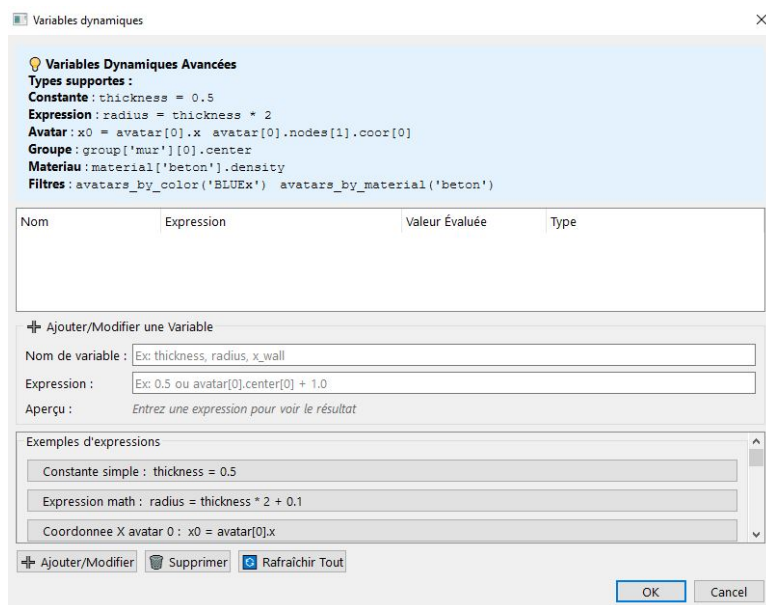
Pour rajouter une commande du postpro, il faut se rendre dans l'onglet "PostPro", puis de choisir la commande voulue dans mon cas "BODY TRACKING", puis de renseigner l'étape, et de cliquer sur le bouton "**Ajouter la Commande**"



captures/postpro.JPG

Variables dynamiques

Les **variables dynamiques** de LMGC90_GUI permettent d'écrire des expressions Python directement dans les champs de saisie de l'interface, en utilisant les données du projet (avatars, matériaux, modèles, groupes) comme des variables. Elles remplacent les valeurs numériques figées par des expressions calculées à la volée, et se propagent automatiquement dans tous les onglets.



captures/variables.JPG

Accès au gestionnaire de variables

Méthode	Action
Menu Outils → Variables dynamiques	Ouvre le gestionnaire
Raccourci	Ctrl+V

Le gestionnaire de variables dynamiques

Le gestionnaire (`DynamicVarsDialog`) utilise `SafeEvaluator` avec `build_eval_context()` pour évaluer les expressions de façon sécurisée. Il affiche un tableau à 4 colonnes : **Nom**, **Expression**, **Valeur évaluée**, **Type**. Chaque variable est une expression Python stockée sous forme de chaîne et évaluée au moment de son utilisation.

Ajouter une variable

1. Saisir un **Nom** — identifiant Python valide (`thickness`, `r_min`, `x_wall`).
2. Saisir une **Expression** — voir les possibilités ci-dessous.
3. L'**Aperçu** se met à jour en temps réel : résultat en vert si valide, erreur en rouge.

4. Cliquer sur **Ajouter / Modifier**.

Modifier une variable

Cliquer sur la ligne dans le tableau → les champs Nom et Expression sont remplis → modifier → **Ajouter / Modifier**.

Supprimer une variable

Sélectionner la ligne et cliquer sur **Supprimer**.

Rafraîchir

Cliquer sur **Rafraîchir** pour réévaluer toutes les variables avec l'état courant du projet (utile après avoir ajouté des avatars).

Persistence : les variables sont sauvegardées dans le fichier `.lmgc90` du projet (`state.dynamic_vars`). Elles sont rechargées automatiquement à l'ouverture.

Types d'expressions supportées

Constantes numériques

```
thickness = 0.5
radius = 0.075
n_cols = 15
pi_val = 3.14159
```

Expressions mathématiques

Toutes les expressions Python numériques sont supportées :

```
diameter = 2 * radius
area = pi * radius**2
step = lx + joint
half = thickness / 2
diag = sqrt(lx**2 + ly**2)
clamp = max(0.0, min(1.0, ratio))
```

Fonctions mathématiques disponibles :

Symbole	Description
<code>pi</code>	$\pi \approx 3.14159\dots$
<code>e</code>	$e \approx 2.71828\dots$
<code>sqrt(x)</code>	Racine carrée
<code>abs(x)</code>	Valeur absolue
<code>min(a, b)</code>	Minimum
<code>max(a, b)</code>	Maximum
<code>round(x, n)</code>	Arrondi à n décimales
<code>sum(liste)</code>	Somme d'une liste
<code>len(objet)</code>	Longueur
<code>math.sin(x)</code>	Sinus (x en radians)

Symbole	Description
<code>math.cos(x)</code>	Cosinus
<code>math.tan(x)</code>	Tangente
<code>math.log(x)</code>	Logarithme naturel
<code>math.log10(x)</code>	Logarithme décimal
<code>math.exp(x)</code>	Exponentielle
<code>math.floor(x)</code>	Partie entière inférieure
<code>math.ceil(x)</code>	Partie entière supérieure
<code>math.radians(deg)</code>	Degrés → radians
<code>math.degrees(rad)</code>	Radians → degrés
<code>np.array(liste)</code>	Tableau numpy
<code>np.linspace(a, b, n)</code>	Tableau régulièrement espacé

etc,

Variables référençant d'autres variables

Les variables sont évaluées dans l'ordre de définition — une variable peut référencer les précédentes :

```
thickness = 0.05
radius = thickness * 3
step = radius + thickness
grid_size = step * 10
```

Accès aux données du projet

Avatars — `avatar[i]`

Accès à un avatar par son index (0-based). Toutes les propriétés sont en lecture seule.

Expression	Description	Type retourné
<code>avatar[i].center</code>	Centre complet <code>[x, y]</code> ou <code>[x, y, z]</code>	<code>list</code>
<code>avatar[i].x</code>	Coordonnée X du centre	<code>float</code>
<code>avatar[i].y</code>	Coordonnée Y du centre	<code>float</code>
<code>avatar[i].z</code>	Coordonnée Z du centre (3D)	<code>float</code> ou <code>None</code>
<code>avatar[i].radius</code>	Rayon de l'avatar	<code>float</code> ou <code>None</code>
<code>avatar[i].color</code>	Couleur LMGC90 ('BLUEx')	<code>str</code>
<code>avatar[i].material_name</code>	Nom du matériau	<code>str</code>
<code>avatar[i].model_name</code>	Nom du modèle	<code>str</code>
<code>avatar[i].avatar_type</code>	Type pylmgc90 ('rigidDisk')	<code>str</code>
<code>avatar[i].origin</code>	Origine : 'manual', 'loop', 'granulo'	<code>str</code>
<code>avatar[i].generation_type</code>	'regular', 'full', 'bevel' ou <code>None</code>	<code>str</code> ou <code>None</code>
<code>avatar[i].is_hollow</code>	Disque creux	<code>bool</code>
<code>avatar[i].nb_vertices</code>	Nombre de sommets (polygone)	<code>int</code> ou <code>None</code>
<code>avatar[i].vertices</code>	Liste des sommets	<code>list</code> ou <code>None</code>

Expression	Description	Type retourné
<code>avatar[i].axis</code>	Axes du jonc {axe1, axe2, axe3}	dict ou None
<code>avatar[i].contactors</code>	Liste des contacteurs	list
<code>avatar[i].wall_params</code>	Paramètres bruts de mur	dict
<code>avatar[i].brick_lx</code>	Longueur brique maçonnerie (<code>wall_params['l']</code>)	float ou None
<code>avatar[i].brick_ly</code>	Hauteur/profondeur brique (<code>wall_params['h']</code>)	float ou None
<code>avatar[i].brick_lz</code>	Hauteur 3D brique (<code>wall_params['lz']</code>)	float ou None
<code>avatar[i].mesh_params</code>	Paramètres de maillage EF	dict ou None
<code>avatar[i].index</code>	Indice de l'avatar dans la liste	int

Nœuds pylmgc90 — `avatar[i].nodes`

La propriété `nodes` simule l'accès aux nœuds pylmgc90. La convention pylmgc90 numérote les nœuds à partir de 1 — `nodes[1]` est le nœud principal, identique à `nodes[0]` en Python.

Expression	Description
<code>avatar[i].nodes[1].coor</code>	Coordonnées du nœud principal [x, y] ou [x, y, z]
<code>avatar[i].nodes[1].coor[0]</code>	Coordonnée X du nœud
<code>avatar[i].nodes[1].coor[1]</code>	Coordonnée Y du nœud
<code>avatar[i].nodes[1].coor[2]</code>	Coordonnée Z du nœud (3D)
<code>avatar[i].nodes[0].coor</code>	Identique à <code>nodes[1]</code> (convention Python)
<code>avatar[i].nodes[k].coor</code>	k-ième sommet pour les polygones ($k \geq 1$)

Pourquoi `nodes[1]` ? pylmgc90 numérote les nœuds à partir de 1. LMGC90_GUI supporte les deux conventions : `nodes[0]` et `nodes[1]` pointent tous les deux vers le nœud principal.

Itération sur les avatars

```
nb = len(avatar) # Nombre total d'avatars
liste = list(avatar) # Tous les avatars comme liste de proxies
centres = [av.center for av in avatar] # Liste de tous les centres
rayons = [av.radius for av in avatar if av.radius is not None]
```

Groupes — `group['nom']`

Accès à un groupe d'avatars par son nom. Retourne une liste d'`AvatarProxy`.

Expression	Description
<code>group['mur_briques']</code>	Liste des avatars du groupe
<code>group['mur_briques'][0].center</code>	Centre du premier avatar du groupe
<code>group['mur_briques'][0].x</code>	Coordonnée X du premier avatar
<code>len(group['mur_briques'])</code>	Nombre d'avatars dans le groupe
<code>'mur_briques' in group</code>	Test d'existence du groupe (bool)
<code>list(group)</code>	Liste de tous les noms de groupes

Exemples :

```
nb_briques = len(group['mur_facade'])
premier_x = group['mur_facade'][0].x
dernier_y = group['mur_facade'][-1].y
all_centers = [av.center for av in group['granulo_box2d']]
```

Matériaux — `material['nom']`

Expression	Description	Type
<code>material['beton'].name</code>	Nom du matériau	<code>str</code>
<code>material['beton'].density</code>	Densité (kg/m³)	<code>float</code>
<code>material['beton'].material_type</code>	Type ('RIGID', 'ELAS'...)	<code>str</code>
<code>material['beton']['young']</code>	Propriété personnalisée (module de Young)	<code>float</code>
<code>material['beton']['nu']</code>	Propriété personnalisée (coefficient de Poisson)	<code>float</code>
<code>material['beton'].young</code>	Idem via attribut	<code>float</code>

Exemples :

```
rho = material['granite'].density
E = material['acier']['young']
nu = material['acier'].nu
```

Modèles — `model['nom']`

Expression	Description	Type
<code>model['rigid'].name</code>	Nom du modèle	<code>str</code>
<code>model['rigid'].physics</code>	Physique ('MECAX', 'THERx'...)	<code>str</code>
<code>model['rigid'].element</code>	Élément fini ('Rxx2D', 'Q4xxx'...)	<code>str</code>
<code>model['rigid'].dimension</code>	Dimension (2 ou 3)	<code>int</code>
<code>model['femxx']['kinematic']</code>	Option numérique	dépend de l'option

Exemple :

```
dim = model['femxx'].dimension
phys = model['rigid'].physics
```

Fonctions de filtrage

Retournent une liste d'`AvatarProxy` filtrée selon un critère.

Fonction	Description	Exemple
<code>avatars_by_color('BLUEx')</code>	Tous les avatars de couleur 'BLUEx'	<code>bleus = avatars_by_color('BLUEx')</code>
<code>avatars_by_material('beton')</code>	Tous les avatars utilisant ce matériau	<code>corps = avatars_by_material('TDURx')</code>

Fonction	Description	Exemple
<code>avatars_by_type('rigidDisk')</code>	Tous les avatars de ce type pylmgc90	<code>disques = avatars_by_type('rigidDisk')</code>
<code>avatars_by_origin('manual')</code>	Avatars créés manuellement	<code>manuels = avatars_by_origin('manual')</code>
<code>avatars_by_origin('loop')</code>	Avatars générés par boucle	
<code>avatars_by_origin('granulo')</code>	Avatars granulaires	

Exemples avancés :

```
nb_bleus = len(avatars_by_color('BLUEx'))
nb_disques_rouges = len([av for av in avatars_by_color('REDxx') if av.avatar_type == 'rigidDisk'])
rayon_moyen = sum(av.radius for av in avatars_by_type('rigidDisk')) / len(avatars_by_type('rigidDisk'))
x_min = min(av.x for av in avatars_by_color('BLUEx'))
x_max = max(av.x for av in avatars_by_color('BLUEx'))
```

Utilisation par onglet

Les variables dynamiques peuvent être utilisées dans **tous les champs** des onglets de *LMGC90_GUI*. Il suffit d'écrire le nom de la variable (ou une expression complète) à la place de la valeur numérique.

Onglet Matériau

Champ	Accepte des expressions
Densité	■ — ex : <code>material['granite'].density * 0.9</code>
Propriétés (<code>key=val</code> , <code>key=val</code>)	■ — ex : <code>young=E</code> , <code>nu=nu_beton</code>

Exemples :

```
Densité : rho
Propriétés : young=E, nu=nu, elas='standard', anisotropy='isotropic'
```

Onglet Avatar vide (Empty Avatar)

Champ	Accepte des expressions
Centre	■ — ex : <code>avatar[0].x + spacing</code> , <code>avatar[0].y</code>
Paramètres des contacteurs	■ — ex : <code>r=radius</code> , <code>axe1=lx/2</code> , <code>axe2=ly/2</code>

Exemples :

```
Centre : avatar[0].x + spacing, 0.0
Paramètres (DISKx) : r=radius
Paramètres (JONCx) : axe1=brick_lx/2, axe2=brick_ly/2
Paramètres (POLYG) : nb_vertices=6, vertices=sommets
```

Onglet Contact

Tous les **paramètres numériques** de toutes les lois de contact acceptent des expressions :

Paramètre	Exemple d'expression
<code>fric</code>	<code>mu</code> ou <code>0.5 * material['beton'].density / 2500</code>
<code>stfr</code> , <code>dyfr</code>	<code>E * 1e3</code>

Paramètre	Exemple d'expression
cohn, coht	cohesion_sol
cn, ct	resistance_traction
stiffness	EA_cable

Exemples :

```
fric : mu
stiffness : EA_cable
cn : sigma_c
```

Onglet Boucles — Boucles **for** génériques

Dans les boucles **for** génériques, **toutes les expressions du template** ont accès aux variables dynamiques. Les bornes (**début**, **fin**, **pas**) et tous les champs du template sont évalués avec le contexte complet :

```
Début : 0
Fin : nb_cols
Pas : 1
center (template avatar) : [i * (lx + joint) + lx/2, offset_y]
radius (template avatar) : r_min + i * (r_max - r_min) / nb_cols
name (template matériau): 'mat' + str(i)
```

Onglet Boucles — Boucles géométriques

Les paramètres des boucles géométriques (Cercle, Grille, Ligne, Spirale) peuvent utiliser les variables dynamiques via le formulaire des boucles **for** :

```
radius (Cercle) : rayon_anneau
step (Grille) : pas_grille
```

Assistants — Wizard déformable (page DOF)

Dans la page Conditions aux Limites de l'assistant déformable, le champ Paramètres utilise `SafeEvaluator.eval_dict()` :

```
component=[1,2], dofty="vlocy", ct=vitesse_imposee
component=[2], ct=deplacement / duree_rampe
```

Expressions avancées supportées

Opérateurs

Opérateur	Exemple
+, -, *, /	lx + joint
//	Division entière
%	Modulo
**	Puissance : radius**2
==, !=, <, <=, >, >=	Comparaisons
and, or, not	Logique

Expressions ternaires

```
r = r_grand if grand else r_petit  
val = x if x > 0 else 0.0
```

Compréhensions de listes

```
centres = [av.center for av in group['mur']]  
rayons = [av.radius for av in avatars_by_type('rigidDisk')]  
xs = [av.x for av in avatar]
```

Listes et tuples

```
center_2d = [avatar[0].x, avatar[0].y]  
center_3d = [0.0, 0.0, avatar[0].z + 0.5]
```

Expressions numpy

```
moyenne = np.array([av.x for av in avatar]).mean()  
coords = np.array([av.center for av in avatar])
```

Comparaison avec un script Python équivalent

Le tableau suivant montre l'équivalence entre une variable dynamique dans l'interface et son code Python en script de pré-traitement.

Variable dynamique (interface)	Équivalent script Python
<code>r = avatar[0].radius</code>	<code>r = bodies[0].nodes[1].coor[...]</code>
<code>cx = avatar[0].x</code>	<code>cx = bodies[0].nodes[1].coor[0]</code>
<code>cy = avatar[1].center[1]</code>	<code>cy = bodies[1].nodes[1].coor[1]</code>
<code>cx = avatar[0].nodes[1].coor[0]</code>	<code>cx = bodies[0].nodes[1].coor[0]</code>
<code>rho = material['beton'].density</code>	<code>rho = mats['beton'].density</code>
<code>E = material['acier']['young']</code>	<code>E = mats['acier'].young</code>
<code>phys = model['rigid'].physics</code>	<code>phys = mods['rigid'].physics</code>
<code>nb = len(avatar)</code>	<code>nb = len(bodies)</code>
<code>nb_g = len(group['mur'])</code>	<code>nb_g = len(group_mur)</code>
<code>x0 = group['mur'][0].x</code>	<code>x0 = bodies[group_mur[0]].nodes[1].coor[0]</code>
<code>bleus = avatars_by_color('BLUEX')</code>	<code>bleus = [b for b in bodies_list if b.color == 'BLUEX']</code>

Exemple complet

Interface — Variables dynamiques :

```
lx = 0.20  
ly = 0.065  
joint = 0.010  
nb_cols = 15  
nb_rows = 10  
spacing = lx + joint  
offset_x = 0.0
```

```
wall_width = nb_cols * spacing
```

Équivalent script Python pré-traitement :

```
lx = 0.20
ly = 0.065
joint = 0.010
nb_cols = 15
nb_rows = 10
spacing = lx + joint
offset_x = 0.0
wall_width = nb_cols * spacing

for row in range(nb_rows):
    for col in range(nb_cols):
        cx = offset_x + col * spacing + lx / 2.0
        cy = row * (ly + joint) + ly / 2.0
        body = pre.rigidDisk(center=[cx, cy], ...)
        bodies.addAvatar(body)
```

Propriétés de l'objet `AvatarProxy` — référence complète

Le tableau suivant liste **toutes les propriétés** accessibles via `avatar[i]` ou dans une liste de groupe.

Propriété	Type	Notes
<code>.center</code>	<code>list[float]</code>	<code>[x, y]</code> en 2D, <code>[x, y, z]</code> en 3D
<code>.x</code>	<code>float</code>	<code>center[0]</code>
<code>.y</code>	<code>float</code>	<code>center[1]</code>
<code>.z</code>	<code>float</code> ou <code>None</code>	<code>center[2]</code> ou <code>None</code> si 2D
<code>.radius</code>	<code>float</code> ou <code>None</code>	Rayon du disque/sphère
<code>.color</code>	<code>str</code>	Code couleur 5 car. ('BLUEx')
<code>.material_name</code>	<code>str</code>	Nom du matériau associé
<code>.model_name</code>	<code>str</code>	Nom du modèle associé
<code>.avatar_type</code>	<code>str</code>	Valeur enum : 'rigidDisk', 'rigidSphere', 'rigidJonc', 'emptyAvatar'...
<code>.origin</code>	<code>str</code>	'manual', 'loop', 'granulo'
<code>.generation_type</code>	<code>str</code> ou <code>None</code>	'regular', 'full', 'bevel'
<code>.is_hollow</code>	<code>bool</code>	Disque creux si <code>True</code>
<code>.nb_vertices</code>	<code>int</code> ou <code>None</code>	Nombre de sommets du polygone
<code>.vertices</code>	<code>list</code> ou <code>None</code>	Coordonnées des sommets <code>[[x1,y1], ...]</code>
<code>.axis</code>	<code>dict</code> ou <code>None</code>	<code>{'axe1': v, 'axe2': v, 'axe3': v}</code> pour joncs/plans
<code>.contactors</code>	<code>list[dict]</code>	Chaque dict : <code>{'shape', 'color', 'params'}</code>
<code>.wall_params</code>	<code>dict</code>	Paramètres de mur maçonnerie
<code>.brick_lx</code>	<code>float</code> ou <code>None</code>	Longueur brique (<code>wall_params['l']</code>)
<code>.brick_ly</code>	<code>float</code> ou <code>None</code>	Hauteur/profondeur brique (<code>wall_params['h']</code>)

Propriété	Type	Notes
<code>.brick_lz</code>	<code>float</code> ou <code>None</code>	Hauteur 3D brique (<code>wall_params['lz']</code>)
<code>.mesh_params</code>	<code>dict</code> ou <code>None</code>	Paramètres maillage EF
<code>.index</code>	<code>int</code>	Index dans <code>state.avatars</code>
<code>.nodes[1].coor</code>	<code>list[float]</code>	Nœud principal (convention pylmgc90)
<code>.nodes[0].coor</code>	<code>list[float]</code>	Idem (convention Python)
<code>.nodes[k].coor</code>	<code>list[float]</code>	k-ième sommet (polygones)

Sécurité — ce qui est autorisé et interdit

Les expressions sont analysées via `SafeEvaluator` qui inspecte l'AST Python avant exécution. Aucun `eval()` direct n'est utilisé.

Autorisé

- Opérations arithmétiques, comparaisons, logique
- Appels de fonctions du contexte (`sqrt`, `math.sin`, `len`, `list`...)
- Compréhensions de listes, tuples, dicts
- Expressions ternaires (`a if cond else b`)
- Accès aux attributs et index
- Conversions : `int(x)`, `float(x)`, `str(x)`, `bool(x)`

Interdit (bloqué par SafeEvaluator)

- `import`, `exec`, `eval`, `open`, `__import__`
- Toute instruction (assignment, boucle `for/while`, `if` — seules les expressions sont autorisées)
- Appels de fonctions non déclarées dans le contexte
- Accès à `__builtins__`, `__class__`, `__dict__`

Si une expression tente une opération interdite, `SafeEvaluator` lève une `ValueError` avec le message "Opération non autorisée : <NomDuNœudAST>".

Messages d'erreur et débogage

Lorsqu'une expression échoue dans un onglet, une boîte de dialogue détaillée liste :

- L'expression fautive
- Le message d'erreur Python
- Les variables dynamiques actuellement définies
- Les références disponibles (`avatar[i].x`, `group['nom']`, etc.)

Causes fréquentes :

Erreur	Cause	Solution
<code>NameError: 'thickness'</code>	Variable pas encore définie	Créer <code>thickness</code> dans le gestionnaire
<code>IndexError: Avatar index 5 invalide</code>	Il y a moins de 6 avatars	Vérifier l'index

Erreur	Cause	Solution
<code>KeyError: Groupe 'mur' introuvable</code>	Le groupe n'existe pas	Vérifier le nom dans l'onglet Boucles/Maçonnerie
<code>Opération non autorisée : Import</code>	Tentative d'import	Utiliser uniquement les fonctions disponibles
<code>Syntaxe invalide</code>	Parenthèses mal fermées	Vérifier la syntaxe Python

Exemples de variables dynamiques prêtes à l'emploi

Géométrie de mur maçonné

```
lx = 0.20 # Longueur brique
ly = 0.065 # Hauteur brique
lz = 0.10 # Profondeur brique (3D)
joint = 0.010 # Épaisseur joint
nb_cols = 15 # Nombre de colonnes
nb_rows = 10 # Nombre de rangs
spacing_x = lx + joint
spacing_y = ly + joint
wall_width = nb_cols * spacing_x
wall_height = nb_rows * spacing_y
```

Paramètres granulaires

```
r_min = 0.05
r_max = 0.15
ratio = r_max / r_min
lx_box = 4.0
ly_box = 4.0
nb_particules = 200
```

Coordonnées relatives à un avatar existant

```
x_ref = avatar[0].x
y_ref = avatar[0].y
x_cible = x_ref + 1.0
y_cible = y_ref + 0.5
dist = sqrt((avatar[1].x - avatar[0].x)**2 + (avatar[1].y - avatar[0].y)**2)
```

Paramètres matériaux

```
rho = material['granite'].density
E = material['acier']['young']
nu = material['acier']['nu']
mu = 0.3 # Coefficient de Coulomb
K = E / (3 * (1 - 2 * nu)) # Module de compressibilité
G = E / (2 * (1 + nu)) # Module de cisaillement
```

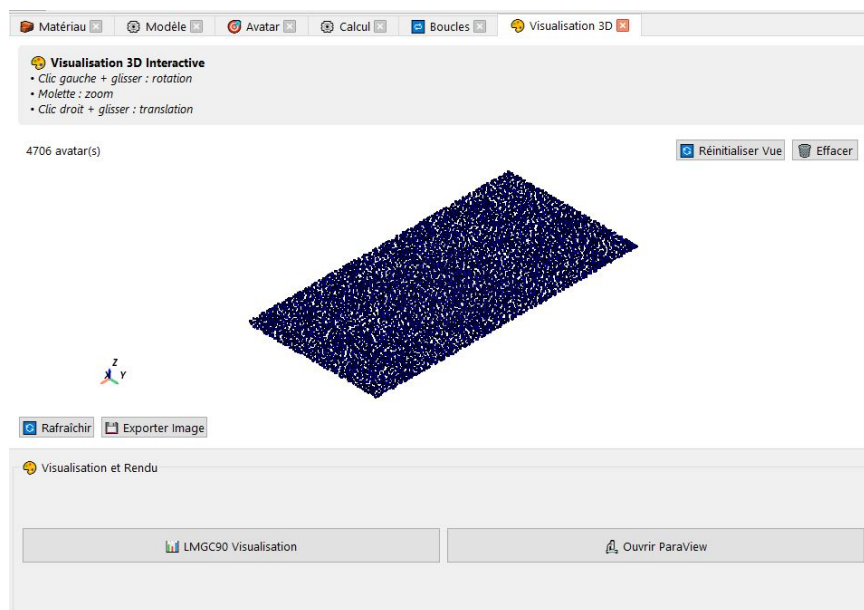
Statistiques sur les avatars

```
nb_total = len(avatar)
nb_granulo = len(avatars_by_origin('granulo'))
nb_manuels = len(avatars_by_origin('manual'))
```

```
rayon_moyen = sum(av.radius for av in avatars_by_type('rigidDisk')) / max(1, nb_total)
x_centre_masse = sum(av.x for av in avatar) / max(1, nb_total)
```

Visualisation

On plus de l'outil de visualisation intégré avec LMGC90 qui n'est d'autre que la fonction du préprocesseur `pre.visuAvatars()`, LMGC90_GUI permet de visualiser vos avatars dans un espace 3D complètement indépendant du solveur, simplement il est dans sa version *bêta* donc il est très gourmand en ressources.



captures/visual.JPG

Version 0.4.0 — Interface graphique pour le code de simulation mécanique LMGC90, l'architecture actuelle va être remplacée dans la version 0.5.0.

1. Vue d'ensemble
2. Structure des fichiers
3. Couche Core (Modèle)
4. Couche Controllers
5. Couche GUI / Views
6. Couche Utils
7. Flux de données
8. Systèmes clés expliqués
9. Cycle de vie d'un projet
10. Conventions et patterns
11. Guide de contribution

LMGC90_GUI est une application de bureau **PyQt6** suivant le pattern **MVC** (Model-View-Controller). Elle permet de créer des simulations mécaniques LMG90 via une interface graphique sans écrire de code Python manuellement.



Dépendances principales

Dépendance	Rôle
PyQt6	Framework UI (widgets, signaux/slots, threads)
pymg90	Bibliothèque LMGC90 (pré-traitement, simulation)
numpy	Calculs vectorisés (granulométrie, positions)
pyvista / pyvistaqt	Visualisation 3D des avatars
gmsh (optionnel)	Maillage de géométries pour corps déformables

2. Structure des fichiers

```
lmgc90_gui/
├── main.py # Point d'entrée, QApplication
├── src/
│   ├── core/ # Couche Modèle (MVC)
│   │   ├── models.py # Dataclasses : Material, Model, Avatar, ...
│   │   ├── validators.py # Validation des données
│   │   ├── generators.py # LoopGenerator, GranuloGenerator
│   │   ├── serializers.py # Sauvegarde/chargement JSON (.lmgc90)
│   │   ├── pymg90_bridge.py # Conversion modèles → objets pymg90
│   │   ├── particle_factory.py # Moteur de génération progressive
│   │   ├── avatar_factory.py # Templates d'avatars prédéfinis
│   │   ├── app_logger.py # Logger applicatif
│   │   └── workers/
│   │       └── granulo_worker.py # QThread pour génération granulo
│   ├── controllers/
│   │   └── project_controller.py # Contrôleur central (logique métier)
│   └── views/ # Couche Vue (MVC)
│       ├── main_window.py # Fenêtre principale (QMainWindow)
│       ├── tree_view.py # Arbre du modèle (QTreeWidget)
│       ├── tabs/ # Onglets de travail
│       │   ├── base_tab.py # Classe de base avec safe_eval
│       │   ├── material_tab.py # Gestion matériaux
│       │   ├── model_tab.py # Gestion modèles EF
│       │   ├── avatar_tab.py # Gestion avatars standards
│       │   ├── empty_avatar_tab.py # Avatars vides (contacteurs manuels)
│       │   ├── loop_tab.py # Boucles de génération
│       │   ├── granulo_tab.py # Génération granulométrique
│       │   ├── dof_tab.py # Conditions aux limites DOF
│       │   ├── contact_tab.py # Lois de contact
│       │   ├── visibility_tab.py # Tables de visibilité
│       │   ├── postpro_tab.py # Post-traitement
│       │   └── viewer_tab.py # Wrapper onglet visualisation 3D
│       └── ...
└── gui/
    ├── dialogs/ # Dialogues et assistants
    │   ├── dialogs.py # DynamicVarsDialog, PreferencesDialog, DuplicateDialog
    │   ├── setup_wizard.py # Assistant de projet
    │   ├── factory_wizard.py # Assistant Particle Factory (+ FactoryTab)
    │   ├── granulo_wizard.py # Assistant granulométrie
    │   ├── mesh_wiz_def.py # Assistant corps déformables (FEM)
    │   ├── masonry_wizard.py # Assistant maçonnerie
    │   └── fast_granulo_dialog.py # Dialog génération rapide numpy
```

```

■ ■ ■■ viewer_3d.py # Widget PyVista (visualisation 3D)
■ ■ ■■ chipy_routines_dialog.py # Config routines chipy
■ ■ ■■ app_log_dialog.py # Visualisation des logs
■ ■ ■■ convert_dialog.py # Conversion scripts pylmgc90
■ ■
■ ■■ utils/
■ ■■ safe_eval.py # Évaluateur sécurisé + proxies contexte projet
■ ■■ script_generator.py # Génération script pre.py
■ ■■ compute_script_generator.py # Génération script chipy (command.py)
■ ■■ fast_granulo_engin.py # Moteur granulo numpy haute perf.
■ ■■ convert.py # Convertisseur script pylmgc90 → .lmgc90

```

3. Couche Core (Modèle)

3.1 models.py — Les dataclasses

Toutes les entités du projet sont des **dataclasses Python**. Elles sont la source de vérité.

```

# Hiérarchie des modèles
ProjectState
■■■ List[Material] # Matériaux (RIGID, ELAS, ...)
■■■ List[Model] # Modèles EF (Rxx2D, T3xxx, ...)
■■■ List[Avatar] # Corps rigides/déformables
■■■ List[ContactLaw] # Lois de contact (IQS_CLB, ...)
■■■ List[VisibilityRule] # Tables de visibilité
■■■ List[DOFOperation] # Conditions aux limites
■■■ List[Loop] # Boucles géométriques (cercle, grille...)
■■■ List[ForLoop] # Boucles for génériques (JSON template)
■■■ List[GranuloGeneration] # Dépôts granulométriques
■■■ List[PostProCommand] # Commandes post-traitement
■■■ Dict[str, List[int]] # Groupes d'avatars (nom → indices)
■■■ Dict[str, Any] # Variables dynamiques (expressions)
■■■ List[dict] # Factories (FactoryConfig sérialisés)
■■■ ProjectPreferences # Préférences utilisateur

```

Chaque **dataclass** expose `to_dict()` et `from_dict()` pour la sérialisation JSON. Exemple :

```

@dataclass
class Avatar:
    avatar_type: AvatarType # Enum : rigidDisk, roughWall, ...
    center: List[float]
    material_name: str
    model_name: str
    color: str = "BLUEx"
    origin: AvatarOrigin = AvatarOrigin.MANUAL
    radius: Optional[float] = None
    # ... autres champs spécifiques au type

```

Enums importants :

Enum	Valeurs notables
AvatarType	RIGID_DISK, ROUGH_WALL, MESH_DEFORMABLE, EMPTY_AVATAR, ...
AvatarOrigin	MANUAL, LOOP, GRANULO
MaterialType	RIGID, ELAS, ELAS_PLAS, ...
ContactLawType	IQS_CLB, MAC_CZM, ELASTIC_WIRE, ...

Enum	Valeurs notables
UnitSystem	SI, CGS

3.2 validators.py — Validation

Chaque entité a son validateur :

```
MaterialValidator.validate_or_raise(material) # Nom ≤ 5 chars, densité > 0
ModelValidator.validate_or_raise(model) # Élément compatible physique+dim
AvatarValidator.validate_or_raise/avatar, model) # Paramètres selon type
ContactLawValidator.validate_or_raise(law) # Propriétés obligatoires
```

Les validateurs lèvent `ValidationError` (hérite de `Exception`) capturée par la Vue pour afficher un `QMessageBox`.

3.3 pylmgc_bridge.py — Pont vers pylmgc90

`LMGC90Bridge` est une classe **statique** qui convertit les dataclasses en objets `pylmgc90.pre` :

```
LMGC90Bridge.create_material(material) → pre.material(...)
LMGC90Bridge.create_model(model) → pre.model(...)
LMGC90Bridge.create_avatar/avatar, mod, mat) → pre.rigidDisk(...) / pre.avatar(...)
LMGC90Bridge.create_contact_law(law) → pre.tact_behav(...)
LMGC90Bridge.create_visibility_rule(rule, b) → pre.see_table(...)
LMGC90Bridge.apply_dof_operation(op, body) → body.translate(...) / body.imposeDrivenDof(...)
```

Cas complexes gérés :

- Corps déformables : `pre.buildMesh2D`, `pre.buildMeshH8`, `pre.readMesh` + `pre.buildMeshedAvatar`
- Briques de maçonnerie : `pre.brick2D/3D` + `brick.rigidBrick(...)`
- Avatars vides avec contacteurs : création pas-à-pas via `pre.avatar()` → `addBulk` → `addNode` → `addContactors`

3.4 generators.py — Génération de positions

```
LoopGenerator.generate_positions(loop: Loop) → List[[x, y]]
# Dispatche vers : generate_circle / generate_grid / generate_line / generate_spiral

GranuloGenerator.generate(config) → (nb_particles, coordinates_array, radii_array)
# Appelle pre.granulo_Random puis pre.depositInBox2D / depositInDisk2D / ...
```

3.5 particle_factory.py — Particle Factory

Système de génération **progressive** de particules (inspiré d'EDEM) :

- `FactoryConfig` : dataclass complète (type, zone, planning, conteneur)
- `ParticleFactory` : moteur (valide, assigne les indices corps, génère le code)
- `PreCodeGenerator` : génère le bloc `pre.py` (création invisible + planning)
- `ChipyCodeGenerator` : génère le bloc `chipy.py` (activation par vagues)

3.6 serializers.py

```
ProjectSerializer.save(state, filepath) # JSON → fichier .lmgc90
ProjectSerializer.load(filepath) # fichier .lmgc90 → ProjectState
```

Le format `.lmgc90` est du **JSON pur**. Seuls les avatars `origin == MANUAL` sont sérialisés ; les avatars générés (boucles, granulo) sont régénérés au chargement.

4. Couche Controllers

4.1 `project_controller.py` — Le contrôleur central

`ProjectController(QObject)` est le **cœur de l'application**. Il :

1. Maintient l'état du projet (`self.state: ProjectState`)
2. Maintient les objets `pylmgc90` en mémoire :

```
self._materials_container # pre.materials()
self._models_container # pre.models()
self._bodies_container # pre.avatars()
self._contact_laws_container # pre.tact_behavs()
self._visibility_container # pre.see_tables()
self._postpro_container # pre.postpro_commands()

self._pylmgc_materials: Dict[str, Any] # nom → objet pylmgc90
self._pylmgc_models: Dict[str, Any]
self._pylmgc_bodies: List[Any] # indexé comme state.avatars
self._pylmgc_laws: Dict[str, Any]
```

3. Émet `state_changed = pyqtSignal()` à chaque modification

Invariant fondamental : `self._pylmgc_bodies[i]` correspond toujours à `self.state.avatars[i]`.

API du contrôleur (méthodes principales)

```
# Projet
controller.new_project(name)
controller.save_project(filepath?)
controller.load_project(filepath) # → reconstruit tout via _rebuild_pylmgc_objects()

# CRUD Matériaux
controller.add_material(material) # valide + crée objet pylmgc + state
controller.update_material(old_name, m) # met à jour refs dans avatars
controller.remove_material(name)

# CRUD Modèles (identique)
# CRUD Avatars
controller.add_avatar/avatar, create_pylmgc=True) # create_pylmgc=False pour perf
controller.update_avatar(index, avatar)
controller.remove_avatar(index)
controller.duplicate_avatar(index, n, offset, group?)
controller.duplicate_group(group_name, n, offset, prefix?)

# Génération
controller.generate_loop(loop) # → crée avatars + ajoute au groupe
controller.generate_granulo(config) # → via GranuloGenerator
controller.generate_for_loop(for_loop) # → boucle for générique

# DATBOX
controller.generate_datbox(output_path) # → pre.writeDatbox(...)
```

`_rebuild_pylmgc_objects()` — Reconstruction au chargement

Au chargement d'un projet, l'ordre est **strict** :

1. Matériaux → modèles → avatars MANUAL → boucles → granulo → boucles For
2. Lois de contact → visibilité → DOF

Si une erreur survient (ex : matériau manquant), elle est stockée dans `state.load_warnings` et affichée dans l'UI.

Mode batch

```
self._batch_mode = True # Désactive state_changed.emit() pendant la création
# ... créer N avatars ...
self._batch_mode = False
self.state_changed.emit() # Un seul signal à la fin
```

5. Couche GUI / Views

5.1 `main_window.py` — Fenêtre principale

`MainWindow(QMainWindow)` orchestre tout :

```
MainWindow
■■■ MenuBar # Fichier, Assistants, Outils, Calcul, Onglets, Aide
■■■ ToolBar # Nouveau, Ouvrir, Sauvegarder, DATBOX, Script
■■■ DockWidget (gauche)
■ ■■■ ModelTreeView # Arbre QTreeWidget
■■■ Central (QSplitter vertical)
■■■ QTabWidget # Onglets de travail (70%)
■ ■■■ MaterialTab
■ ■■■ ModelTab
■ ■■■ AvatarTab
■ ■■■ ...
■■■ QWidget (bas 30%) # Boutons LMGC90 Viz + ParaView
```

Gestion des onglets : Les onglets peuvent être ouverts/fermés dynamiquement. `material_tab` et `model_tab` sont **essentiels** (non fermables). Chaque onglet est instancié une seule fois et caché/montré.

Connexion des signaux :

```
# Chaque onglet émet des signaux → MainWindow._refresh_all()
self.material_tab.material_created.connect(self._refresh_all)
# _refresh_all() appelle tree_view.refresh() + tab.refresh() sur tous les onglets
```

5.2 `tree_view.py` — Arbre du modèle

`ModelTreeView(QObject)` gère un `QTreeWidget` affichant la structure complète :

```
Modèle LMGC90
■■■ Matériaux (N)
■■■ Modèles (N)
■■■ Avatars (N) [filtrés selon préférence show_granulo_individually]
■■■ Groupes d'avatars
■■■ Lois de contact
■■■ Tables de visibilité
■■■ Opérations DOF
■■■ Boucles
■■■ Dépôts Granulo
■■■ Post-Processing
```

Signal émis : `item_selected = pyqtSignal(str, object)` → type d'élément + données. `MainWindow` reçoit ce signal et charge l'élément dans l'onglet approprié.

Menu contextuel : Clic droit sur Avatar → `DuplicateDialog`. Clic droit sur Groupe → duplication du groupe entier.

5.3 `base_tab.py` — Classe de base des onglets

Tous les onglets héritent de `BaseTab(QWidget)`. Elle fournit :

```
# Évaluation sécurisée d'expressions (utilise SafeEvaluator)
self.eval_float(text, default, field_name) # "0.5 * pi" → 1.5707...
self.eval_int(text, default, field_name)
self.eval_list(text, expected_length, field_name) # "1.0, 2.0" → [1.0, 2.0]
self.eval_dict(text, field_name) # "k=1, nu=0.3" → {"k": 1, "nu": 0.3}

# Label d'aide contextuelle
self.add_expression_help_label(layout)
```

L'évaluateur donne accès au **contexte projet complet** dans les champs de formulaire : `avatar[0].x`, `group['mur'][0].radius`, `material['acier'].density`, etc.

5.4 Les onglets (tabs/)

Chaque onglet suit le même **pattern CRUD** :

```
Tab
■■■ QTreeWidget # Liste des éléments existants
■■■ Boutons (=■ Modifier, ■ Supprimer)
■■■ QFormLayout # Formulaire de création/édition
■■■ Boutons (■ Créer, ■ Enregistrer, ■ Annuler, ■ Réinitialiser)
■■■ Signaux émis # element_created, element_updated, element_deleted
```

Méthodes à implémenter dans chaque onglet :

```
def _setup_ui(self) # Construction de l'interface
def _connect_signals(self) # Connexion signaux/slots
def _on_create(self) # Créer un élément
def _on_edit_from_tree(self) # Charger pour édition depuis l'arbre
def _on_update(self) # Enregistrer les modifications
def _on_delete(self) # Supprimer
def load_for_edit(self, ...) # Remplir le formulaire depuis un objet
def refresh(self) # Rafraîchir l'affichage
```

`avatar_tab.py` — Onglet Avatar

Gère 18+ types d'avatars 2D/3D. La méthode `_on_type_changed()` affiche/masque dynamiquement les champs selon le type sélectionné. `_build_avatar_from_form()` construit l'objet `Avatar` depuis les champs visibles.

`model_tab.py` — Onglet Modèle

Gestion complexe des options selon la physique (MECAx/THERx/POROX/MULTI) et l'élément. `_on_element_changed()` reconstruit dynamiquement les combos d'options.

`granulo_tab.py` — Génération granulométrique

Utilise un `QThread` (`GranuloWorker`) pour les calculs. La création des avatars se fait par **batches progressifs** via un `QTimer` pour ne pas bloquer l'UI :

```

_on_generate() → GranuloWorker.run() → data_ready signal
↓
_on_data_ready() → QTimer(0ms)
↓
_create_next_avatar() × N [batches de 50-100]
↓
_on_creation_completed()

```

5.5 viewer_3d.py — Visualisation 3D

Viewer3D(QWidget) wrapant **pyvistaqt.QtInteractor**. Ne se rafraîchit **jamais automatiquement** pour éviter les freezes. L'utilisateur clique "■ Rafraîchir la scène".

Construction des meshes :

```

build_avatar_mesh/avatar) → pv.PolyData
# Dispatche vers _MESH_BUILDERS[avatar.avatar_type]
# Ex: _mesh_rigid_disk → pv.Circle().extrude(h)
# _mesh_rigid_polygon → pv.PolyData(vertices)
# _mesh_deformable → reconstruit depuis mesh_params (geom)

```

Modes couleur : LMGC90 (par code couleur) | Par type | Par matériau | Par origine **Modes interaction :** Navigation | Sélection (avatar_clicked signal) | Règle (mesure distance)

5.6 Les Wizards (dialogs/)

Assistants **QWizard** multi-pages pour des tâches complexes :

Wizard	Pages	Résultat
ProjectSetupWizard	Projet → Dim → Mat → Mod → Avatar → Contact → Visibilité → Résumé	Projet complet initialisé
GranuloWizard	Distribution → Conteneur → Propriétés → Résumé	Dépôt granulo généré
MeshWizard	Intro → Dim → Mat → Mod → Géom → Raffin → Boundary → Résumé	Corps déformable FEM
MasonryWizard	Config → Modèle → Résumé	Mur de maçonnerie
FactoryWizard	Intro → Zone → Particules → Conteneur → Planning → Résumé	Factory configurée

6. Couche Utils

6.1 safe_eval.py — Évaluation sécurisée

Problème résolu : Permettre aux utilisateurs d'entrer des expressions Python (**avatar[0].x + 0.1, sqrt(2) * radius**) dans les formulaires sans risque de sécurité.

Architecture :

```

# 1. SafeEvaluator – vérification AST + eval isolé
ev = SafeEvaluator(allowed_names=context_dict)
result = ev.eval_expression("avatar[0].x * 2")

# 2. Proxies – accès aux données du projet

```

```

AvatarCollectionProxy # avatar[i], len(avatar), list(avatar)
AvatarProxy # .center, .x, .y, .z, .radius, .nodes[1].coor, ...
GroupProxy # group['nom'][0].center
MaterialProxy # material['acier'].density
ModelProxy # model['rigid'].physics

# 3. build_eval_context() – construit le contexte complet
ctx = build_eval_context(controller)
# ctx contient: math, np, avatar, group, material, model,
# avatars_by_color(), avatars_by_material(), ...
# + toutes les variables dynamiques du projet

```

Sécurité : `SafeEvaluator._check_safe()` parcourt l'AST et rejette tout nœud non autorisé (import, exec, attributs dangereux, etc.).

6.2 script_generator.py — Génération script pre.py

`ScriptGenerator(controller)` génère un script Python reproduisant le projet :

```

generate(output_path)
■■■■ _write_header() # Commentaire d'en-tête
■■■■ _write_imports() # from pylmgc90 import pre, numpy, math
■■■■ _write_dynamic_vars() # Évaluation et injection des vars dynamiques
■■■■ _write_containers() # mats, mods, bodies, tacts, sees, posts
■■■■ _write_materials() # mat_NOM = pre.material(...)
■■■■ _write_models() # mod_NOM = pre.model(...)
■■■■ _write_avatars_manual() # Avatars MANUAL (avec option boucle ou individuel)
■ ■■■■ _write_avatars_manual_loop() # Mode boucle (par groupes)
■ ■ ■■■■ _write_masonry_group_loop() # Boucles briques de maçonnerie
■ ■ ■■■■ _write_standard_group_loop() # Boucle for + liste de centers
■ ■■■■ _write_single_avatar() # Avatar individuel
■■■■ _write_for_loops() # Boucles For génériques
■■■■ _write_loops() # Boucles géométriques (cercle, grille...)
■■■■ _write_granulo() # pre.granulo_Random + depositIn*
■■■■ _write_contact_laws() # pre.tact_behav(...)
■■■■ _write_visibility() # pre.see_table(...)
■■■■ _write_dof_operations() # body.translate / imposeDrivenDof
■■■■ _write_postpro() # pre.postpro_command(...)
■■■■ _write_factories() # Particle Factories (code pré-calculé)
■■■■ _write_datbox() # pre.writeDatbox(...)

```

Préférence `script_use_loop` : Si activée, les avatars d'un même groupe homogène sont regroupés dans une boucle `for _c in _centers_<groupe>` : pour un script plus compact.

6.3 compute_script_generator.py — Génération script chipy

`ComputeScriptGenerator(controller)` génère `command.py` (boucle de simulation) selon les paramètres configurés dans `ChipyRoutinesDialog`.

Sections générées :

1. Configuration chipy (`SetDimension`, `ReadDatbox`, ...)
2. Initialisation des Particle Factories (invisibilité + planning)
3. Boucle principale `for k in range(nb_steps)`:
 - FreeVelocity RBDY2/RBDY3/FEM
 - Détection contact (détecteurs sélectionnés)
 - Résolution NLGS

- Activation des vagues de factory
- ComputeDof + UpdateStep
- Extraction (énergie, GBV, inspection)
- WriteOut + WriteDisplayFiles

4. Finalisation

6.4 fast_granulo_engin.py — Granulométrie numpy

Moteur **sans pylmgc90**, entièrement numpy, pour générer des milliers de particules sans bloquer l'UI :

```
GranuloFastEngine.generate(nb, rmin, rmax, container_type, ...)
# → Placement par batches (candidats → filtrage bbox → filtrage collisions vectorisé)
# → FastGranuloResult avec List[FastParticle]

GranuloFileWriter.write(result, output_dir)
# → Écrit DATBOX/BODIES.DAT directement (bypass pylmgc90)

GranuloStateIntegrator.integrate(result, controller)
# → Ajoute les avatars en batch unique dans controller.state
```

6.5 convert.py — Convertisseur de scripts

Convertit un script **pre.py** existant en fichier **.lmgc90** en **exécutant le script** avec un module **_MockPre** qui intercepte tous les appels **pre.*** :

```
class _MockPre: # Remplace pylmgc90.pre pendant l'exec
class _AvatarObj # Proxy d'un avatar rigide
class _MeshAvatarObj # Proxy d'un corps déformable
class _EmptyAvatarObj # Proxy d'un avatar vide
class _BrickObj # Proxy d'une brique de maçonnerie
class _WallObj # Proxy d'un mur de maçonnerie

class Converter:
run() # Exécute le script avec le mock
to_lmgc90_dict() # Construit le JSON projet

# CLI :
# python convert.py mon_script.py -o sortie.lmgc90
```

7. Flux de données

7.1 Création d'un avatar (exemple complet)

```
Utilisateur remplit AvatarTab → clique "■ Créer Avatar"
↓
AvatarTab._on_create()
→ _build_avatar_from_form() # Valeurs → Avatar dataclass
→ controller.add_avatar/avatar)
↓
ProjectController.add_avatar()
→ AvatarValidator.validate_or_raise/avatar, model) # Validation
→ mat_obj = self._pylmgc_materials[avatar.material_name]
→ mod_obj = self._pylmgc_models[avatar.model_name]
→ body_obj = LMG90Bridge.create_avatar/avatar, mod_obj, mat_obj)
```

```

→ self._bodies_container.addAvatar(body_obj)
→ self._pylmgc_bodies.append(body_obj)
→ self.state.avatars.append(avatar)
→ self.state_changed.emit() # si pas en batch_mode
↓
AvatarTab.avatar_created.emit()
↓
MainWindow._refresh_all()
→ tree_view.refresh() # Arbre mis à jour
→ [tous les onglets].refresh() # Combos mis à jour

```

7.2 Sauvegarde et chargement

```

Sauvegarde :
ProjectController.save_project()
→ ProjectSerializer.save(state, filepath)
→ state.to_dict() # Seulement avatars MANUAL
→ json.dump(data, file) # Fichier .lmgc90

Chargement :
ProjectController.load_project(filepath)
→ ProjectSerializer.load(filepath)
→ ProjectState.from_dict(data) # Dataclasses reconstruites
→ _rebuild_pylmgc_objects() # Reconstruction ordre strict
1. Matériaux + Modèles # Objets pylmgc90 recréés
2. Avatars MANUAL # bridge.create_avatar()
3. Boucles → generate_loop() # Avatars LOOP recréés
4. Granulo → generate_granulo() # Avatars GRANULO recréés
5. Boucles For → generate_for_loop()
6. Lois + Visibilité + DOF

```

8. Systèmes clés expliqués

8.1 Indexation des avatars

L'index d'un avatar dans `state.avatars` est son identifiant partout :

- `state.avatar_groups["mur"] = [0, 1, 2, 3]` → indices dans `state.avatars`
- `state.operations[0].target_value = 5` → avatar #5
- `self._pylmgc_bodies[5]` → objet pylmgc90 de l'avatar #5

■ ■ **Attention lors de la suppression** : `remove_avatar(index)` fait un `pop(index)` qui décale tous les indices suivants. Les groupes et opérations qui référencent des avatars d'index supérieur deviennent invalides. C'est une limitation connue qui nécessite une refonte future (IDs stables).

8.2 Variables dynamiques

Les variables dynamiques (`state.dynamic_vars`) sont des **expressions Python** :

```
{"thickness": "0.5", "radius": "thickness * 2 + 0.1", "x_wall": "avatar[0].x"}
```

Elles sont évaluées dans l'ordre de définition et injectées dans le contexte de `SafeEvaluator`. Ainsi, dans un formulaire, l'utilisateur peut écrire `radius` et obtenir la valeur calculée.

8.3 Système de préférences

`ProjectPreferences` est stocké dans `state.preferences` et sauvegardé avec le projet. Les préférences importantes :

Préférence	Impact
<code>show_granulo_individually</code>	Masque les avatars GRANULO dans l'arbre et les onglets
<code>create_pylmgc_on_generate</code>	Désactive la création pylmgc pendant génération massive
<code>script_use_loop</code>	Génère des boucles compactes dans le script pre.py
<code>auto_refresh_viewer</code>	(réservé) Rafraîchissement auto de la vue 3D

8.4 Particle Factory

Les factories sont persistées comme `List[dict]` dans `state.factories`. Au chargement et à la génération du script, `ParticleFactory.from_list_of_dicts()` reconstruit l'engine, recalcule les indices bodies, et génère les blocs de code.

9. Cycle de vie d'un projet

```
1. NOUVEAU PROJET
MainWindow._on_new_project()
→ controller.new_project(name)
→ _reset_containers() [conteneurs pylmgc vides]
→ state = ProjectState(name)

2. CONFIGURATION
Onglet Matériaux → add_material()
Onglet Modèles → add_model()
Onglet Avatars → add_avatar()
... (lois, visibilité, DOF, boucles, granulo)

3. SAUVEGARDE
Ctrl+S → save_project()
→ state.to_dict() → JSON

4. GÉNÉRATION DATBOX
Outils → DATBOX → controller.generate_datbox(path)
→ pre.writeDatbox(dim, mats, mods, bodies, tacts, sees, post)

5. GÉNÉRATION SCRIPT PRE.PY
Outils → Script Python → ScriptGenerator.generate(path)

6. GÉNÉRATION SCRIPT CHIPY (command.py)
Calcul → Générer Script → ComputeScriptGenerator.generate(path, params)

7. CALCUL
Calcul → Lancer → compute_tab.run_computation()
→ Exécute command.py dans un subprocess
→ Affiche les logs LMGC90

8. CHARGEMENT
Fichier → Ouvrir → controller.load_project(path)
→ ProjectState.from_dict() + _rebuild_pylmgc_objects()
```

10. Conventions et patterns

Conventions de nommage

Élément	Convention	Exemple
Classes	PascalCase	AvatarTab, LMGC90Bridge
Méthodes	snake_case	_on_create(), load_for_edit()
Slots Qt	Préfixe _on_	_on_type_changed()
Signaux Qt	Suffixe descriptif	avatar_created, state_changed
Méthodes privées	Préfixe _	_build_avatar_from_form()
Conteneurs pylmgc	Préfixe _pylmgc_	_pylmgc_materials

Pattern signal/slot

```
# Dans un onglet :
class AvatarTab(BaseTab):
    avatar_created = pyqtSignal() # Déclaration au niveau classe

    def _on_create(self):
        ...
        self.avatar_created.emit() # Émission après action réussie

# Dans MainWindow :
self.avatar_tab.avatar_created.connect(self._refresh_all)
```

Gestion des erreurs

```
try:
    # Code métier
    avatar = self._build_avatar_from_form()
    self.controller.add_avatar(avatar)
    # Succès
    QMessageBox.information(self, "Succès", "■ Avatar créé")
except ValidationError as e:
    QMessageBox.warning(self, "Validation", str(e))
except ValueError as e:
    QMessageBox.critical(self, "Erreur", f"Valeurs invalides : \n{e}")
except Exception as e:
    QMessageBox.critical(self, "Erreur", f"Création échouée : \n{e}")
```

11. Guide de contribution

Ajouter un nouveau type d'avatar

1. `models.py` : Ajouter la valeur dans `AvatarType`
2. `validators.py` : Ajouter la validation dans `AvatarValidator.validate()`
3. `pylmgc_bridge.py` : Ajouter le cas dans `LMGC90Bridge.create_avatar()`
4. `avatar_tab.py` :
 - Ajouter le type dans `AVATAR_TYPES_2D` ou `AVATAR_TYPES_3D`
 - Gérer l'affichage des champs dans `_on_type_changed()`
 - Construire l'avatar dans `_build_avatar_from_form()`
5. `viewer_3d.py` : Ajouter le builder de mesh dans `_MESH_BUILDERS`
6. `script_generator.py` : Gérer la génération dans `_write_single_avatar()`

Ajouter un nouvel onglet

1. Créer `src/views/tabs/mon_tab.py` héritant de `BaseTab`
2. Implémenter le pattern CRUD complet
3. Déclarer les signaux nécessaires
4. Importer dans `src/views/tabs/__init__.py`
5. Instancier dans `MainWindow._create_tabs()`
6. Ajouter dans `MainWindow.all_tabs`
7. Connecter les signaux dans `MainWindow._connect_signals()`

Ajouter une nouvelle loi de contact

1. `models.py` : Ajouter dans `ContactLawType` et la catégorie appropriée dans `CONTACT_LAW_CATEGORIES`
2. `validators.py` : Ajouter propriétés requises dans `ContactLawValidator._REQUIRED_PROPS`
3. `pymgc_bridge.py` : Ajouter le cas dans `create_contact_law()`
4. `contact_tab.py` : Ajouter les champs UI dans `_on_type_changed()` et `_build_law_from_form()`

Ajouter un paramètre de préférences

1. `models.py` : Ajouter le champ dans `ProjectPreferences` avec valeur par défaut
2. Ajouter dans `to_dict()` et `from_dict()`
3. `dialogs.py` : Ajouter le widget dans `PreferencesDialog._build_perf_tab()` (ou autre onglet)
4. Ajouter dans `_load_preferences()` et `get_preferences()`
5. Utiliser la préférence via `getattr(self.controller.state.preferences, 'ma_pref', default)`

Tests et débogage

- **Logger applicatif** : `from src.core.app_logger import get_logger; _log = get_logger('mon_module')`
- **Journal** : Calcul → Journal de l'application (F7)
- **Logs LMGC90** : Calcul → Voir Logs LMGC90 (F6)
- **Variables dynamiques** : Outils → Variables dynamiques (Ctrl+V)

Annexe — Fichiers de référence rapide

Besoin	Fichier
Ajouter/modifier un type de données	<code>src/core/models.py</code>
Modifier la validation	<code>src/core/validators.py</code>
Modifier l'appel pymgc90	<code>src/core/pymgc_bridge.py</code>
Ajouter une logique métier	<code>src/controllers/project_controller.py</code>
Modifier l'UI d'un onglet	<code>src/views/tabs/<nom>_tab.py</code>
Modifier la vue 3D	<code>src/gui/dialogs/viewer_3d.py</code>
Modifier le script pre.py généré	<code>src/utils/script_generator.py</code>
Modifier le script chipy généré	<code>src/utils/compute_script_generator.py</code>
Modifier l'évaluation des expressions	<code>src/utils/safe_eval.py</code>
Modifier la sérialisation JSON	<code>src/core/serializers.py</code> + <code>models.py</code>